Module 1: Introduction to R

307312 Applied Statistics | University of Petra

Module contents

Click a topic below, or use the PDF reader bookmarks.
Every page has links to contents and the previous/next topic.

Overview 2

Module 1: R and RStudio 3

Module 1: Basic Concepts in R 11

Module 1: Data Manipulation with dplyr 49

Module 1: Visualization with ggplot2 55

Module 1: Basic Concepts — Practice 87

Module 1: dplyr Practice 90

Module 1: ggplot2 Practice 91

Module 1: Introduction to R

Lessons 1–4 | Foundations for the course

Table of contents

Reduced teaching material 1
What this module covers 1
Lessons 1
Student exercises 1

Reduced teaching material

Use the concise slides during class and the extended lessons below for full explanations, worked R code, and practice. Both follow the same lesson sequence.

- Reduced teaching slides (PDF)

What this module covers

R setup, basic concepts, data manipulation, and ggplot2. The dplyr lesson and practical exercises use the checked textbook CSVs. Lesson, exercise, and solution numbers match.

Lessons

- 1. R and RStudio
- 2. Basic concepts in R
- 3. Data manipulation with dplyr
- 4. Visualization with ggplot2

Student exercises

- Basic concepts
- Data manipulation with dplyr
- Visualization with ggplot2

Return to the course home.

Module 1: R and RStudio

Lesson 1 | Getting started

Reduced teaching slides (PDF) · Module overview

This extended lesson provides the full explanation and reproducible examples behind the slides.

R is a powerful language and environment for statistical computing and graphics.

- It is widely used among statisticians and data analysts for developing statistical software and data analysis.
- R is open source and freely available under the GNU General Public License, making it an accessible tool for data analysis, statistical modeling, and graphical representation.

History of R

- R was created by Ross Ihaka and Robert Gentleman at the University of Auckland, New Zealand, in the early 1990s.
- It was conceived as an implementation of the S programming language combined with lexical scoping semantics inspired by Scheme.
- The name ‘R’ was partly derived from the first letters of the creators’ first names, as well as a play on the name of S.
- R’s official release to the public came in 1995, and since then, it has evolved with contributions from many statisticians and programmers worldwide.

Main Features of R

- **Comprehensive Statistical Analysis Tool:** R provides a wide array of statistical techniques including linear and nonlinear modeling, classical statistical tests, time-series analysis, classification, clustering, and more.
- **Graphical Capabilities:** R excels in creating quality plots and graphs, offering a vast number of techniques to visualize data, from basic charts to complex graphics.
- **Extensible:** One of R’s most powerful features is its package system, which allows users to create and share sets of functions, data, and compiled code. The Comprehensive R Archive Network (CRAN) hosts thousands of packages covering a wide range of statistical, graphical, and data manipulation techniques.
- **Programming Features:** R includes conditional statements, loops, user-defined recursive functions, and input and output facilities.
- **Data Handling and Storage:** R handles various data types, including vectors, matrices, arrays, data frames, and lists. It provides facilities for data manipulation, calculation, and storage.
- **Community and Support:** R benefits from a large and active community. Users can seek help and share knowledge through forums, mailing lists, blogs, and user-contributed documentation.

Advantages of R

- **Open Source and Free:** Being open-source, R is freely available, and users can inspect, modify, and distribute the source code.
- **Cross-Platform:** R runs on various operating systems including Windows, MacOS, and Linux.
- **Active Community:** The vibrant community supports users at all levels, contributing packages, tools, and documentation.
- **Integration:** R can be integrated with other programming languages (like C, C++, and Python), enabling the use of R in diverse environments and applications.

Data Science and R

- **Versatile Tool for Data Manipulation:** R provides powerful libraries (dplyr, tidyr) for data cleaning, transformation, and preparation.
- **Advanced Statistical Analysis:** With its origins in statistics, R excels at performing complex statistical computations essential for data science.
- **Data Visualization:** R's ggplot2 and other plotting packages offer superior data visualization capabilities, allowing for the creation of professional and informative graphics.

Career Opportunities with R

Careers that utilize R span across various industries, leveraging its powerful capabilities in data analysis, statistical modeling, visualization, and more. Here's a list of careers where proficiency in R is highly valued:

1. **Data Analyst:** Analyzes data to help businesses make informed decisions. Uses R for data cleaning, preparation, and visualization.
2. **Data Scientist:** Uses statistical models and machine learning algorithms to analyze complex datasets. R is crucial for data manipulation, analysis, and predictive modeling.
3. **Quantitative Analyst:** Works primarily in finance, using R to develop complex models that inform investment strategies, risk management, and financial forecasting.
4. **Statistician:** Applies mathematical and statistical techniques to solve real-world problems. Uses R for statistical testing, data analysis, and experiment design.
5. **Bioinformatician/Biostatistician:** Uses R in the analysis of biological data, such as genetic sequencing, drug development, and epidemiological studies.
6. **Market Research Analyst:** Analyzes market conditions to examine potential sales of a product or service. Uses R for survey analysis, consumer behavior studies, and trend forecasting.
7. **Machine Learning Engineer:** Develops algorithms that enable computers to learn from and make decisions based on data. R is used for prototyping and developing statistical models.
8. **Econometrician:** Specializes in econometrics, using R to analyze economic data, forecast economic trends, and develop economic models.
9. **Environmental Scientist:** Uses R for analyzing environmental data, modeling environmental processes, and evaluating the impacts of environmental policies.
10. **Public Health Analyst:** Utilizes R in the analysis of health data to inform public health policy, program decisions, and research on health trends.
11. **Educational Researcher:** Employs R for analyzing educational data, evaluating the effectiveness of educational programs, and researching teaching methods and outcomes.

12. **Actuary:** Uses R for analyzing risk in the insurance and finance industries, helping companies set policies' prices and provisions.
13. **Operations Research Analyst:** Applies mathematical and analytical methods to help organizations investigate complex issues, identify and solve problems, and make better decisions. R is used for optimization, simulation, and decision analysis.
14. **Sports Analyst:** Analyzes sports data to improve team performance, develop game strategies, and evaluate player performance. R is used for statistical analysis and predictive modeling in sports analytics.
15. **Clinical Researcher:** Utilizes R in the design, execution, and analysis of clinical trials, including statistical analysis to interpret study results.

These careers demonstrate the broad applicability of R across different sectors, emphasizing the importance of data-driven decision-making in today's world. Proficiency in R can open doors to a wide range of job opportunities where data analysis and statistical modeling are crucial.

Installing R: A Step-by-Step Guide

Installing R is straightforward, whether you're using Windows, macOS, or a Linux distribution. Below is a step-by-step guide to help you through the process, which you can use to instruct your course attendees.

For Windows Users

1. **Download R:**
   - Visit the Comprehensive R Archive Network (CRAN) at https://cran.r-project.org/.
   - Click on "Download R for Windows".
   - Go to "install R for the first time" and click on "Download R x.x.x for Windows" (x.x.x denotes the latest version).
2. **Install R:**
   - Run the downloaded `.exe` file and follow the installation instructions.
   - Choose your preferred installation directory.
   - Select components to install (you can keep the default settings).
   - Choose the start menu folder (or accept the default), and decide whether to create a desktop icon or Quick Launch shortcut.
   - Review the installation options and click 'Finish' once the installation is complete.

For macOS Users

1. **Download R:**
   - Visit CRAN at https://cran.r-project.org/.
   - Click on "Download R for (Mac) OS X".
   - Select the latest version of R and download the `.pkg` file suitable for your version of macOS.
2. **Install R:**
   - Open the downloaded `.pkg` file.
   - Follow the installation instructions provided by the installer.
   - If you encounter a warning about software from an unidentified developer, go to your system preferences under "Security & Privacy" and allow the R installation.

For Linux Users

- The installation process for R on Linux varies depending on the distribution. Below is a general guide for Debian/Ubuntu-based systems. For other distributions, refer to the official documentation or community forums for specific instructions.

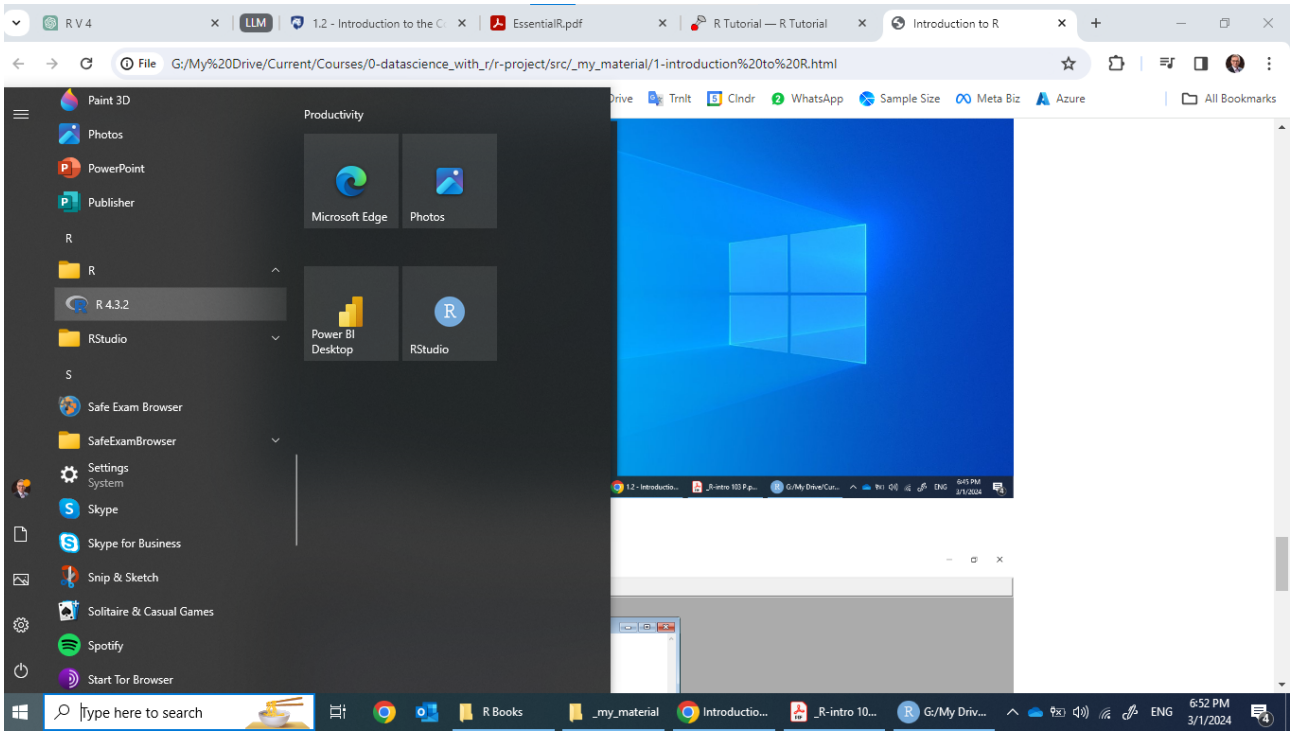
1. **Open Terminal.**
2. **Add CRAN to your repository list (optional, for the latest versions):**
    - For Ubuntu, add a CRAN repository by typing `sudo add-apt-repository 'deb https://<my.favorite.cran.mirror>/bin/linux/ubuntu <ubuntu_version_codename>/'` in the terminal. Replace `<my.favorite.cran.mirror>` with your preferred CRAN mirror and `<ubuntu_version_codename>` with your Ubuntu version codename (e.g., bionic, focal).
3. **Update package lists:**
    - Run `sudo apt update`.
4. **Install R:**
    - Install R by running `sudo apt install r-base`.

Additional Steps for All Users

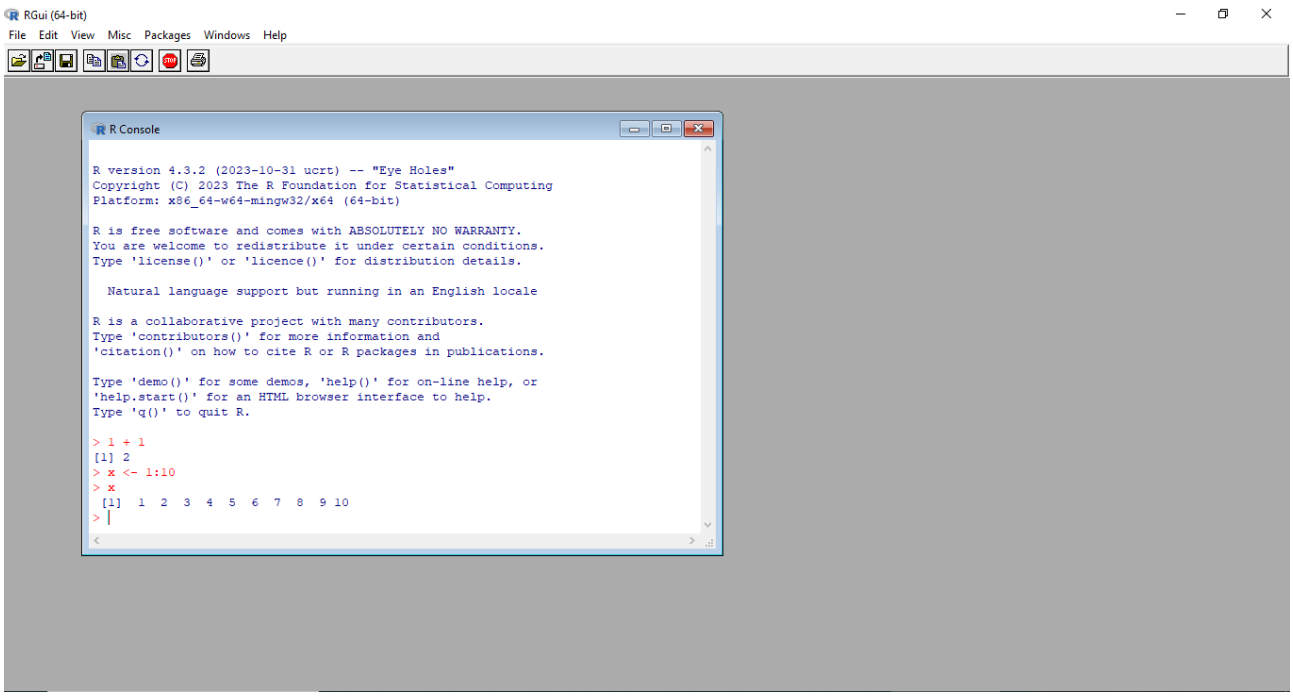
- **Install RStudio (Optional):**
    - For a more user-friendly interface, download and install RStudio, a popular IDE for R, from https://www.rstudio.com/products/rstudio/download/.
    - Choose the appropriate installer for your operating system, download it, and run the installation file, following the on-screen instructions.
- **Verify Installation:**
    - Open R or RStudio and type `sessionInfo()` in the console. This command will display your R session information, confirming that R is installed correctly.

Presenting the R Console

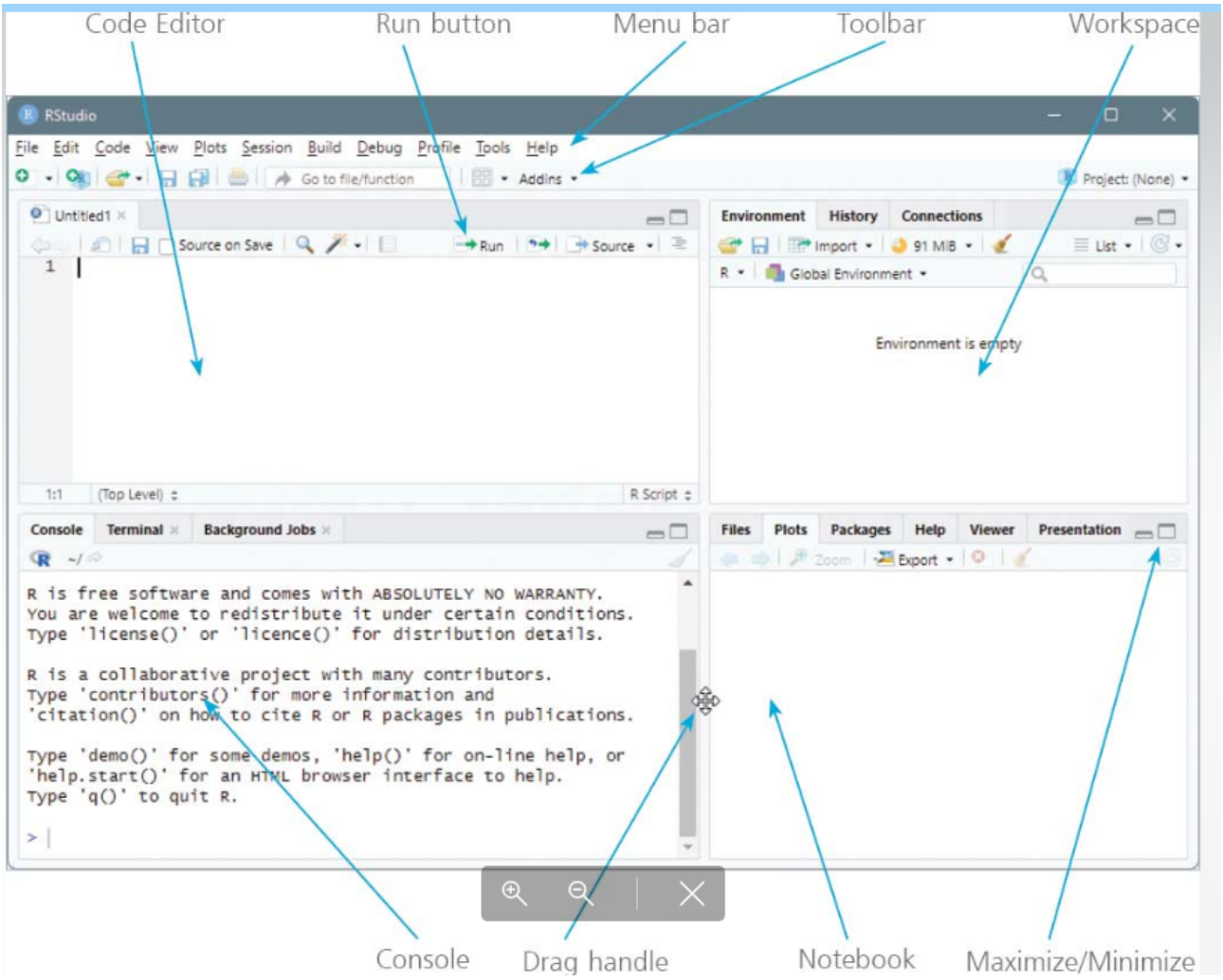
Click on start menu and scroll to find R folder



Then open the R Console

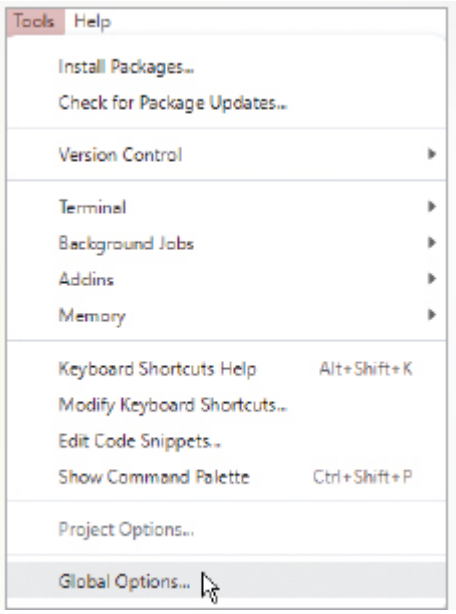


Presenting the R Studio

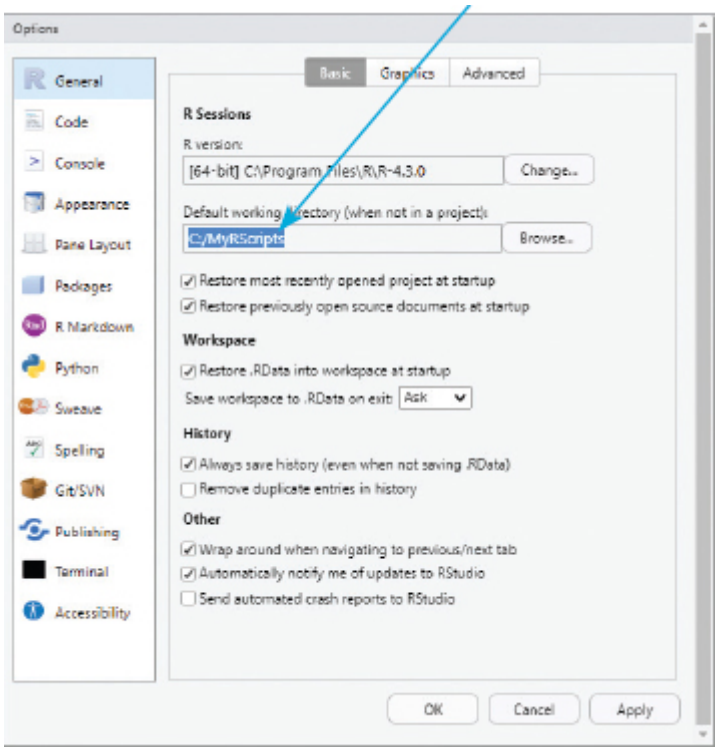


Settings - Working Directory

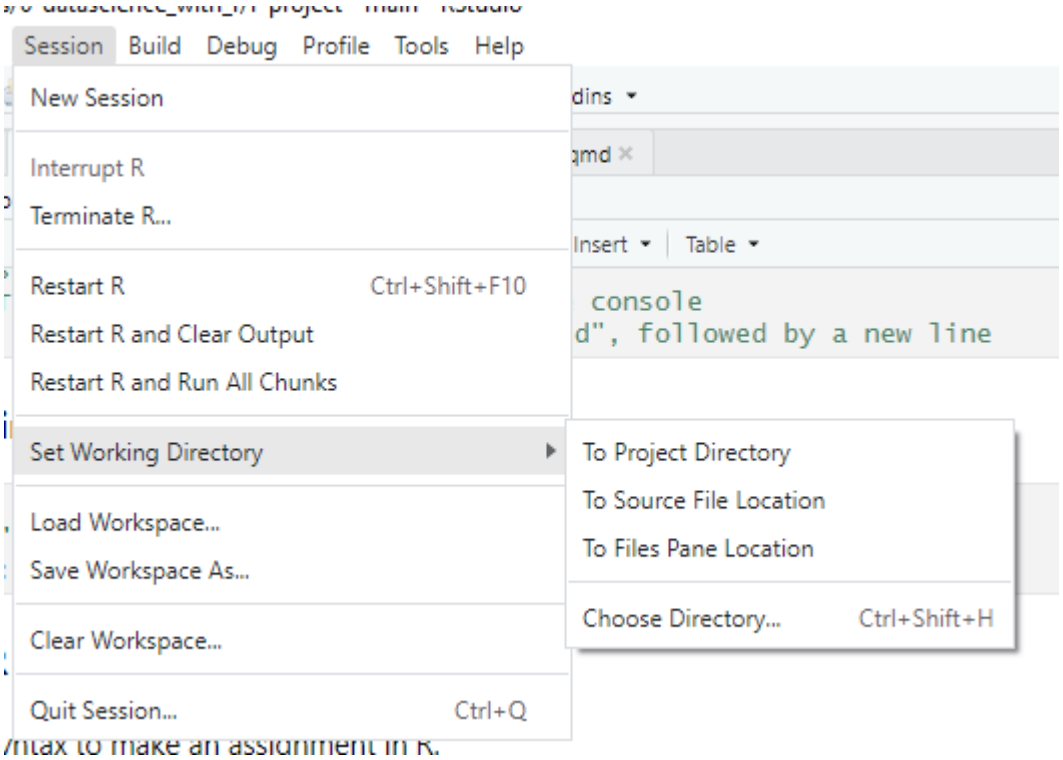
Select General in the left panel of the “Options” dialog, then enter the path to the directory you created into the Default working directory box



Set the default working directory to your preferred folder.



We can also change the working directory from the sessions menu.



References

Gen AI.

In addition, we will be using the following sources for the next sections.

https://online.stat.psu.edu/stat484/

https://online.stat.psu.edu/statprogram/sites/statprogram/files/EssentialR.pdf

https://www.cyclismo.org/tutorial/R/index.html

R for Data Analysis in easy steps R Programming Essentials - Mike McGrath

Module 1: Basic Concepts in R

Lesson 2 | Objects, vectors, and data frames

Table of contents

Hello World Program 1
R Command Prompt 1
Writing R Script File 2
Comments 2
Variables and Basic Data Types 2
Numeric Variables 2
Complext Data Types and Data Structures 5
Other Data Structures (Matrix, Array, List and Data frames) 15
Miscellaneous Related Topics - عيضاوم ةقرفتم ذات ةقاعل 27
Flow of Control 30
Creating Functions in R 36

Reduced teaching slides (PDF) · Module overview

This extended lesson provides the full explanation and reproducible examples behind the slides.

Hello World Program

Write the “Hello World” program using both (1) R command prompt and (2) writing a script.

R Command Prompt

Once you have R environment setup, then it’s easy to start your R command prompt by just typing the following command at your command prompt:

```
$ R
```

This will launch R interpreter and you will get a prompt > where you can start typing your program as follows:

```
# print() displays the supplied value or the contents of a saved object.
# Use <- to assign a result to a name before reusing it in later expressions.
myString <- "Hello, World!"
print (myString)
```

```
[1] "Hello, World!"
```

Here first statement defines a string variable myString, where we assign a string “Hello, World!” and then next statement print() is being used to print the value stored in variable myString.

Note The bracketed number [1] that appears before the output indicates that the line begins with the first value of the result. Some results may have multiple values that fill several lines, so this indicator is occasionally useful but can generally be ignored.

Writing R Script File

Usually, you will do your programming by writing your programs in script files and then you execute those scripts at your command prompt with the help of R interpreter called Rscript.

So let's start with writing following code in a text file called test.R:

```
# print() displays the supplied value or the contents of a saved object.
# Use <- to assign a result to a name before reusing it in later expressions.
# My first program in R Programming
myString <- "Hello, World!"
print ( myString)
```

```
[1] "Hello, World!"
```

Save the above code in a file test.R and execute it at Linux command prompt as given below.

Even if you are using Windows or other system, syntax will remain same.

```
$ Rscript test.R
```

Comments

Comments are like helping text in your R program and they are ignored by the interpreter while executing your actual program. Single comment is written using # in the beginning of the statement as follows:

```
# My first program in R Programming
```

Variables and Basic Data Types

- Generally, while doing programming in any programming language, you need to use various variables to store various information.
- Variables are reserved memory locations to store values. This means that, when you create a variable you reserve some space in memory.
- You may need to store information of various data types like character, wide character, integer, floating point, double floating point, Boolean etc.
- Based on the data type of a variable, the operating system allocates memory and decides what can be stored in the reserved memory.
- In contrast to other programming languages like C and java in R, the variables are not declared as some data type. The variables are assigned with R-Objects and the data type of the R-object becomes the data type of the variable. There are many types of R-objects.
- The frequently used ones are – **Vectors Lists - Matrices - Arrays - Factors - Data Frames**

Numeric Variables

Numeric variables are the most common type of variable and are used to store real numbers.

- **Integer**: A subtype of numeric that represents whole numbers.

```r
# Assign the expression on the right to the object named on the left.
# Later calculations use the stored value; assignment replaces any previous value with
 ↪  that name.
# Integer variable
count <- 42L # The 'L' specifies it as an integer

# Floating-point number
weight <- 72.5
```

- **Complex Numbers**: Numbers with a real and an imaginary part.

```r
# The suffix i denotes the imaginary component of a complex number in R.
# This example stores the value with real part 3 and imaginary part 4.
# Complex number
complex_num <- 3+4i
```

- **Character Variables**

Character variables store text and are enclosed in quotes.

```r
# Assign the expression on the right to the object named on the left.
# Later calculations use the stored value; assignment replaces any previous value with
 ↪  that name.
# Single character string
name <- "Alice"

# Multiple characters string
sentence <- "Hello, World!"
```

- **Logical Variables**

Logical variables can only take two values: TRUE or FALSE. They are the result of conditions or logical operations.

```r
# Logical values record TRUE or FALSE; comparisons create logical results.
# & requires both conditions, while | accepts either; prefer TRUE and FALSE over the
 ↪  reassignable T and F names.
# Logical variable
is_valid <- TRUE

# Result of a logical operation
test_result <- 5 > 3 # Returns TRUE

a <- T
b <- F

a | b
```

```
[1] TRUE
```

```r
a & b
```

```
[1] FALSE
```

R is Case Sensitive

The R language is case-sensitive, so typing the command as Print( ) or PRINT( ) will simply produce an error.

```r
# print(): displays the supplied value or object.
# Print(): has intentionally incorrect capitalization in this error example; R requires
 ↪  print().
# R is case sensitive: age, Age, and the function names print and Print are different
 ↪  names.
# The mismatched names below intentionally demonstrate errors rather than a working
 ↪  calculation.
age <- 25
print(age)
```

```
[1] 25
```

```r
print(Age)
```

```
Error:
! object 'Age' not found
```

```r
Print(age) # P is capitalized in print
```

```
Error in `Print()`:
! could not find function "Print"
```

Get Data Type of a Variable

```r
# c(): combines values into a vector, converting them to a common type when needed.
# data.frame() combines equally aligned vectors into a rectangular table.
# Columns describe variables and rows describe observations; retain the meaning and units
 ↪  of each column.
BMI <- data.frame(
gender = c("Male", "Male","Female"),
height = c(152, 171.5, 165),
weight = c(81,93, 78),
Age = c(42,38,26)
)
```

```r
# print(): displays the supplied value or object.
# str() displays structure, including column types and example values.
# class() reports the object class, while typeof() describes its underlying storage type.
print("The class of BMI is:")
```

```
[1] "The class of BMI is:"
```

```r
class(BMI)
```

```
[1] "data.frame"
```

```r
print("The type of BMI is:")
```

```
[1] "The type of BMI is:"
```

```r
typeof(BMI)
```

```
[1] "list"
```

```r
print("The structure of BMI is:")
```

```
[1] "The structure of BMI is:"
```

```r
str(BMI)
```

```
'data.frame':   3 obs. of  4 variables:
 $ gender: chr  "Male" "Male" "Female"
 $ height: num  152 172 165
 $ weight: num  81 93 78
 $ Age   : num  42 38 26
```

Logical operators in R:

<	less than
>	great than
<=	less than or equal
>=	greater than or equal
==	equal to
!=	not equal to
|	or
!	not
&	and

Complext Data Types and Data Structures

Vectors

Vectors are one-dimensional arrays that can hold numeric, character, or logical values, but all elements must be of the same type.

Patient ID	Height (CM)	Weight (KG)	Systolic (mmHg)	Diastolic (mmHg)
10200	165	71	120	80
3600	180	92	110	85
8437	172	76	118	82
5911	167	82	115	78

```r
# c(): combines values into a vector, converting them to a common type when needed.
# Logical values record TRUE or FALSE; comparisons create logical results.
# & requires both conditions, while | accepts either; prefer TRUE and FALSE over the
 ↪  reassignable T and F names.
# Numeric vector
ages <- c(25, 30, 45)

# Character vector
colors <- c("red", "green", "blue")

# Logical vector
answers <- c(TRUE, FALSE, TRUE)
```

Creating Sequences in R

Sequences are ordered sets of numbers that follow a specific pattern.

In R, sequences are commonly used for tasks such as iteration in loops, subsetting data structures, and during plotting.

R provides several functions to create sequences efficiently.

Using the Colon Operator :

The colon operator is the simplest way to create a sequence of numbers in R. It generates a sequence from a starting point to an endpoint with a step of 1.

Syntax:

```
# The colon operator creates a sequence in steps of one from start toward end.
# This is a syntax template; replace start and end with actual endpoint values.
start:end
```

Example:

Create a sequence from 1 to 10.

```
# print() displays the supplied value or the contents of a saved object.
# Use <- to assign a result to a name before reusing it in later expressions.
seq_1_to_10 <- 1:10
print(seq_1_to_10)
```

```
 [1]  1  2  3  4  5  6  7  8  9 10
```

The `seq()` Function

The `seq()` function is more versatile than the colon operator and can create sequences with specified increments (the `by` argument) or of a specified length (the `length.out` argument).

Syntax:

```
# seq() creates a sequence: from/to set endpoints, by sets spacing, and length.out sets
 ↪  the number of values.
# This is a syntax template; normally specify by or length.out, not conflicting values for
 ↪  both.
seq(from, to, by, length.out)
```

Examples:

- Create a sequence from 1 to 10 with a step of 2:

```
# print(): displays the supplied value or object.
# seq() creates evenly spaced values between the specified limits.
# by sets the step size; length.out instead requests a chosen number of evenly spaced
 ↪  values.
seq_by_2 <- seq(from = 1, to = 10, by = 2)
print(seq_by_2)
```

```
[1] 1 3 5 7 9
```

- Create a sequence from 5 to 10, consisting of 3 equally spaced numbers:

```r
# print(): displays the supplied value or object.
# seq() creates evenly spaced values between the specified limits.
# by sets the step size; length.out instead requests a chosen number of evenly spaced
 ↪  values.
seq_length_out <- seq(from = 5, to = 10, length.out = 3)
print(seq_length_out)
```

```
[1]  5.0  7.5 10.0
```

The rep() Function

The rep() function is used to replicate the values in x. It is a powerful function for repeating sequences.

Syntax:

```r
# rep() repeats values: times repeats the vector, each repeats its elements, and
 ↪  length.out sets output length.
# This is a syntax template; supply x and the repetition arguments needed for the task.
rep(x, times, each, length.out)
```

Examples:

- Repeat the sequence of numbers 1, 2, 3, three times:

```r
# print(): displays the supplied value or object.
# rep() repeats supplied values to construct a vector.
# times repeats the entire sequence; each repeats individual elements before moving on.
rep_times <- rep(x = 1:3, times = 3)
print(rep_times)
```

```
[1] 1 2 3 1 2 3 1 2 3
```

- Repeat each number in the sequence 1, 2, 3, three times:

```r
# print(): displays the supplied value or object.
# rep() repeats supplied values to construct a vector.
# times repeats the entire sequence; each repeats individual elements before moving on.
rep_each <- rep(x = 1:3, each = 3)
print(rep_each)
```

```
[1] 1 1 1 2 2 2 3 3 3
```

Combining Sequences

Sequences can be combined using the c() function to create longer and more complex sequences.

Example:

Combine two sequences into one.

```r
# c(): combines values into a vector, converting them to a common type when needed.
# print(): displays the supplied value or object.
# seq() creates evenly spaced values between the specified limits.
# by sets the step size; length.out instead requests a chosen number of evenly spaced
 ↪  values.
combined_seq <- c(seq(1, 5), seq(8, 10))
print(combined_seq)
```

```
[1]  1  2  3  4  5  8  9 10
```

Practice Vectors

- Create a vector that combines the numbers 1, 2, and 3 with the letters ‘a’, ‘b’, and ‘c’.
- Create a vector containing the numbers 1 to 10, but only include every second number.
- Create a vector that contains the sequence 1, 2, 3, followed by the sequence 4, 5, 6, each sequence repeated twice.
- Create a vector that starts with the sequence 1, 2, 3 and then repeats the number 4 five times.
- Create a vector containing the first 10 even numbers.
- Create a vector containing the first 10 odd numbers.
- Create a vector that uses the `seq()` function to generate numbers from 1 to 10, but each number should appear twice.
- Create a vector that uses the `rep()` function to repeat the sequence 1, 2, 3, but the entire sequence should appear three times.

Access Vector Elements

Accessing elements from a vector in R is straightforward and can be achieved using indexing with square brackets `[ ]`.

You can specify the index or indices of the elements you want to extract. Remember, R is 1-based indexing, meaning the first element of a vector has an index of 1.

Single Element Access

To access a single element from a vector, you provide its index within square brackets immediately following the vector name.

```r
# print(): displays the supplied value or object.
# [] selects vector values by position; R starts counting positions at 1.
# c(1,3,5) selects several positions, while 2:4 selects a consecutive range.
# Creating a numeric vector
numbers <- c(10, 20, 30, 40, 50)

# Access the third element
third_number <- numbers[3]
print(third_number)  # Outputs 30
```

```
[1] 30
```

Multiple Elements Access

To access multiple elements, you can pass a vector of indices inside the square brackets.

```r
# print(): displays the supplied value or object.
# [] selects vector values by position; R starts counting positions at 1.
# c(1,3,5) selects several positions, while 2:4 selects a consecutive range.
# Access the first, third, and fifth elements
selected_numbers <- numbers[c(1, 3, 5)]
print(selected_numbers)  # Outputs 10, 30, 50
```

```
[1] 10 30 50
```

Sequential Elements Access

R allows you to use the colon : operator to create a sequence, which can be used for accessing a range of elements.

```r
# print(): displays the supplied value or object.
# [] selects vector values by position; R starts counting positions at 1.
# c(1,3,5) selects several positions, while 2:4 selects a consecutive range.
# Access the second to the fourth elements
range_numbers <- numbers[2:4]
print(range_numbers)  # Outputs 20, 30, 40
```

```
[1] 20 30 40
```

Conditional Access

You can also access elements based on conditions. This method is useful for filtering elements.

```r
# print(): displays the supplied value or object.
# A logical condition inside [] selects the entries for which it is TRUE.
# The result contains matching values in their original order, rather than their integer
 ↪  positions.
# Access elements greater than 25
filtered_numbers <- numbers[numbers > 25]
print(filtered_numbers)  # Outputs 30, 40, 50
```

```
[1] 30 40 50
```

Find the Index of Matching Elements

```r
# which() returns the positions where a logical condition is TRUE.
# Use these integer positions to select matching entries without changing the original
 ↪  vector.
which(numbers > 25)
```

```
[1] 3 4 5
```

Excluding Elements

Adding a minus sign - before an index or a vector of indices tells R to exclude those elements.

```r
# print(): displays the supplied value or object.
# Negative indices inside [] exclude the specified positions from a vector.
# Parentheses keep a multi-position exclusion together, as in -(2:4).
# Exclude the second element
exclude_second <- numbers[-2]
print(exclude_second)  # Outputs 10, 30, 40, 50
```

```
[1] 10 30 40 50
```

```r
# Exclude second to fourth elements
exclude_range <- numbers[-(2:4)]
print(exclude_range)  # Outputs 10, 50
```

```
[1] 10 50
```

Named Vectors

For named vectors, you can access elements using their names in a similar manner.

```
# print(): displays the supplied value or object.
# c() combines compatible values into a vector in the supplied order.
# Assignments save the vector under a name; printing that name displays its current
 ↪  values.
# Creating a named vector
fruits <- c(apple = 10, banana = 5, cherry = 8)

# Access the element named 'banana'
banana_count <- fruits["banana"]
print(banana_count)  # Outputs 5
```

```
banana
     5
```

```
# Access multiple elements by name
some_fruits <- fruits[c("apple", "cherry")]
print(some_fruits)  # Outputs apple 10, cherry 8
```

```
 apple cherry
    10      8
```

Basic Operations on Vectors

- Once you have a vector (or a list of numbers) in memory most basic operations are available.
- Most of the basic operations will act on a whole vector and can be used to quickly perform a large number of calculations with a single command.
- There is one thing to note, if you perform an operation on more than one vector it is often necessary that the vectors all contain the same number of entries.
- Here we first define a vector which we will call “a” and will look at how to add and subtract constant numbers from all of the numbers in the vector.
- First, the vector will contain the numbers 1, 2, 3, and 4.
- We then see how to add 5 to each of the numbers, subtract 10 from each of the numbers, multiply each number by 4, and divide each number by 5.

```
# c() combines compatible values into a vector in the supplied order.
# Assignments save the vector under a name; printing that name displays its current
 ↪  values.
a <- c(1,2,3,4)
a
```

```
[1] 1 2 3 4
```

```
a <- a + 5
a
```

```
[1] 6 7 8 9
```

```
a <- a - 10
a
```

```
[1] -4 -3 -2 -1
```

```
a <- a / 5
a
```

```
[1] -0.8 -0.6 -0.4 -0.2
```

We can save the results in another vector called b:

```
# Assign the expression on the right to the object named on the left.
# Later calculations use the stored value; assignment replaces any previous value with
 ↪  that name.
b <- a
b
```

```
[1] -0.8 -0.6 -0.4 -0.2
```

If you want to raise to a power, take the square root, find e raised to each number, the logarithm, etc., then the usual commands can be used:

```
# c(): combines values into a vector, converting them to a common type when needed.
# exp(): computes e raised to each supplied value.
# log(): computes natural logarithms by default; base selects another logarithm base.
# sqrt() returns the square root of each input value.
# Arithmetic on vectors is element by element; check domains before taking square roots or
 ↪  logarithms.
a <- c(1,2,3,4)
a ^ 2
```

```
[1]  1  4  9 16
```

```
sqrt(a)
```

```
[1] 1.000000 1.414214 1.732051 2.000000
```

```
exp(a)
```

```
[1]  2.718282  7.389056 20.085537 54.598150
```

```
log(a)
```

```
[1] 0.0000000 0.6931472 1.0986123 1.3862944
```

```
exp(log(a))
```

```
[1] 1 2 3 4
```

By combining operations and using parentheses you can make more complicated expressions:

```
# exp(): computes e raised to each supplied value.
# sqrt() returns the square root of each input value.
# Arithmetic on vectors is element by element; check domains before taking square roots or
 ↪  logarithms.
c <- (a + sqrt(a))/(exp(2)+1)
c
```

```
[1] 0.2384058 0.4069842 0.5640743 0.7152175
```

Note that you can do the same operations with vector arguments.

For example to add the elements in vector a to the elements in vector b use the following command:

```
# Apply the arithmetic operation element by element to the supplied vectors.
# Unequal vector lengths invoke recycling, so check lengths before interpreting the
 ↪  result.
a + b
```

```
[1] 0.2 1.4 2.6 3.8
```

The operation is performed on an element by element basis.

Note this is true for almost all of the basic functions.

So you can bring together all kinds of complicated expressions:

```
# sqrt() returns the square root of each input value.
# Arithmetic on vectors is element by element; check domains before taking square roots or
 ↪  logarithms.
a*b
```

```
[1] -0.8 -1.2 -1.2 -0.8
```

```
a/b
```

```
[1]  -1.250000  -3.333333  -7.500000 -20.000000
```

```
(a+3)/(sqrt(1-b)*2-1)
```

```
[1] 2.376311 3.268354 4.390998 5.877956
```

You need to be careful of one thing. When you do operations on vectors they are performed on an element by element basis. One ramification of this is that all of the vectors in an expression must be the same length. If the lengths of the vectors differ then you may get an error message, or worse, a warning message and unpredictable results:

```
# c() combines compatible values into a vector in the supplied order.
# Assignments save the vector under a name; printing that name displays its current
 ↪  values.
a <- c(1,2,3)
b <- c(10,11,12,13)
a+b
```

Basic Numerical Descriptors

The following commands can be used to get the mean, median, quantiles, minimum, maximum, variance, and standard deviation of a set of numbers:

```
# quantile(): computes sample percentiles; probs gives probabilities and type selects the
 ↪  interpolation rule.
# min(): returns the smallest supplied value.
# max(): returns the largest supplied value.
# mean() and median() describe center; var() and sd() summarize numerical spread.
# var() uses the sample denominator n-1; sd() expresses that spread in the original units.
a <- 1:10
mean(a)
```

```
[1] 5.5
```

```
median(a)
```

```
[1] 5.5
```

```
quantile(a)
```

```
   0%   25%   50%   75%  100%
 1.00  3.25  5.50  7.75 10.00
```

```
var(a)
```

```
[1] 9.166667
```

```
sd(a)
```

```
[1] 3.02765
```

```
min(a)
```

```
[1] 1
```

```
max(a)
```

```
[1] 10
```

Finally, the summary command will print out the min, max, mean, median, and quantiles:

```
# summary() reports an overview appropriate to the object or its columns.
# For numerical vectors it shows the range, quartiles, median, and mean; inspect the
 ↪  object type too.
summary(a)
```

```
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
   1.00    3.25    5.50    5.50    7.75   10.00
```

The summary command is especially nice because if you give it a data frame it will print out the summary for every vector in the data frame:

```
# summary() reports an overview appropriate to the object or its columns.
# For numerical vectors it shows the range, quartiles, median, and mean; inspect the
 ↪  object type too.
summary(mtcars)
```

```
      mpg             cyl             disp             hp
 Min.   :10.40   Min.   :4.000   Min.   : 71.1   Min.   : 52.0
 1st Qu.:15.43   1st Qu.:4.000   1st Qu.:120.8   1st Qu.: 96.5
 Median :19.20   Median :6.000   Median :196.3   Median :123.0
 Mean   :20.09   Mean   :6.188   Mean   :230.7   Mean   :146.7
 3rd Qu.:22.80   3rd Qu.:8.000   3rd Qu.:326.0   3rd Qu.:180.0
 Max.   :33.90   Max.   :8.000   Max.   :472.0   Max.   :335.0
      drat             wt             qsec             vs
 Min.   :2.760   Min.   :1.513   Min.   :14.50   Min.   :0.0000
 1st Qu.:3.080   1st Qu.:2.581   1st Qu.:16.89   1st Qu.:0.0000
 Median :3.695   Median :3.325   Median :17.71   Median :0.0000
 Mean   :3.597   Mean   :3.217   Mean   :17.85   Mean   :0.4375
 3rd Qu.:3.920   3rd Qu.:3.610   3rd Qu.:18.90   3rd Qu.:1.0000
 Max.   :4.930   Max.   :5.424   Max.   :22.90   Max.   :1.0000
       am              gear            carb
 Min.   :0.0000   Min.   :3.000   Min.   :1.000
 1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:2.000
 Median :0.0000   Median :4.000   Median :2.000
 Mean   :0.4062   Mean   :3.688   Mean   :2.812
 3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:4.000
 Max.   :1.0000   Max.   :5.000   Max.   :8.000
```

Sorting Vectors

Here we look at some commonly used commands that perform operations on lists.

The commands include the sort, min, max, and sum commands.

First, the sort command can sort the given vector in either ascending or descending order:

```r
# c(): combines values into a vector, converting them to a common type when needed.
# sort() rearranges values into increasing order by default.
# decreasing=TRUE reverses the order; this returns values rather than their original
 ↪  positions.
a = c(2,4,6,3,1,5)
b = sort(a)
c = sort(a,decreasing = TRUE)
```

Speed of Vectorized Operations

Using vectorization to multiply numbers vs using for loop

```r
# rnorm(): generates artificial normal draws; n is the draw count and sd is standard
 ↪  deviation.
# rep(): repeats values; times repeats a vector and each repeats individual elements.
# A for loop repeats its body once for each value in the supplied sequence.
# The loop variable identifies the current value or position; vectorized arithmetic can
 ↪  often replace the loop.
N <- 100
a <- rnorm(N)
b <- rnorm(N)

# Vectorized approach
c <- a * b

# De-vectorized approach
d <- rep(NA, N)
for(i in 1:N){
  d[i] <- a[i] * b[i]
}
```

```r
# rnorm(): generates artificial normal draws; n is the draw count and sd is standard
 ↪  deviation.
# system.time(): measures the computing and elapsed time needed to evaluate an expression.
# rep(): repeats values; times repeats a vector and each repeats individual elements.
# print(): displays the supplied value or object.
# A for loop repeats its body once for each value in the supplied sequence.
# The loop variable identifies the current value or position; vectorized arithmetic can
 ↪  often replace the loop.
# Use 100,000 values to keep memory use modest on student computers.
# Timings vary by computer and run; compare approaches rather than expecting fixed
 ↪  results.
N <- 100000
a <- rnorm(N)
b <- rnorm(N)

# Measure the time for the vectorized approach
vectorized_time <- system.time({
  c <- a * b
})

# Measure the time for the de-vectorized approach
de_vectorized_time <- system.time({
  d <- rep(NA, N)
  for(i in 1:N){
    d[i] <- a[i] * b[i]
  }
})

# Print the timings
print(vectorized_time)
```

```
   user  system elapsed
      0       0       0
```

```
print(de_vectorized_time)
```

```
   user  system elapsed
  0.004   0.000   0.004
```

Other Data Structures (Matrix, Array, List and Data frames)

Matrices

Matrices are two-dimensional arrays that contain elements of the same type. They have rows and columns.

```
# matrix() arranges values into rows and columns of one underlying data type.
# R fills columns first by default; nrow and ncol specify the requested dimensions.
# Matrix with 3 rows and 2 columns
matrix_data <- matrix(1:6, nrow = 3, ncol = 2)
```

Note: Check the NBA Example Quarto File

Arrays

Arrays are similar to matrices but can have more than two dimensions.

```
# c(): combines values into a vector, converting them to a common type when needed.
# array() arranges values into a multidimensional object of one underlying type.
# dim specifies the size of each dimension, in the order used when indexing the array.
# 3-dimensional array
array_data <- array(1:24, dim = c(3, 4, 2))
```

Lists

Lists can hold elements of different types, including numbers, strings, vectors, and even other lists.

```
# c(): combines values into a vector, converting them to a common type when needed.
# print(): displays the supplied value or object.
# list() keeps several objects together even when their types or lengths differ.
# Single brackets return a sublist; double brackets extract an element, and $ can select a
 ↪  named element.
# List containing different types
list_data <- list(name = "Alice", age = 25, scores = c(90, 80, 85))

print(list_data[1])
```

```
$name
[1] "Alice"
```

Data Frames

Data frames are used to store tabular data. They are similar to matrices but can contain different types of variables.

```
# c(): combines values into a vector, converting them to a common type when needed.
# data.frame() combines equally aligned vectors into a rectangular table.
# Columns describe variables and rows describe observations; retain the meaning and units
 ↪  of each column.
# Data frame
df <- data.frame(
  Name = c("Alice", "Bob"),
  Age = c(25, 30),
  Weight = c(55.5, 85.7)
)
```

Access Data Frame Elements

R built-in datasets

R provides several datasets that can be used for practice.

Use the data() command to examine these datasets.

```
#data()

# we will use mtcars dataset for demonstration
#?mtcars
#View(trees)
```

Using $ to Access Columns

The **$** operator is used to access a column directly by its name, returning the column as a vector.

```
# print(): displays the supplied value or object.
# mpg in mtcars measures miles per gallon, not miles per hour.
# $ extracts a column vector; single-column ["mpg"] indexing retains a data frame.
miles_per_hour <- mtcars$mpg
print(miles_per_hour)
```

```
 [1] 21.0 21.0 22.8 21.4 18.7 18.1 14.3 24.4 22.8 19.2 17.8 16.4 17.3 15.2 10.4
[16] 10.4 14.7 32.4 30.4 33.9 21.5 15.5 15.2 13.3 19.2 27.3 26.0 30.4 15.8 19.7
[31] 15.0 21.4
```

Using [] for Row and Column Indexing

The **[ ]** operator allows for more flexible sub-setting, including selecting rows, columns, or both, using numeric indexes or column names.

- **Select a single column by name**:

```
# print(): displays the supplied value or object.
# mpg in mtcars measures miles per gallon, not miles per hour.
# $ extracts a column vector; single-column ["mpg"] indexing retains a data frame.
miles_per_hour <- mtcars["mpg"]
print(miles_per_hour)
```

```
                   mpg
Mazda RX4         21.0
Mazda RX4 Wag     21.0
Datsun 710        22.8
Hornet 4 Drive    21.4
Hornet Sportabout 18.7
Valiant           18.1
Duster 360        14.3
Merc 240D         24.4
Merc 230          22.8
Merc 280          19.2
Merc 280C         17.8
Merc 450SE        16.4
Merc 450SL        17.3
Merc 450SLC       15.2
Cadillac Fleetwood  10.4
Lincoln Continental 10.4
Chrysler Imperial   14.7
Fiat 128          32.4
Honda Civic       30.4
Toyota Corolla    33.9
Toyota Corona     21.5
Dodge Challenger  15.5
AMC Javelin       15.2
Camaro Z28        13.3
Pontiac Firebird  19.2
Fiat X1-9         27.3
Porsche 914-2     26.0
Lotus Europa      30.4
Ford Pantera L    15.8
Ferrari Dino      19.7
Maserati Bora     15.0
Volvo 142E        21.4
```

Select multiple columns by name

```r
# c(): combines values into a vector, converting them to a common type when needed.
# print(): displays the supplied value or object.
# A data frame uses [row, column] indexing to select records and variables.
# A blank row or column position keeps all entries in that dimension.
subset_df <- mtcars[c("mpg", "cyl")]
print(subset_df)
```

```
                   mpg cyl
Mazda RX4         21.0   6
Mazda RX4 Wag     21.0   6
Datsun 710        22.8   4
Hornet 4 Drive    21.4   6
Hornet Sportabout 18.7   8
Valiant           18.1   6
Duster 360        14.3   8
Merc 240D         24.4   4
Merc 230          22.8   4
Merc 280          19.2   6
Merc 280C         17.8   6
Merc 450SE        16.4   8
Merc 450SL        17.3   8
Merc 450SLC       15.2   8
Cadillac Fleetwood  10.4   8
Lincoln Continental 10.4   8
Chrysler Imperial   14.7   8
Fiat 128          32.4   4
Honda Civic       30.4   4
Toyota Corolla    33.9   4
Toyota Corona     21.5   4
Dodge Challenger  15.5   8
AMC Javelin       15.2   8
```

```
Camaro Z28          13.3   8
Pontiac Firebird    19.2   8
Fiat X1-9           27.3   4
Porsche 914-2       26.0   4
Lotus Europa        30.4   4
Ford Pantera L      15.8   8
Ferrari Dino        19.7   6
Maserati Bora       15.0   8
Volvo 142E          21.4   4
```

```r
# print(): displays the supplied value or object.
# head(): shows the first n entries or rows, six by default.
# c(): combines values into a vector, converting them to a common type when needed.
# length(): counts elements in a vector or list.
# nrow(): returns the number of rows in a table or matrix.
# ncol(): returns the number of columns in a table or matrix.
# mean(): computes an average; logical TRUE/FALSE values contribute 1/0, giving a
 ↪  proportion.
# mpg in mtcars measures miles per gallon, not miles per hour.
# $ extracts a column vector; single-column ["mpg"] indexing retains a data frame.
# In R, rownames is a function that retrieves or sets the row names of a matrix or # data
 ↪  frame.
# rownames are essentially a character vector of unique identifiers that label the rows,
 ↪  similar to how column names label the columns.

# Get all rownames
car_names <- rownames(mtcars)
print(car_names)
```

```
 [1] "Mazda RX4"           "Mazda RX4 Wag"       "Datsun 710"
 [4] "Hornet 4 Drive"      "Hornet Sportabout"   "Valiant"
 [7] "Duster 360"          "Merc 240D"           "Merc 230"
[10] "Merc 280"            "Merc 280C"           "Merc 450SE"
[13] "Merc 450SL"          "Merc 450SLC"         "Cadillac Fleetwood"
[16] "Lincoln Continental" "Chrysler Imperial"   "Fiat 128"
[19] "Honda Civic"         "Toyota Corolla"      "Toyota Corona"
[22] "Dodge Challenger"    "AMC Javelin"         "Camaro Z28"
[25] "Pontiac Firebird"    "Fiat X1-9"           "Porsche 914-2"
[28] "Lotus Europa"        "Ford Pantera L"      "Ferrari Dino"
[31] "Maserati Bora"       "Volvo 142E"
```

```r
# Get first rowname only
first_car_name <- rownames(mtcars)[1]
print(first_car_name)
```

```
[1] "Mazda RX4"
```

```r
# Find record by rowname
datsun_710 <- mtcars["Datsun 710", ]
print(datsun_710)
```

```
           mpg cyl disp hp drat   wt  qsec vs am gear carb
Datsun 710 22.8   4  108 93 3.85 2.32 18.61  1  1    4    1
```

```r
mtcars$car_brand <- rownames(mtcars)
# rownames(mtcars) <- NULL
head(mtcars)
```

```
                  mpg cyl disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6  160 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6  160 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4  108  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6  258 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8  360 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.460 20.22  1  0    3    1
                         car_brand
```

```
Mazda RX4                 Mazda RX4
Mazda RX4 Wag         Mazda RX4 Wag
Datsun 710               Datsun 710
Hornet 4 Drive       Hornet 4 Drive
Hornet Sportabout Hornet Sportabout
Valiant                     Valiant
```

```
#View(mtcars)

# select rows 5 to 7
mtcars
```

```
                   mpg cyl  disp  hp drat    wt  qsec vs am gear carb
Mazda RX4         21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
Mazda RX4 Wag     21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
Duster 360        14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
Merc 240D         24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
Merc 230          22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
Merc 280          19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
Merc 280C         17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
Merc 450SE        16.4   8 275.8 180 3.07 4.070 17.40  0  0    3    3
Merc 450SL        17.3   8 275.8 180 3.07 3.730 17.60  0  0    3    3
Merc 450SLC       15.2   8 275.8 180 3.07 3.780 18.00  0  0    3    3
Cadillac Fleetwood  10.4   8 472.0 205 2.93 5.250 17.98  0  0    3    4
Lincoln Continental 10.4   8 460.0 215 3.00 5.424 17.82  0  0    3    4
Chrysler Imperial   14.7   8 440.0 230 3.23 5.345 17.42  0  0    3    4
Fiat 128          32.4   4  78.7  66 4.08 2.200 19.47  1  1    4    1
Honda Civic       30.4   4  75.7  52 4.93 1.615 18.52  1  1    4    2
Toyota Corolla    33.9   4  71.1  65 4.22 1.835 19.90  1  1    4    1
Toyota Corona     21.5   4 120.1  97 3.70 2.465 20.01  1  0    3    1
Dodge Challenger  15.5   8 318.0 150 2.76 3.520 16.87  0  0    3    2
AMC Javelin       15.2   8 304.0 150 3.15 3.435 17.30  0  0    3    2
Camaro Z28        13.3   8 350.0 245 3.73 3.840 15.41  0  0    3    4
Pontiac Firebird  19.2   8 400.0 175 3.08 3.845 17.05  0  0    3    2
Fiat X1-9         27.3   4  79.0  66 4.08 1.935 18.90  1  1    4    1
Porsche 914-2     26.0   4 120.3  91 4.43 2.140 16.70  0  1    5    2
Lotus Europa      30.4   4  95.1 113 3.77 1.513 16.90  1  1    5    2
Ford Pantera L    15.8   8 351.0 264 4.22 3.170 14.50  0  1    5    4
Ferrari Dino      19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
Maserati Bora     15.0   8 301.0 335 3.54 3.570 14.60  0  1    5    8
Volvo 142E        21.4   4 121.0 109 4.11 2.780 18.60  1  1    4    2
                           car_brand
Mazda RX4                   Mazda RX4
Mazda RX4 Wag           Mazda RX4 Wag
Datsun 710                 Datsun 710
Hornet 4 Drive        Hornet 4 Drive
Hornet Sportabout   Hornet Sportabout
Valiant                       Valiant
Duster 360                 Duster 360
Merc 240D                   Merc 240D
Merc 230                     Merc 230
Merc 280                     Merc 280
Merc 280C                   Merc 280C
Merc 450SE                 Merc 450SE
Merc 450SL                 Merc 450SL
Merc 450SLC               Merc 450SLC
Cadillac Fleetwood   Cadillac Fleetwood
Lincoln Continental Lincoln Continental
Chrysler Imperial     Chrysler Imperial
Fiat 128                       Fiat 128
Honda Civic                 Honda Civic
Toyota Corolla           Toyota Corolla
Toyota Corona             Toyota Corona
```

```
Dodge Challenger       Dodge Challenger
AMC Javelin                 AMC Javelin
Camaro Z28                   Camaro Z28
Pontiac Firebird       Pontiac Firebird
Fiat X1-9                     Fiat X1-9
Porsche 914-2              Porsche 914-2
Lotus Europa                Lotus Europa
Ford Pantera L            Ford Pantera L
Ferrari Dino                Ferrari Dino
Maserati Bora              Maserati Bora
Volvo 142E                    Volvo 142E
```

```
mtcars[5:7,]
```

```
                   mpg cyl disp  hp drat   wt  qsec vs am gear carb
Hornet Sportabout 18.7   8  360 175 3.15 3.44 17.02  0  0    3    2
Valiant           18.1   6  225 105 2.76 3.46 20.22  1  0    3    1
Duster 360        14.3   8  360 245 3.21 3.57 15.84  0  0    3    4
                          car_brand
Hornet Sportabout Hornet Sportabout
Valiant                     Valiant
Duster 360                Duster 360
```

```
# select columns 2 to 9
mtcars[2:9]
```

```
                  cyl  disp  hp drat    wt  qsec vs am
Mazda RX4           6 160.0 110 3.90 2.620 16.46  0  1
Mazda RX4 Wag       6 160.0 110 3.90 2.875 17.02  0  1
Datsun 710          4 108.0  93 3.85 2.320 18.61  1  1
Hornet 4 Drive      6 258.0 110 3.08 3.215 19.44  1  0
Hornet Sportabout   8 360.0 175 3.15 3.440 17.02  0  0
Valiant             6 225.0 105 2.76 3.460 20.22  1  0
Duster 360          8 360.0 245 3.21 3.570 15.84  0  0
Merc 240D           4 146.7  62 3.69 3.190 20.00  1  0
Merc 230            4 140.8  95 3.92 3.150 22.90  1  0
Merc 280            6 167.6 123 3.92 3.440 18.30  1  0
Merc 280C           6 167.6 123 3.92 3.440 18.90  1  0
Merc 450SE          8 275.8 180 3.07 4.070 17.40  0  0
Merc 450SL          8 275.8 180 3.07 3.730 17.60  0  0
Merc 450SLC         8 275.8 180 3.07 3.780 18.00  0  0
Cadillac Fleetwood  8 472.0 205 2.93 5.250 17.98  0  0
Lincoln Continental 8 460.0 215 3.00 5.424 17.82  0  0
Chrysler Imperial   8 440.0 230 3.23 5.345 17.42  0  0
Fiat 128            4  78.7  66 4.08 2.200 19.47  1  1
Honda Civic         4  75.7  52 4.93 1.615 18.52  1  1
Toyota Corolla      4  71.1  65 4.22 1.835 19.90  1  1
Toyota Corona       4 120.1  97 3.70 2.465 20.01  1  0
Dodge Challenger    8 318.0 150 2.76 3.520 16.87  0  0
AMC Javelin         8 304.0 150 3.15 3.435 17.30  0  0
Camaro Z28          8 350.0 245 3.73 3.840 15.41  0  0
Pontiac Firebird    8 400.0 175 3.08 3.845 17.05  0  0
Fiat X1-9           4  79.0  66 4.08 1.935 18.90  1  1
Porsche 914-2       4 120.3  91 4.43 2.140 16.70  0  1
Lotus Europa        4  95.1 113 3.77 1.513 16.90  1  1
Ford Pantera L      8 351.0 264 4.22 3.170 14.50  0  1
Ferrari Dino        6 145.0 175 3.62 2.770 15.50  0  1
Maserati Bora       8 301.0 335 3.54 3.570 14.60  0  1
Volvo 142E          4 121.0 109 4.11 2.780 18.60  1  1
```

```
mtcars[,2:9]
```

```
                  cyl  disp  hp drat    wt  qsec vs am
Mazda RX4           6 160.0 110 3.90 2.620 16.46  0  1
Mazda RX4 Wag       6 160.0 110 3.90 2.875 17.02  0  1
Datsun 710          4 108.0  93 3.85 2.320 18.61  1  1
```

```
Hornet 4 Drive        6 258.0 110 3.08 3.215 19.44  1  0
Hornet Sportabout     8 360.0 175 3.15 3.440 17.02  0  0
Valiant               6 225.0 105 2.76 3.460 20.22  1  0
Duster 360            8 360.0 245 3.21 3.570 15.84  0  0
Merc 240D             4 146.7  62 3.69 3.190 20.00  1  0
Merc 230              4 140.8  95 3.92 3.150 22.90  1  0
Merc 280              6 167.6 123 3.92 3.440 18.30  1  0
Merc 280C             6 167.6 123 3.92 3.440 18.90  1  0
Merc 450SE            8 275.8 180 3.07 4.070 17.40  0  0
Merc 450SL            8 275.8 180 3.07 3.730 17.60  0  0
Merc 450SLC           8 275.8 180 3.07 3.780 18.00  0  0
Cadillac Fleetwood    8 472.0 205 2.93 5.250 17.98  0  0
Lincoln Continental   8 460.0 215 3.00 5.424 17.82  0  0
Chrysler Imperial     8 440.0 230 3.23 5.345 17.42  0  0
Fiat 128              4  78.7  66 4.08 2.200 19.47  1  1
Honda Civic           4  75.7  52 4.93 1.615 18.52  1  1
Toyota Corolla        4  71.1  65 4.22 1.835 19.90  1  1
Toyota Corona         4 120.1  97 3.70 2.465 20.01  1  0
Dodge Challenger      8 318.0 150 2.76 3.520 16.87  0  0
AMC Javelin           8 304.0 150 3.15 3.435 17.30  0  0
Camaro Z28            8 350.0 245 3.73 3.840 15.41  0  0
Pontiac Firebird      8 400.0 175 3.08 3.845 17.05  0  0
Fiat X1-9             4  79.0  66 4.08 1.935 18.90  1  1
Porsche 914-2         4 120.3  91 4.43 2.140 16.70  0  1
Lotus Europa          4  95.1 113 3.77 1.513 16.90  1  1
Ford Pantera L        8 351.0 264 4.22 3.170 14.50  0  1
Ferrari Dino          6 145.0 175 3.62 2.770 15.50  0  1
Maserati Bora         8 301.0 335 3.54 3.570 14.60  0  1
Volvo 142E            4 121.0 109 4.11 2.780 18.60  1  1
```

```
mtcars[2,9]
```

```
[1] 1
```

```
mtcars[2:9,]
```

```
                  mpg cyl  disp  hp drat    wt  qsec vs am gear carb
Mazda RX4 Wag     21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
Datsun 710        22.8   4 108.0  93 3.85 2.320 18.61  1  1    4    1
Hornet 4 Drive    21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
Hornet Sportabout 18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
Valiant           18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
Duster 360        14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
Merc 240D         24.4   4 146.7  62 3.69 3.190 20.00  1  0    4    2
Merc 230          22.8   4 140.8  95 3.92 3.150 22.90  1  0    4    2
                         car_brand
Mazda RX4 Wag         Mazda RX4 Wag
Datsun 710               Datsun 710
Hornet 4 Drive       Hornet 4 Drive
Hornet Sportabout Hornet Sportabout
Valiant                     Valiant
Duster 360               Duster 360
Merc 240D                 Merc 240D
Merc 230                   Merc 230
```

```
# select rows 20 to 30 and columns 8 to 10
mtcars[c(20:30),c(8:10)]
```

```
                vs am gear
Toyota Corolla   1  1    4
Toyota Corona    1  0    3
Dodge Challenger 0  0    3
AMC Javelin      0  0    3
Camaro Z28       0  0    3
Pontiac Firebird 0  0    3
Fiat X1-9        1  1    4
```

```
Porsche 914-2     0  1    5
Lotus Europa      1  1    5
Ford Pantera L    0  1    5
Ferrari Dino      0  1    5
```

```
mtcars[20:30,8:10]
```

```
                 vs am gear
Toyota Corolla    1  1    4
Toyota Corona     1  0    3
Dodge Challenger  0  0    3
AMC Javelin       0  0    3
Camaro Z28        0  0    3
Pontiac Firebird  0  0    3
Fiat X1-9         1  1    4
Porsche 914-2     0  1    5
Lotus Europa      1  1    5
Ford Pantera L    0  1    5
Ferrari Dino      0  1    5
```

```
# select row 5 only
mtcars[5,]
```

```
                   mpg cyl disp  hp drat   wt  qsec vs am gear carb
Hornet Sportabout 18.7   8  360 175 3.15 3.44 17.02  0  0    3    2
                         car_brand
Hornet Sportabout Hornet Sportabout
```

```
# select column 2 and 6 only
mtcars[c(2,6)]
```

```
                   cyl    wt
Mazda RX4            6 2.620
Mazda RX4 Wag        6 2.875
Datsun 710           4 2.320
Hornet 4 Drive       6 3.215
Hornet Sportabout    8 3.440
Valiant              6 3.460
Duster 360           8 3.570
Merc 240D            4 3.190
Merc 230             4 3.150
Merc 280             6 3.440
Merc 280C            6 3.440
Merc 450SE           8 4.070
Merc 450SL           8 3.730
Merc 450SLC          8 3.780
Cadillac Fleetwood   8 5.250
Lincoln Continental  8 5.424
Chrysler Imperial    8 5.345
Fiat 128             4 2.200
Honda Civic          4 1.615
Toyota Corolla       4 1.835
Toyota Corona        4 2.465
Dodge Challenger     8 3.520
AMC Javelin          8 3.435
Camaro Z28           8 3.840
Pontiac Firebird     8 3.845
Fiat X1-9            4 1.935
Porsche 914-2        4 2.140
Lotus Europa         4 1.513
Ford Pantera L       8 3.170
Ferrari Dino         6 2.770
Maserati Bora        8 3.570
Volvo 142E           4 2.780
```

```
mtcars[,c(2,6)]
```

```
                   cyl    wt
Mazda RX4            6 2.620
Mazda RX4 Wag        6 2.875
Datsun 710           4 2.320
Hornet 4 Drive       6 3.215
Hornet Sportabout    8 3.440
Valiant              6 3.460
Duster 360           8 3.570
Merc 240D            4 3.190
Merc 230             4 3.150
Merc 280             6 3.440
Merc 280C            6 3.440
Merc 450SE           8 4.070
Merc 450SL           8 3.730
Merc 450SLC          8 3.780
Cadillac Fleetwood   8 5.250
Lincoln Continental  8 5.424
Chrysler Imperial    8 5.345
Fiat 128             4 2.200
Honda Civic          4 1.615
Toyota Corolla       4 1.835
Toyota Corona        4 2.465
Dodge Challenger     8 3.520
AMC Javelin          8 3.435
Camaro Z28           8 3.840
Pontiac Firebird     8 3.845
Fiat X1-9            4 1.935
Porsche 914-2        4 2.140
Lotus Europa         4 1.513
Ford Pantera L       8 3.170
Ferrari Dino         6 2.770
Maserati Bora        8 3.570
Volvo 142E           4 2.780
```

```
mtcars[c(2,6),]
```

```
              mpg cyl disp  hp drat    wt  qsec vs am gear carb     car_brand
Mazda RX4 Wag 21.0   6  160 110 3.90 2.875 17.02  0  1    4    4 Mazda RX4 Wag
Valiant       18.1   6  225 105 2.76 3.460 20.22  1  0    3    1       Valiant
```

```
# select columns 2 to 6
mtcars[2:6]
```

```
                   cyl  disp  hp drat    wt
Mazda RX4            6 160.0 110 3.90 2.620
Mazda RX4 Wag        6 160.0 110 3.90 2.875
Datsun 710           4 108.0  93 3.85 2.320
Hornet 4 Drive       6 258.0 110 3.08 3.215
Hornet Sportabout    8 360.0 175 3.15 3.440
Valiant              6 225.0 105 2.76 3.460
Duster 360           8 360.0 245 3.21 3.570
Merc 240D            4 146.7  62 3.69 3.190
Merc 230             4 140.8  95 3.92 3.150
Merc 280             6 167.6 123 3.92 3.440
Merc 280C            6 167.6 123 3.92 3.440
Merc 450SE           8 275.8 180 3.07 4.070
Merc 450SL           8 275.8 180 3.07 3.730
Merc 450SLC          8 275.8 180 3.07 3.780
Cadillac Fleetwood   8 472.0 205 2.93 5.250
Lincoln Continental  8 460.0 215 3.00 5.424
Chrysler Imperial    8 440.0 230 3.23 5.345
Fiat 128             4  78.7  66 4.08 2.200
Honda Civic          4  75.7  52 4.93 1.615
Toyota Corolla       4  71.1  65 4.22 1.835
```

```
Toyota Corona         4 120.1  97 3.70 2.465
Dodge Challenger      8 318.0 150 2.76 3.520
AMC Javelin           8 304.0 150 3.15 3.435
Camaro Z28            8 350.0 245 3.73 3.840
Pontiac Firebird      8 400.0 175 3.08 3.845
Fiat X1-9             4  79.0  66 4.08 1.935
Porsche 914-2         4 120.3  91 4.43 2.140
Lotus Europa          4  95.1 113 3.77 1.513
Ford Pantera L        8 351.0 264 4.22 3.170
Ferrari Dino          6 145.0 175 3.62 2.770
Maserati Bora         8 301.0 335 3.54 3.570
Volvo 142E            4 121.0 109 4.11 2.780
```

```r
# select all the cars that have 4 cylinders
m <- mtcars[mtcars$cyl == 4,]
length(m)
```

```
[1] 12
```

```r
length(mtcars[,1])
```

```
[1] 32
```

```r
nrow(mtcars)
```

```
[1] 32
```

```r
ncol(mtcars)
```

```
[1] 12
```

```r
# how many cars have automatic transmission
nrow(mtcars[mtcars$am == 0,])
```

```
[1] 19
```

```r
# select "Fiat 128" row by its rowname
# mtcars["Fiat 128"]
mtcars["Fiat 128",]
```

```
          mpg cyl disp hp drat  wt  qsec vs am gear carb car_brand
Fiat 128 32.4   4 78.7 66 4.08 2.2 19.47  1  1    4    1  Fiat 128
```

```r
mean(c(40,60,40,60))
```

```
[1] 50
```

```r
mtcars[mtcars$mpg < mean(mtcars$mpg),]
```

```
                   mpg cyl  disp  hp drat    wt  qsec vs am gear carb
Hornet Sportabout  18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
Valiant            18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
Duster 360         14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
Merc 280           19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
Merc 280C          17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
Merc 450SE         16.4   8 275.8 180 3.07 4.070 17.40  0  0    3    3
Merc 450SL         17.3   8 275.8 180 3.07 3.730 17.60  0  0    3    3
Merc 450SLC        15.2   8 275.8 180 3.07 3.780 18.00  0  0    3    3
Cadillac Fleetwood 10.4   8 472.0 205 2.93 5.250 17.98  0  0    3    4
Lincoln Continental 10.4  8 460.0 215 3.00 5.424 17.82  0  0    3    4
Chrysler Imperial  14.7   8 440.0 230 3.23 5.345 17.42  0  0    3    4
Dodge Challenger   15.5   8 318.0 150 2.76 3.520 16.87  0  0    3    2
AMC Javelin        15.2   8 304.0 150 3.15 3.435 17.30  0  0    3    2
Camaro Z28         13.3   8 350.0 245 3.73 3.840 15.41  0  0    3    4
```

```
Pontiac Firebird    19.2   8 400.0 175 3.08 3.845 17.05  0  0    3    2
Ford Pantera L      15.8   8 351.0 264 4.22 3.170 14.50  0  1    5    4
Ferrari Dino        19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
Maserati Bora       15.0   8 301.0 335 3.54 3.570 14.60  0  1    5    8
                             car_brand
Hornet Sportabout     Hornet Sportabout
Valiant                         Valiant
Duster 360                   Duster 360
Merc 280                       Merc 280
Merc 280C                     Merc 280C
Merc 450SE                   Merc 450SE
Merc 450SL                   Merc 450SL
Merc 450SLC                 Merc 450SLC
Cadillac Fleetwood   Cadillac Fleetwood
Lincoln Continental Lincoln Continental
Chrysler Imperial     Chrysler Imperial
Dodge Challenger       Dodge Challenger
AMC Javelin                 AMC Javelin
Camaro Z28                   Camaro Z28
Pontiac Firebird       Pontiac Firebird
Ford Pantera L           Ford Pantera L
Ferrari Dino               Ferrari Dino
Maserati Bora             Maserati Bora
```

Select a single row

```r
# print(): displays the supplied value or object.
# A data frame uses [row, column] indexing to select records and variables.
# A blank row or column position keeps all entries in that dimension.
first_row <- mtcars[1, ]
print(first_row)
```

```
          mpg cyl disp  hp drat   wt  qsec vs am gear carb car_brand
Mazda RX4  21   6  160 110  3.9 2.62 16.46  0  1    4    4 Mazda RX4
```

Select a specific element

```r
# print(): displays the supplied value or object.
# A data frame uses [row, column] indexing to select records and variables.
# A blank row or column position keeps all entries in that dimension.
# Accessing the Age of the second person
mpg <- mtcars[2, "mpg"]
print(mpg)
```

```
[1] 21
```

Using `subset()` Function

The **`subset()`** function can be used for more complex subsetting based on conditions.

- **Select rows where Age is greater than 25**:

  ```r
  # print(): displays the supplied value or object.
  # subset() selects observations satisfying the supplied condition.
  # Conditions refer to column names within the selected data frame; check missing
   ↪  values separately.
  more_than_4_cyl <- subset(mtcars, cyl > 4)
  print(more_than_4_cyl)
  ```

  ```
                     mpg cyl  disp  hp drat    wt  qsec vs am gear carb
  Mazda RX4         21.0   6 160.0 110 3.90 2.620 16.46  0  1    4    4
  Mazda RX4 Wag     21.0   6 160.0 110 3.90 2.875 17.02  0  1    4    4
  Hornet 4 Drive    21.4   6 258.0 110 3.08 3.215 19.44  1  0    3    1
  Hornet Sportabout 18.7   8 360.0 175 3.15 3.440 17.02  0  0    3    2
  Valiant           18.1   6 225.0 105 2.76 3.460 20.22  1  0    3    1
  Duster 360        14.3   8 360.0 245 3.21 3.570 15.84  0  0    3    4
  Merc 280          19.2   6 167.6 123 3.92 3.440 18.30  1  0    4    4
  Merc 280C         17.8   6 167.6 123 3.92 3.440 18.90  1  0    4    4
  ```

```
Merc 450SE          16.4   8 275.8 180 3.07 4.070 17.40  0  0    3    3
Merc 450SL          17.3   8 275.8 180 3.07 3.730 17.60  0  0    3    3
Merc 450SLC         15.2   8 275.8 180 3.07 3.780 18.00  0  0    3    3
Cadillac Fleetwood  10.4   8 472.0 205 2.93 5.250 17.98  0  0    3    4
Lincoln Continental 10.4   8 460.0 215 3.00 5.424 17.82  0  0    3    4
Chrysler Imperial   14.7   8 440.0 230 3.23 5.345 17.42  0  0    3    4
Dodge Challenger    15.5   8 318.0 150 2.76 3.520 16.87  0  0    3    2
AMC Javelin         15.2   8 304.0 150 3.15 3.435 17.30  0  0    3    2
Camaro Z28          13.3   8 350.0 245 3.73 3.840 15.41  0  0    3    4
Pontiac Firebird    19.2   8 400.0 175 3.08 3.845 17.05  0  0    3    2
Ford Pantera L      15.8   8 351.0 264 4.22 3.170 14.50  0  1    5    4
Ferrari Dino        19.7   6 145.0 175 3.62 2.770 15.50  0  1    5    6
Maserati Bora       15.0   8 301.0 335 3.54 3.570 14.60  0  1    5    8
                              car_brand
Mazda RX4                     Mazda RX4
Mazda RX4 Wag             Mazda RX4 Wag
Hornet 4 Drive           Hornet 4 Drive
Hornet Sportabout     Hornet Sportabout
Valiant                         Valiant
Duster 360                   Duster 360
Merc 280                       Merc 280
Merc 280C                     Merc 280C
Merc 450SE                   Merc 450SE
Merc 450SL                   Merc 450SL
Merc 450SLC                 Merc 450SLC
Cadillac Fleetwood   Cadillac Fleetwood
Lincoln Continental Lincoln Continental
Chrysler Imperial     Chrysler Imperial
Dodge Challenger       Dodge Challenger
AMC Javelin                 AMC Javelin
Camaro Z28                   Camaro Z28
Pontiac Firebird       Pontiac Firebird
Ford Pantera L           Ford Pantera L
Ferrari Dino               Ferrari Dino
Maserati Bora             Maserati Bora
```

Filtering data frames

```r
# c(): combines values into a vector, converting them to a common type when needed.
# print(): displays the supplied value or object.
# data.frame() combines equally aligned vectors into a rectangular table.
# Columns describe variables and rows describe observations; retain the meaning and units
 ↪  of each column.
# Example of creating a data.frame
name <- c("John", "Alice", "Bob", "Eva")
age <- c(23, 25, 30, 22)
gender <- c("Male", "Female", "Male", "Female")

# Combine vectors into a data.frame
students <- data.frame(name, age, gender)

# Display the data.frame
print(students)
```

```
   name age gender
1  John  23   Male
2 Alice  25 Female
3   Bob  30   Male
4   Eva  22 Female
```

```r
# Accessing a column using $
print(students$age)
```

```
[1] 23 25 30 22
```

```r
# Accessing a column using indexing
print(students["gender"])
```

```
  gender
1   Male
2 Female
3   Male
4 Female
```

```r
# Access the element in the 2nd row, 1st column
print(students[2, 1])
```

```
[1] "Alice"
```

```r
# Access the entire 3rd row
print(students[3, ])
```

```
  name age gender
3  Bob  30   Male
```

```r
# Access the entire 2nd column
print(students[, 2])
```

```
[1] 23 25 30 22
```

```r
# Filter rows where age is greater than 23
older_students <- students[students$age > 23, ]

# Display the filtered data.frame
print(older_students)
```

```
   name age gender
2 Alice  25 Female
3   Bob  30   Male
```

Miscellaneous Related Topics - عيضاوم ةقرفتم ذات ةقلع

Printing Output

In R, you have several functions at your disposal to print output. Here are the most commonly used ones:

1. **`print()`** - This is the basic R function to print objects to the console. It's a generic function, which means that it can have different methods for different object types.

   ```r
   # print() displays the supplied value or the contents of a saved object.
   # Use <- to assign a result to a name before reusing it in later expressions.
   print("Hello, World!")
   ```

   ```
   [1] "Hello, World!"
   ```

2. **`cat()`** - Concatenates and prints objects. It is often used for printing character strings and does not automatically add a newline unless you specify it with `"\n"`.

   ```r
   # cat() writes values together as console output without the usual vector labels.
   # Use "\n" to insert a new line; this displays text rather than returning a joined
    ↪  character vector.
   cat("Hello,", "World!", "\n")
   ```

   ```
   Hello, World!
   ```

3. The **paste** function in R is used to concatenate strings. Unlike `print` or `cat`, `paste` does not print to the console but returns a single string that is the concatenation of its arguments. The `paste` function is particularly useful when you want to combine strings and variables to create a new string.

Here's a basic usage of `paste`:

```
# paste() joins values into character strings.
# sep controls the separator between inputs; collapse joins elements of a vector into one
 ↪  string.
# This will not print to the console but will return a string.
result <- paste("Hello", "World", sep = " ")

# To print the result, you can use print() or cat()
print(result)  # This will print "Hello World" to the console
```

```
[1] "Hello World"
```

```
cat(result, "\n")  # This will also print "Hello World", followed by a new line
```

```
Hello World
```

Requesting user input

Test this command in the console window not in quarto.

```
# readline(): reads one line of interactive console input as text; it is not suitable for
 ↪  unattended input.
# cat() writes values together as console output without the usual vector labels.
# Use "\n" to insert a new line; this displays text rather than returning a joined
 ↪  character vector.
name <- readline("Please enter your name: ")
cat("Your name is:", name)
```

Values Assignments in R

We can use different syntax to make an assignment in R.

The preferred syntax is to use the Values Gets e.g. x <- 10, which means x Gets the value 10.

We can also use the conventional assignment operator.

The Gets operator has the advantage of escaping the ambiguity of the assignment and equality operators = vs ==

```
# Assignments store a value in an object named on the receiving side of <-, ->, or =.
# The final 1 = x line is intentionally invalid because a literal number cannot receive an
 ↪  assignment.
x <- 1 # valid, x gets 1
x
```

```
[1] 1
```

```
1 -> x # valid, x gets 1
x
```

```
[1] 1
```

```r
x = 1 # valid, x = 1
x
```

```
[1] 1
```

```r
1 = x # invalid, x
```

```
Error in `1 = x`:
! invalid (do_set) left-hand side to assignment
```

List Memory, Clean Memory

```r
# rm() removes named objects from the current R environment.
# ls() lists object names; rm(list=ls()) clears them, so use that form only deliberately.
# list variables
x <- 1
y <- 2
ls()
```

```
 [1] "a"                 "age"               "ages"
 [4] "answers"           "array_data"        "b"
 [7] "banana_count"      "BMI"               "c"
[10] "car_names"         "colors"            "combined_seq"
[13] "complex_num"       "count"             "d"
[16] "datsun_710"        "de_vectorized_time" "df"
[19] "exclude_range"     "exclude_second"    "filtered_numbers"
[22] "first_car_name"    "first_row"         "fruits"
[25] "gender"            "i"                 "is_valid"
[28] "list_data"         "m"                 "matrix_data"
[31] "miles_per_hour"    "more_than_4_cyl"   "mpg"
[34] "mtcars"            "myString"          "N"
[37] "name"              "numbers"           "older_students"
[40] "pandoc_dir"        "quarto_bin_path"   "range_numbers"
[43] "rep_each"          "rep_times"         "result"
[46] "selected_numbers"  "sentence"          "seq_1_to_10"
[49] "seq_by_2"          "seq_length_out"    "some_fruits"
[52] "students"          "subset_df"         "test_result"
[55] "third_number"      "vectorized_time"   "weight"
[58] "x"                 "y"
```

```r
# clear specific values
rm(x,y)
ls()
```

```
 [1] "a"                 "age"               "ages"
 [4] "answers"           "array_data"        "b"
 [7] "banana_count"      "BMI"               "c"
[10] "car_names"         "colors"            "combined_seq"
[13] "complex_num"       "count"             "d"
[16] "datsun_710"        "de_vectorized_time" "df"
[19] "exclude_range"     "exclude_second"    "filtered_numbers"
[22] "first_car_name"    "first_row"         "fruits"
[25] "gender"            "i"                 "is_valid"
[28] "list_data"         "m"                 "matrix_data"
[31] "miles_per_hour"    "more_than_4_cyl"   "mpg"
[34] "mtcars"            "myString"          "N"
[37] "name"              "numbers"           "older_students"
[40] "pandoc_dir"        "quarto_bin_path"   "range_numbers"
[43] "rep_each"          "rep_times"         "result"
[46] "selected_numbers"  "sentence"          "seq_1_to_10"
[49] "seq_by_2"          "seq_length_out"    "some_fruits"
[52] "students"          "subset_df"         "test_result"
[55] "third_number"      "vectorized_time"   "weight"
```

```r
# clear all values
rm(list = ls())
ls()
```

```
character(0)
```

- We can also click on the “Broom” icon in the Environment tab to clear all objects.
- Alternatively, use the **`Session > Clear Workspace...`** menu option.
- Finally, restarting the R session is a surefire way to clear all objects and start fresh. In RStudio, you can do this via the menu **`Session > Restart R`** or by using the **`Ctrl+Shift+F10`** shortcut on Windows/Linux or **`Cmd+Shift+F10`** on macOS.

Flow of Control

Repetition in R (Loops)

Loops are fundamental constructs in programming that allow you to execute a block of code repeatedly. In R, the primary looping constructs are the `for` loop, the `while` loop, and the `repeat` loop. This section will guide you through the syntax and usage of each, with examples to illustrate how they can be used in data analysis tasks.

The `for` Loop

The `for` loop is used to iterate over a sequence or a vector and execute a block of code for each element.

Syntax:

```r
# A for loop repeats its body once for each value in the supplied sequence.
# The loop variable identifies the current value or position; vectorized arithmetic can
 ↪  often replace the loop.
for (variable in sequence) {
  # Code to execute for each element of the sequence
}
```

Example:

Let’s iterate over a vector of numbers and print each number squared.

```r
# print(): displays the supplied value or object.
# A for loop repeats its body once for each value in the supplied sequence.
# The loop variable identifies the current value or position; vectorized arithmetic can
 ↪  often replace the loop.
# Create a vector of numbers
numbers <- 1:5

# Use a for loop to iterate and print squares
for (num in numbers) {
  print(num^2)
}
```

```
[1] 1
[1] 4
[1] 9
[1] 16
[1] 25
```

What are the correct values for the *s in the code below so that it will iterate over all the items of x array?

```
# This unfinished loop is a practice template: replace *:* with suitable integer
 ↪  positions.
# Use x[i] to select each position and keep the exercise unevaluated until the loop is
 ↪  complete.
x <- 50:60
for (i in *:*)
{
  print(x[i])
}
```

In this example, the loop runs five times, squaring each number from 1 to 5.

The `while` Loop

The `while` loop continues to execute as long as a specified condition is true. It's useful when the number of iterations is not known beforehand.

Syntax:

```
# while() repeats its body as long as its condition remains TRUE.
# Update the value used in the condition so a running example can eventually stop.
while (condition) {
  # Code to execute as long as the condition is true
}
```

Example:

We will use a `while` loop to simulate a countdown.

```
# print(): displays the supplied value or object.
# paste(): joins text using sep between pieces; collapse combines a vector into one
 ↪  string.
# while() repeats its body as long as its condition remains TRUE.
# Update the value used in the condition so a running example can eventually stop.
# Set the starting point of the countdown
countdown <- 5

# Begin the while loop
while (countdown > 0) {
  print(paste("Counting down:", countdown))
  countdown <- countdown - 1
}
```

```
[1] "Counting down: 5"
[1] "Counting down: 4"
[1] "Counting down: 3"
[1] "Counting down: 2"
[1] "Counting down: 1"
```

```
# When the loop ends
print("Lift off!")
```

```
[1] "Lift off!"
```

This loop will run until `countdown` is no longer greater than 0.

The repeat Loop

The repeat loop executes an indefinite number of times until explicitly told to stop with a break statement. It's the most basic form of loop and is less commonly used than for and while loops due to its potential to create infinite loops if not handled carefully.

Syntax:

```
# repeat executes its body until an explicit break exits the loop.
# The if condition decides when to stop; update the working value on each unsuccessful
 ↪  pass.
repeat {
  # Code to execute indefinitely
  if (stop_condition) {
    break
  }
}
```

Example:

Here's a repeat loop that finds the first number divisible by 13 greater than 100.

```
# print(): displays the supplied value or object.
# paste(): joins text using sep between pieces; collapse combines a vector into one
 ↪  string.
# repeat executes its body until an explicit break exits the loop.
# The if condition decides when to stop; update the working value on each unsuccessful
 ↪  pass.
# Initialize the number
number <- 101

# Start the repeat loop
repeat {
  # Check if the number is divisible by 13
  if (number %% 13 == 0) {
    print(paste("Found a number divisible by 13:", number))
    break
  }
  # Increment the number
  number <- number + 1
}
```

```
[1] "Found a number divisible by 13: 104"
```

The loop runs indefinitely, incrementing number until it finds one that satisfies the condition.

Nested Loops

Loops can be nested inside other loops, which is useful for iterating over multiple dimensions, such as rows and columns in a matrix.

Example:

```
# matrix(): arranges values in rows and columns; byrow=TRUE fills rows rather than the
 ↪  default columns.
# nrow(): returns the number of rows in a table or matrix.
# ncol(): returns the number of columns in a table or matrix.
# print(): displays the supplied value or object.
# A for loop repeats its body once for each value in the supplied sequence.
```

```r
# The loop variable identifies the current value or position; vectorized arithmetic can
 ↪  often replace the loop.
# Create a 3x3 matrix
my_matrix <- matrix(1:9, nrow = 3)

# Iterate over each element of the matrix
for (row in 1:nrow(my_matrix)) {
  for (col in 1:ncol(my_matrix)) {
    # Multiply each element by 2
    my_matrix[row, col] <- my_matrix[row, col] * 2
  }
}
print(my_matrix)
```

```
     [,1] [,2] [,3]
[1,]    2    8   14
[2,]    4   10   16
[3,]    6   12   18
```

In this nested loop example, every element of `my_matrix` is doubled.

Conditional Branching in R

Introduction to Conditional Branching

Conditional branching in R allows you to control the flow of execution based on conditions. It's a fundamental concept in programming that lets you execute certain sections of code while skipping others, depending on whether specified conditions are TRUE or FALSE. This section will explain how to use conditional statements in R, such as `if`, `else`, and `else if`, along with practical examples.

The `if` Statement

The `if` statement is the simplest form of conditional branching. It evaluates a condition and, if that condition is TRUE, it executes a block of code.

Syntax:

```r
# if evaluates a condition and runs its body when the condition is TRUE.
# else or else if handles alternative cases; replace placeholder conditions before
 ↪  running a template.
if (condition) {
  # Code to execute if the condition is TRUE
}
```

Example:

Here, we check if a number is positive.

```r
# print(): displays the supplied value or object.
# if evaluates a condition and runs its body when the condition is TRUE.
# else or else if handles alternative cases; replace placeholder conditions before
 ↪  running a template.
number <- 5

if (number > 0) {
  print("The number is positive.")
}
```

```
[1] "The number is positive."
```

The `else` Statement

The `else` statement is used together with an `if` statement to execute code when the `if` condition is not TRUE.

Syntax:

```
# if evaluates a condition and runs its body when the condition is TRUE.
# else or else if handles alternative cases; replace placeholder conditions before
 ↪  running a template.
if (condition) {
  # Code to execute if the condition is TRUE
} else {
  # Code to execute if the condition is FALSE
}
```

Example:

Expanding on the previous example, we also handle the case when the number is not positive.

```
# print(): displays the supplied value or object.
# if evaluates a condition and runs its body when the condition is TRUE.
# else or else if handles alternative cases; replace placeholder conditions before
 ↪  running a template.
number <- -3

if (number > 0) {
  print("The number is positive.")
} else {
  print("The number is not positive.")
}
```

```
[1] "The number is not positive."
```

The `else if` Statement

The `else if` statement allows you to check multiple conditions. If the condition for `if` is FALSE, it checks the condition for `else if`, and so on.

Syntax:

```
# if evaluates a condition and runs its body when the condition is TRUE.
# else or else if handles alternative cases; replace placeholder conditions before
 ↪  running a template.
if (condition1) {
  # Code to execute if condition1 is TRUE
} else if (condition2) {
  # Code to execute if condition1 is FALSE and condition2 is TRUE
} else {
  # Code to execute if none of the above conditions are TRUE
}
```

Example:

Here, we classify a number as positive, negative, or zero.

```
# print(): displays the supplied value or object.
# if evaluates a condition and runs its body when the condition is TRUE.
# else or else if handles alternative cases; replace placeholder conditions before
 ↪  running a template.
number <- 0

if (number > 0) {
  print("The number is positive.")
} else if (number < 0) {
  print("The number is negative.")
} else {
  print("The number is zero.")
}
```

```
[1] "The number is zero."
```

The `switch` Statement

The `switch` statement is a multi-way branch used when you want to compare the same variable (or expression) with many different possible values and execute different pieces of code for each value.

Syntax:

```
# switch() chooses a branch using the supplied expression or character label.
# The final unnamed branch can provide a fallback when none of the named labels matches.
switch(EXPR,
       "case1" = {# Code for case 1},
       "case2" = {# Code for case 2},
       # ...,
       {# Default case code}
)
```

Example:

The following example shows how `switch` can be used to print different messages based on the value of a character variable.

```
# print(): displays the supplied value or object.
# switch() chooses a branch using the supplied expression or character label.
# The final unnamed branch can provide a fallback when none of the named labels matches.
day <- "Tue"

switch(day,
       "Mon" = {print("Start of the work week")},
       "Tue" = {print("Second day of the work week")},
       "Wed" = {print("Middle of the work week")},
       "Thu" = {print("Approaching the end")},
       "Fri" = {print("Last day of the work week")},
       {print("It must be the weekend")}
)
```

```
[1] "Second day of the work week"
```

Creating Functions in R

Introduction

Functions in R are crucial for efficient and organized code, allowing for repetitive tasks to be encapsulated into single, callable entities. This chapter demonstrates how to craft your own functions, enabling the automation of tasks and making your code more modular and readable.

Basic Function Structure

A function is defined using the `function` keyword, followed by parentheses containing any arguments, and a body enclosed in curly braces.

Example: Simple Addition Function

```
# print(): displays the supplied value or object.
# add_two_numbers(): calls the function defined in this lesson to return the sum of its
 ↪  two inputs.
# function() defines a reusable calculation with named input arguments.
# The body computes a result for those inputs; return() explicitly chooses the value
 ↪  passed back.
add_two_numbers <- function(a, b) {
  result <- a + b
  return(result)
}

# Using the function
print(add_two_numbers(5, 3))
```

```
[1] 8
```

Function Arguments and Return Values

Functions can take arguments and return results. R functions return the result of the last line in the function body by default, but the `return()` function can be used for clarity or early returns.

Example: Greeting Function

```
# paste(): joins text using sep between pieces; collapse combines a vector into one
 ↪  string.
# print(): displays the supplied value or object.
# greet_person(): calls the locally defined function to format a greeting from a name.
# function() defines a reusable calculation with named input arguments.
# The body computes a result for those inputs; return() explicitly chooses the value
 ↪  passed back.
greet_person <- function(name) {
  greeting <- paste("Hello,", name)
  return(greeting)
}

# Using the function
print(greet_person("Alice"))
```

```
[1] "Hello, Alice"
```

Default Argument Values

You can specify default values for arguments, making them optional.

Example: Power Function

```
# print(): displays the supplied value or object.
# raise_to_power(): calls the locally defined function using its supplied or default
 ↪  exponent.
# function() defines a reusable calculation with named input arguments.
# The body computes a result for those inputs; return() explicitly chooses the value
 ↪  passed back.
raise_to_power <- function(base, exponent = 2) {
  result <- base ^ exponent
  return(result)
}

# Using the function with default exponent
print(raise_to_power(4))
```

```
[1] 16
```

```
# Specifying both arguments
print(raise_to_power(4, 3))
```

```
[1] 64
```

Returning Multiple Values

Functions in R can return multiple values using a list.

Example: Statistics Function

```
# mean(): computes an average; logical TRUE/FALSE values contribute 1/0, giving a
 ↪  proportion.
# sum(): adds vector entries; on logical values it counts TRUE entries.
# list(): stores named or unnamed objects that may have different types and lengths.
# calculate_stats(): calls the locally defined function to return the summaries listed in
 ↪  its body.
# c(): combines values into a vector, converting them to a common type when needed.
# print(): displays the supplied value or object.
# function() defines a reusable calculation with named input arguments.
# The body computes a result for those inputs; return() explicitly chooses the value
 ↪  passed back.
calculate_stats <- function(numbers) {
  mean_val <- mean(numbers)
  sum_val <- sum(numbers)
  return(list(mean = mean_val, sum = sum_val))
}

# Using the function
stats <- calculate_stats(c(1, 2, 3, 4, 5))
print(stats)
```

```
$mean
[1] 3

$sum
[1] 15
```

Anonymous Functions

For short, one-time operations, anonymous functions can be used.

Example: Doubling Numbers

```
# print(): displays the supplied value or object.
# sapply() applies a calculation to each input element and simplifies compatible results
 ↪  to a vector.
# function(x) x * 2 defines the operation: double each element without changing the input
 ↪  vector.
numbers <- 1:5
doubled_numbers <- sapply(numbers, function(x) x * 2)
print(doubled_numbers)
```

```
[1]  2  4  6  8 10
```

Scope and Environment

Variables defined inside a function are local to that function.

Example: Increment Function

```
# increment(): calls the locally defined function to increase its input as specified in
 ↪  its body.
# print(): displays the supplied value or object.
# function() defines a reusable calculation with named input arguments.
# The body computes a result for those inputs; return() explicitly chooses the value
 ↪  passed back.
increment <- function(x) {
  y <- x + 1
  return(y)
}

incremented_value <- increment(5)
print(incremented_value)
```

```
[1] 6
```

```
# Trying to print 'y' here would result in an error because 'y' is not defined in this
 ↪  scope.
```

Module 1: Data Manipulation with dplyr

Lesson 3 | Prepare and summarize the textbook data

Table of contents

Purpose and data 1
Select columns 2
Filter rows 2
Pipes and readable steps 3
Arrange and create variables 3
Count categories and summarize groups 4
Missing values and duplicates 5
Join on a key 5
Reshape a summary 6
Export a derived result 6

Reduced teaching slides (PDF) · Module overview

This extended lesson provides the full explanation and reproducible examples behind the slides.

Purpose and data

Data analysis starts with a clear question and a well-defined table. This lesson uses the checked Restaurant CSV that we will revisit in Module 2. We focus on R operations here; the statistical interpretation comes later.

An analyst wants to select restaurants by rating and price, then summarize prices within each quality group. **One row is one restaurant**, identified by `restaurant_id`. Prices are in USD, and ratings are ordered Good, Very Good, Excellent.

يريد محلّل اختيار المطاعم حسب تقييم الجودة والسعر، ثم تلخيص الأسعار داخل كل فئة من فئات الجودة. **يمثّل كل صف مطعمًا واحدًا** يحدّده المتغيّر `restaurant_id`. الأسعار بالدولار الأمريكي، وفئات الجودة مرتبة: جيد، وجيد جدًا، وممتاز.

```
# library(): loads a package so its functions can be used in this session.
# read.csv(): reads a comma-separated file into a data frame; its first row supplies
 ↪  column names by default.
# c(): combines values into a vector, converting them to a common type when needed.
# head(): shows the first n entries or rows, six by default.
# glimpse(): shows table dimensions, column types, and example values.
# factor() gives categories explicit levels and an optional order.
# The level order controls grouping and display; ordered=TRUE records ordinal category
 ↪  order.
library(dplyr)
restaurants <- read.csv("data/module02/restaurant_ratings.csv")
restaurants$quality_rating <- factor(restaurants$quality_rating,
  levels = c("Good", "Very Good", "Excellent"), ordered = TRUE)
head(restaurants)
```

```
  restaurant_id quality_rating meal_price_usd
1             1           Good             18
2             2      Very Good             22
3             3           Good             28
4             4      Excellent             38
5             5      Very Good             33
6             6           Good             28
```

```
glimpse(restaurants)
```

```
Rows: 300
Columns: 3
$ restaurant_id  <int> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, ~
$ quality_rating <ord> Good, Very Good, Good, Excellent, Very Good, Good, Very~
$ meal_price_usd <int> 18, 22, 28, 38, 33, 28, 19, 11, 23, 13, 33, 44, 42, 34,~
```

Use the RStudio project so these paths resolve from the 20261 folder. Keep the source CSV unchanged. Assign transformed tables to new objects.

Select columns

`select()` chooses variables. Use meaningful names rather than column positions when practical.

```
# head(): shows the first n entries or rows, six by default.
# select() retains or renames the specified columns.
# It changes the variables included in the result without filtering observations.
prices <- restaurants |> select(restaurant_id, meal_price_usd)
head(prices)
```

```
  restaurant_id meal_price_usd
1             1             18
2             2             22
3             3             28
4             4             38
5             5             33
6             6             28
```

Filter rows

`filter()` retains observations meeting conditions. A comma combines conditions with AND. Use `|` for OR and `%in%` for membership in a set.

```
# head(): shows the first n entries or rows, six by default.
# nrow(): returns the number of rows in a table or matrix.
# c(): combines values into a vector, converting them to a common type when needed.
# filter() keeps rows whose conditions are TRUE; commas combine conditions with AND.
# Missing comparisons do not pass the filter; the source object stays unchanged unless
 ↪  reassigned.
selected <- restaurants |>
  filter(quality_rating == "Excellent", meal_price_usd >= 30)
head(selected)
```

```
  restaurant_id quality_rating meal_price_usd
1             4      Excellent             38
2            13      Excellent             42
3            14      Excellent             34
4            23      Excellent             32
5            24      Excellent             33
6            33      Excellent             44
```

```
nrow(selected)
```

```
[1] 50
```

```
restaurants |>
  filter(quality_rating %in% c("Good", "Very Good"), meal_price_usd < 20) |>
  head()
```

```
  restaurant_id quality_rating meal_price_usd
1             1           Good             18
2             7      Very Good             19
3             8      Very Good             11
4            10           Good             13
5            20           Good             19
6            36           Good             10
```

Missing values need deliberate treatment: a comparison that evaluates to NA will not pass `filter()`. Check for missingness before interpreting the remaining rows.

Pipes and readable steps

The base pipe |> sends one result into the next operation. Read the chain from top to bottom. `dplyr` also supports %>%; use one style consistently in a report.

```
# select(): chooses columns by name or selection rule.
# arrange(): orders rows by supplied columns; desc() reverses a column to descending
 ↪  order.
# desc(): reverses sorting order to descending inside arrange().
# head(): shows the first n entries or rows, six by default.
# filter() keeps rows whose conditions are TRUE; commas combine conditions with AND.
# Missing comparisons do not pass the filter; the source object stays unchanged unless
 ↪  reassigned.
restaurants |>
  filter(meal_price_usd >= 30) |>
  select(restaurant_id, quality_rating, meal_price_usd) |>
  arrange(desc(meal_price_usd), restaurant_id) |>
  head(5)
```

```
  restaurant_id quality_rating meal_price_usd
1            67      Very Good             48
2           170      Excellent             48
3           271      Excellent             48
4            35      Excellent             47
5           269      Excellent             47
```

Arrange and create variables

`arrange()` sorts rows. `desc()` reverses an ordering. `mutate()` creates or changes columns while retaining the rows.

```
# mutate(): creates or replaces columns while retaining the original rows.
# c(): combines values into a vector, converting them to a common type when needed.
# arrange(): orders rows by supplied columns; desc() reverses a column to descending
 ↪  order.
# head(): shows the first n entries or rows, six by default.
# cut() assigns numerical values to intervals defined by breaks.
# Check right and include.lowest to determine which interval contains a boundary value.
prepared <- restaurants |>
  mutate(price_at_least_30 = meal_price_usd >= 30,
```

```
        price_band = cut(meal_price_usd, c(10,20,30,40,50), right = FALSE)) |>
  arrange(quality_rating, meal_price_usd)
head(prepared)
```

```
  restaurant_id quality_rating meal_price_usd price_at_least_30 price_band
1            36           Good             10             FALSE    [10,20)
2            52           Good             10             FALSE    [10,20)
3            60           Good             10             FALSE    [10,20)
4           125           Good             10             FALSE    [10,20)
5           182           Good             10             FALSE    [10,20)
6           189           Good             10             FALSE    [10,20)
```

The price bands include the lower boundary and exclude the upper one. These are derived labels, not corrections to the original price.

Count categories and summarize groups

`count()` counts observations by categories. `group_by()` defines groups for subsequent operations. `summarise()` reduces each group to summary rows.

```
# count(): counts rows for each combination of supplied categories.
# n(): counts rows in the current dplyr group.
# mean(): computes an average; logical TRUE/FALSE values contribute 1/0, giving a
 ↪  proportion.
# median(): returns the middle sorted value, averaging the middle pair for even sample
 ↪  sizes.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# sum(): adds vector entries; on logical values it counts TRUE entries.
# nrow(): returns the number of rows in a table or matrix.
# summarise() reduces each group to the statistics specified here.
# group_by() defines the groups; .groups = "drop" removes grouping from the result.
restaurants |> count(quality_rating, name = "restaurants")
```

```
  quality_rating restaurants
1           Good          84
2      Very Good         150
3      Excellent          66
```

```
group_summary <- restaurants |>
  group_by(quality_rating) |>
  summarise(n = n(), mean_price_usd = mean(meal_price_usd),
            median_price_usd = median(meal_price_usd), .groups = "drop")
group_summary
```

```
# A tibble: 3 x 4
  quality_rating     n mean_price_usd median_price_usd
  <ord>          <int>          <dbl>            <dbl>
1 Good              84           19.7             19.5
2 Very Good        150           25.6             25
3 Excellent         66           34.3             34.5
```

```
stopifnot(sum(group_summary$n) == nrow(restaurants))
```

Using `.groups = "drop"` makes the result ungrouped. Otherwise, grouping metadata can affect later calculations. `ungroup()` explicitly removes grouping when needed. Do not average arbitrary numeric category codes.

Missing values and duplicates

```
# colSums(): sums each column separately.
# is.na(): marks missing values with TRUE; na.rm=TRUE elsewhere removes missing values
 ↪  only when justified.
# anyDuplicated() returns 0 when no duplicate value is found, otherwise its first
 ↪  position.
# Check identifiers for duplicate records; equal measurements alone do not imply
 ↪  duplicates.
colSums(is.na(restaurants))
```

```
 restaurant_id quality_rating meal_price_usd
             0              0              0
```

```
anyDuplicated(restaurants$restaurant_id)
```

```
[1] 0
```

A duplicate identifier and two restaurants with the same price are different issues. Investigate before dropping rows. Use `na.rm = TRUE` only when you can explain which observations are omitted and how that affects the summary.

Join on a key

A join combines tables through shared keys. This lookup is **derived from our own group summary**. Each quality category occurs once in the lookup, so a left join should preserve the 300 restaurant rows.

```
# select(): chooses columns by name or selection rule.
# stopifnot(): stops execution if any supplied condition is FALSE; successful checks
 ↪  produce no output.
# nrow(): returns the number of rows in a table or matrix.
# anyNA(): returns TRUE if any value is missing; !anyNA(...) requires no missing values.
# head(): shows the first n entries or rows, six by default.
# left_join() adds matching lookup information using the specified key.
# Repeated lookup keys can multiply rows, so check key uniqueness and the result row count.
lookup <- group_summary |>
  select(quality_rating, group_median_usd = median_price_usd)
with_group_median <- restaurants |>
  left_join(lookup, by = "quality_rating")
stopifnot(nrow(with_group_median) == nrow(restaurants),
          !anyNA(with_group_median$group_median_usd))
head(with_group_median)
```

```
  restaurant_id quality_rating meal_price_usd group_median_usd
1             1           Good             18             19.5
2             2      Very Good             22             25.0
3             3           Good             28             19.5
4             4      Excellent             38             34.5
5             5      Very Good             33             25.0
6             6           Good             28             19.5
```

Repeated keys on both sides can multiply rows. Check key uniqueness and row counts rather than assuming a join is harmless. `inner_join()` keeps matching keys; `left_join()` retains all rows from the left table.

Reshape a summary

`pivot_longer()` moves selected columns into name/value rows. `pivot_wider()` reverses that organization when the keys uniquely identify cells.

```
# library(): loads a package so its functions can be used in this session.
# select(): chooses columns by name or selection rule.
# pivot_wider(): turns name-value rows into columns using names_from and values_from.
# pivot_longer() turns selected columns into name-value rows.
# names_to and values_to name the new columns; untouched columns identify the original
 ↪  groups.
library(tidyr)
long_summary <- group_summary |>
  select(quality_rating, mean_price_usd, median_price_usd) |>
  pivot_longer(-quality_rating, names_to = "measure", values_to = "price_usd")
long_summary
```

```
# A tibble: 6 x 3
  quality_rating measure          price_usd
  <ord>          <chr>                <dbl>
1 Good           mean_price_usd        19.7
2 Good           median_price_usd      19.5
3 Very Good      mean_price_usd        25.6
4 Very Good      median_price_usd      25
5 Excellent      mean_price_usd        34.3
6 Excellent      median_price_usd      34.5
```

```
long_summary |>
  pivot_wider(names_from = measure, values_from = price_usd)
```

```
# A tibble: 3 x 3
  quality_rating mean_price_usd median_price_usd
  <ord>                   <dbl>            <dbl>
1 Good                     19.7             19.5
2 Very Good                25.6             25
3 Excellent                34.3             34.5
```

Export a derived result

Run this manually when you want a report output. Generated results belong in `_output/`, separate from input data.

```
# dir.create() prepares an output folder; write.csv() saves the derived summary as a CSV.
# row.names = FALSE omits row-number columns and leaves the original input file unchanged.
dir.create("_output", showWarnings = FALSE)
write.csv(group_summary, "_output/restaurant-price-summary.csv", row.names = FALSE)
```

Practice Lesson 3. The source CSV and verification limits are documented in the data guide.

Module 1: Visualization with ggplot2

Lesson 4 | Construct and customize plots

Table of contents

Introduction to ggplot2 1
Demonstrating Different GEOM Types 15

Reduced teaching slides (PDF) · Module overview

This extended lesson provides the full explanation and reproducible examples behind the slides.

Introduction to ggplot2

What is ggplot2?

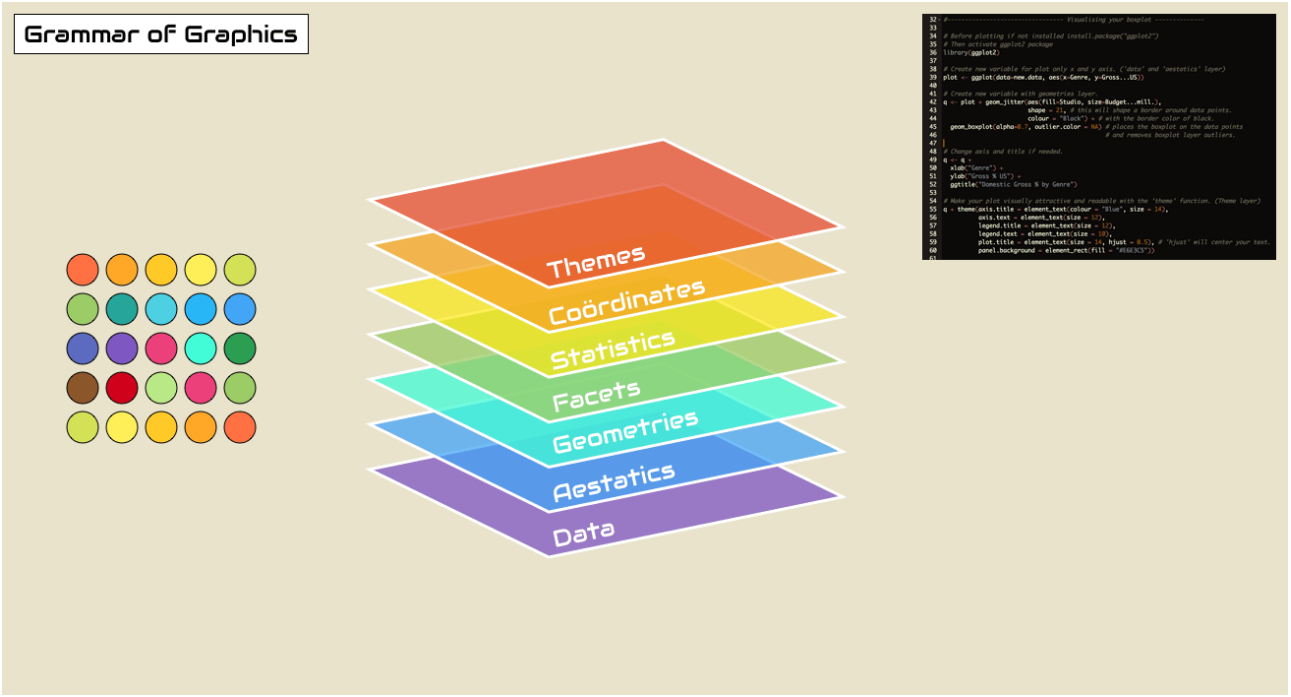
`ggplot2` is a powerful and highly versatile plotting system for R, designed to help researchers, analysts, and data scientists create meaningful and visually appealing data visualizations. Based on the principles of the Grammar of Graphics, `ggplot2` allows users to construct plots by specifying graphical primitives that represent data in a declarative way. This approach enables a clear and logical structure for creating a wide variety of charts, from simple scatter plots to complex multi-panel graphics.

At the core of `ggplot2` is the idea that a plot can be broken down into components or layers. These layers include: -

- **Data**: The actual data that you want to visualize.
- **Aesthetics (Aes)**: Mappings between the data and visual properties of the plot such as axes, colors, sizes, or shapes.
- **Geometries (Geoms)**: The geometric shapes that represent the data, such as bars in a bar chart or lines in a line chart.
- **Scales**: Control how data values are mapped to visual properties like colors or sizes.
- **Coordinate systems**: The space in which data is plotted, for instance, Cartesian or polar coordinates.
- **Faceting**: Creates grids of plots by splitting the data on one or more variables.
- **Statistical transformations**: Summarizes data in many useful ways, e.g., binning, counting, or calculating means.
- **Themes**: Control the finer details of the display like fonts, labels, legends, and background.



Grammar of Graphics: A layered approach to elegant data stories



ggplot2's layering approach makes it distinctively powerful for iterative data exploration.

You can start with a basic plot and incrementally add layers until you reach the desired level of complexity or aesthetics.

This makes it not only an effective tool for exploratory data analysis but also for creating sophisticated static and interactive graphics tailored for publications or presentations.

In the following sections, we will explore how to use ggplot2 through step-by-step instructions, building from simple examples to more complex plots, and discussing how to adjust and customize each component of a plot.

This guide will equip you with the knowledge to harness the full potential of ggplot2 in your data visualization tasks.

Step 1: Loading ggplot2 and Data

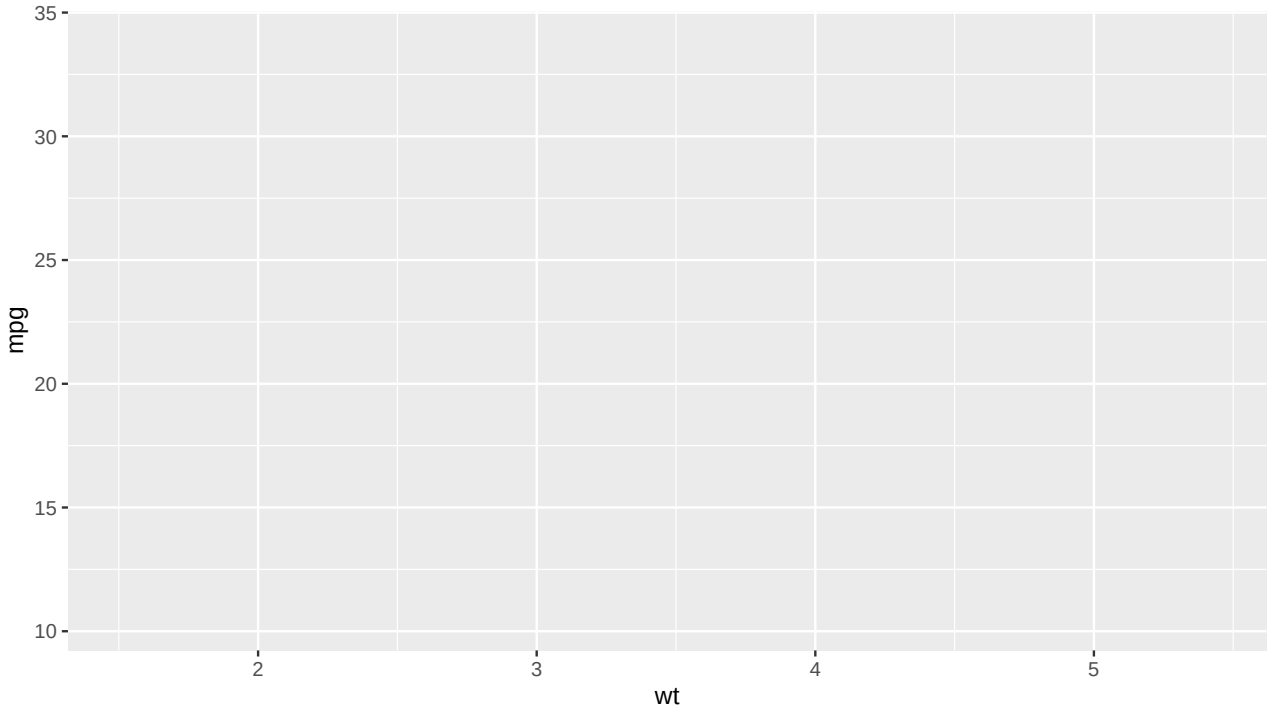
To begin working with `ggplot2`, you first need to load the package and prepare a dataset. For our examples, we will use the `mtcars` dataset which is built into R.

```
# library(ggplot2) makes its plotting functions available in this R session.
# data(mtcars) loads the built-in car dataset used in the following plotting examples.
library(ggplot2)
data(mtcars)
```

Step 2: Initiating a ggplot Object

Every ggplot2 graph begins with the `ggplot` function. It sets up a plot with data and aesthetic mappings. Aesthetics describe how variables in the data are mapped to visual properties (aesthetics) of the graph.

```
# ggplot() defines the data and aes() maps variables to visual properties such as x, y,
 ↪  and color.
# geom_point() adds paired observations when present; labs() and scale functions control
 ↪  labels and axes.
ggplot(data = mtcars, aes(x = wt, y = mpg))
```

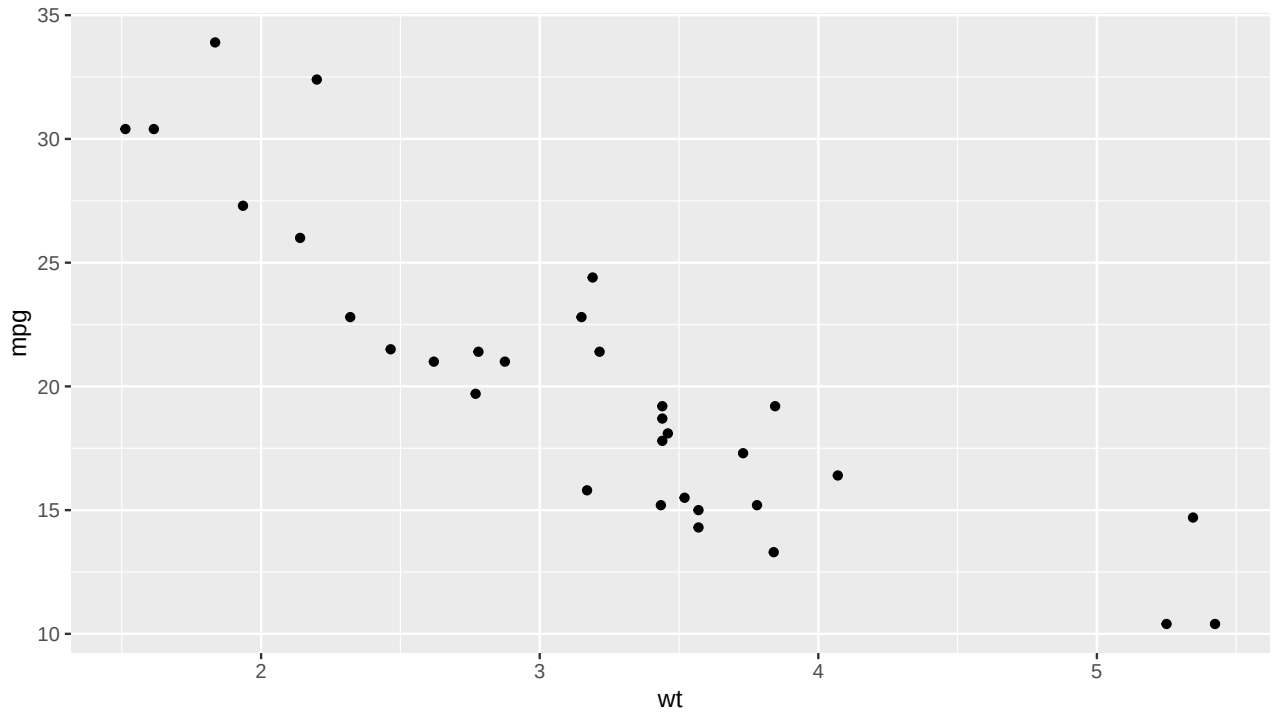


Here, `data` specifies the dataset to use, and `aes` is used to define the primary aesthetic mappings - placing car weight on the x-axis and miles per gallon on the y-axis.

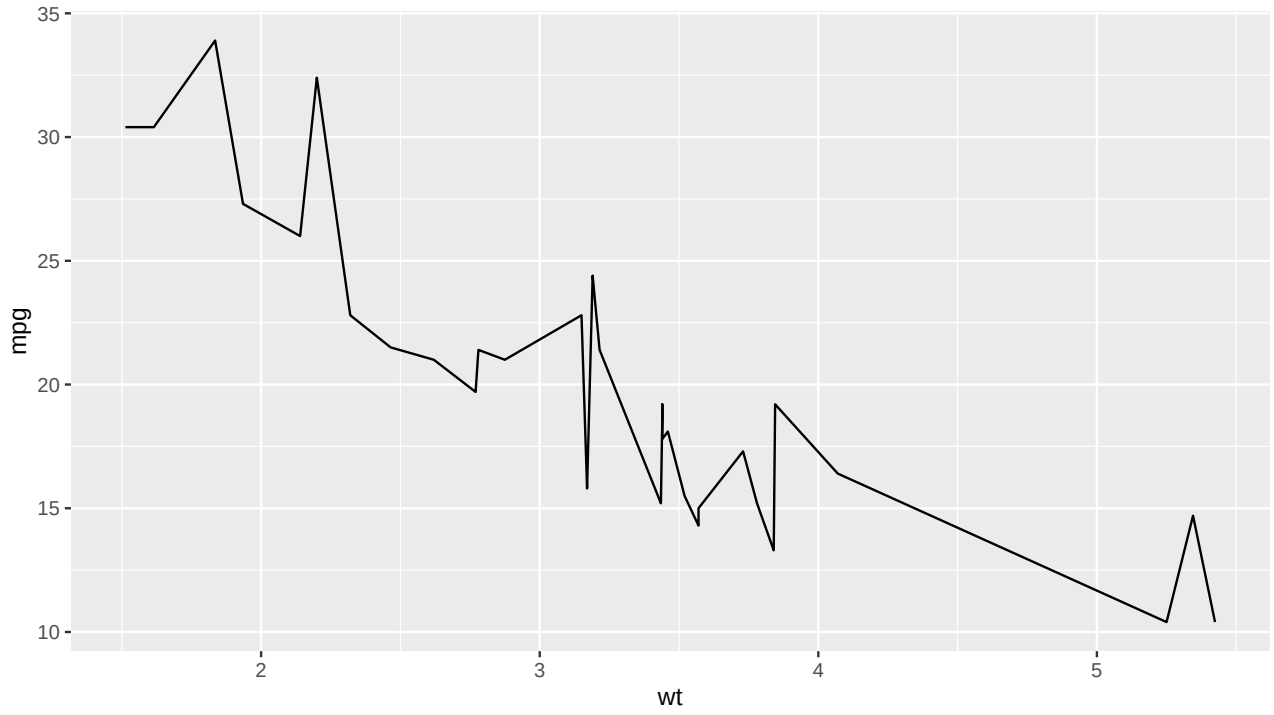
Step 3: Adding Geometric Objects

Geometric objects (geoms) visually represent data. For a scatter plot, we use `geom_point()`, which plots a point for each observation in the dataset.

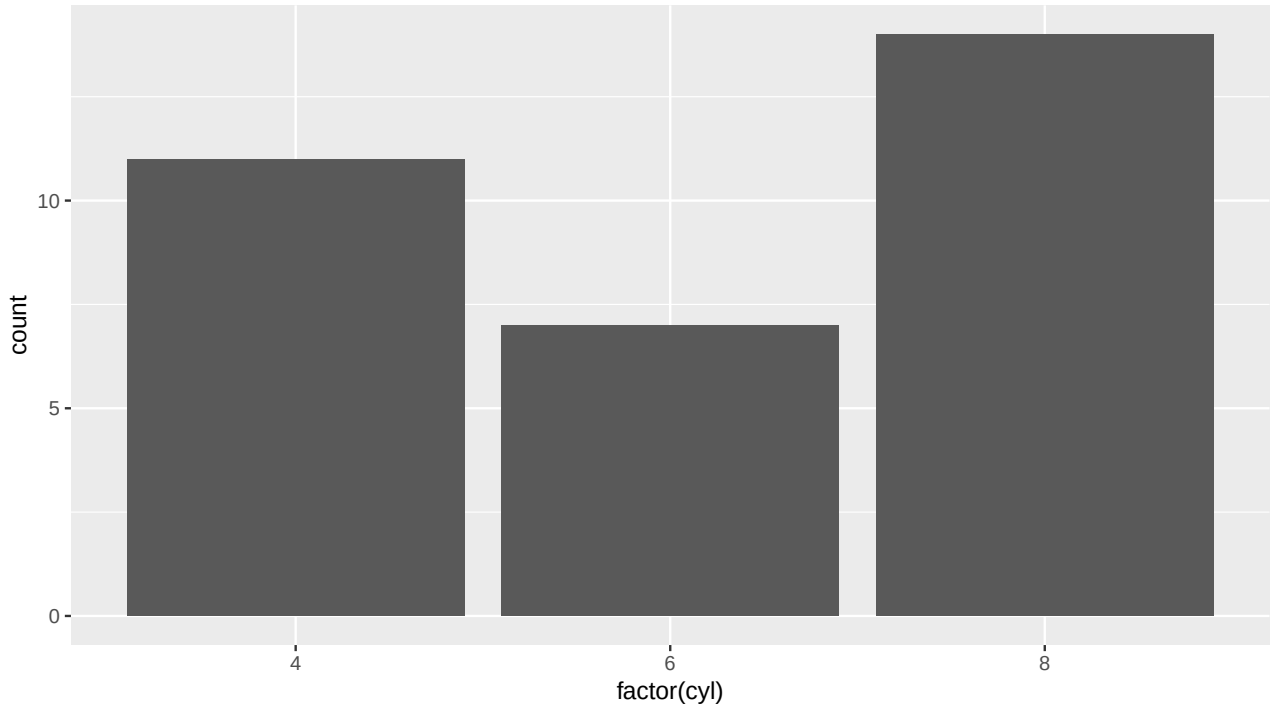
```r
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_point(): draws a point for each row at its mapped x and y values.
# geom_line(): joins observations in x order; group determines which observations form
 ↪  each line.
# factor(): stores categorical values; levels fixes their permitted order and labels can
 ↪  rename their display.
# geom_bar() counts raw observations within each x category by default.
# A fill mapping subdivides the counts; position="dodge" places subgroup bars side by
 ↪  side.
# Scatter plot with points
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point()
```




```r
# Line plot
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_line()
```




```
# Bar plot
ggplot(mtcars, aes(x = factor(cyl))) +
  geom_bar()
```

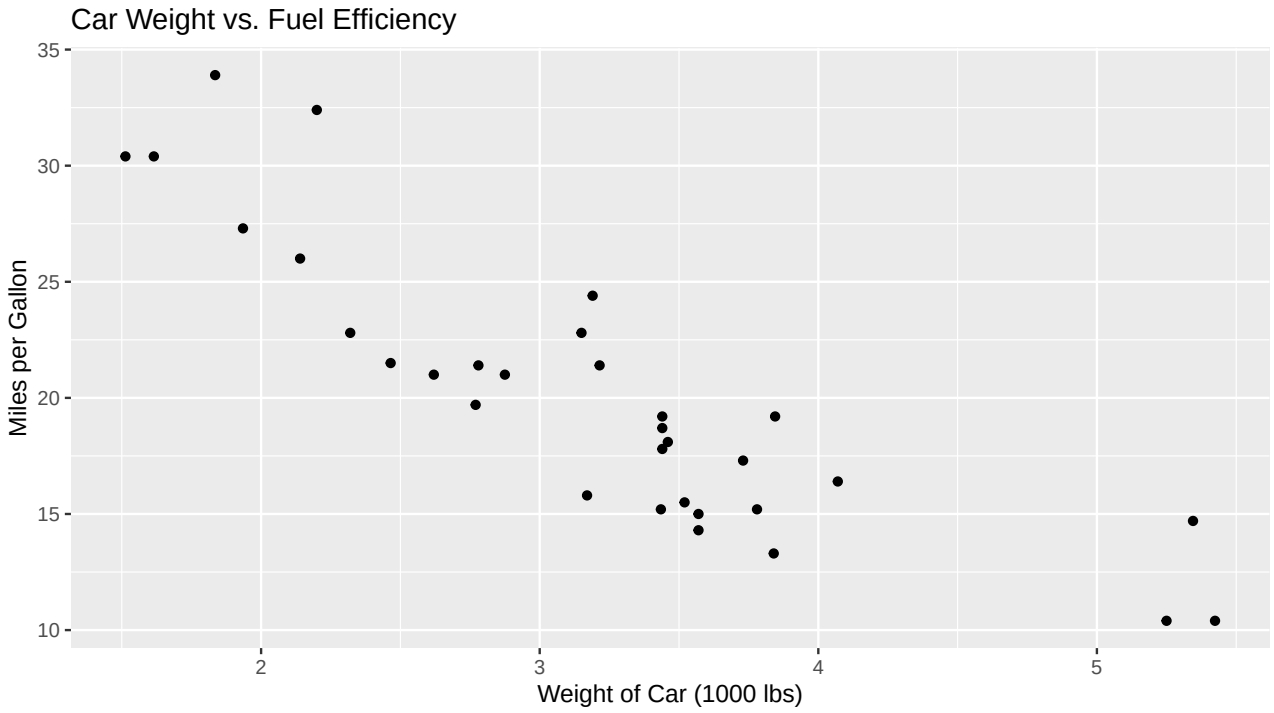


This line of code adds a layer to the ggplot object that plots points based on the aesthetic mappings we defined.

Step 4: Enhancing the Plot with Labels and Titles

Labels and titles help make your plots informative. `labs()` allows you to add a main title, as well as custom labels for the x and y axes.

```
# ggplot() defines the data and aes() maps variables to visual properties such as x, y,
 ↪  and color.
# geom_point() adds paired observations when present; labs() and scale functions control
 ↪  labels and axes.
ggplot(data = mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  labs(title = "Car Weight vs. Fuel Efficiency",
       x = "Weight of Car (1000 lbs)",
       y = "Miles per Gallon")
```

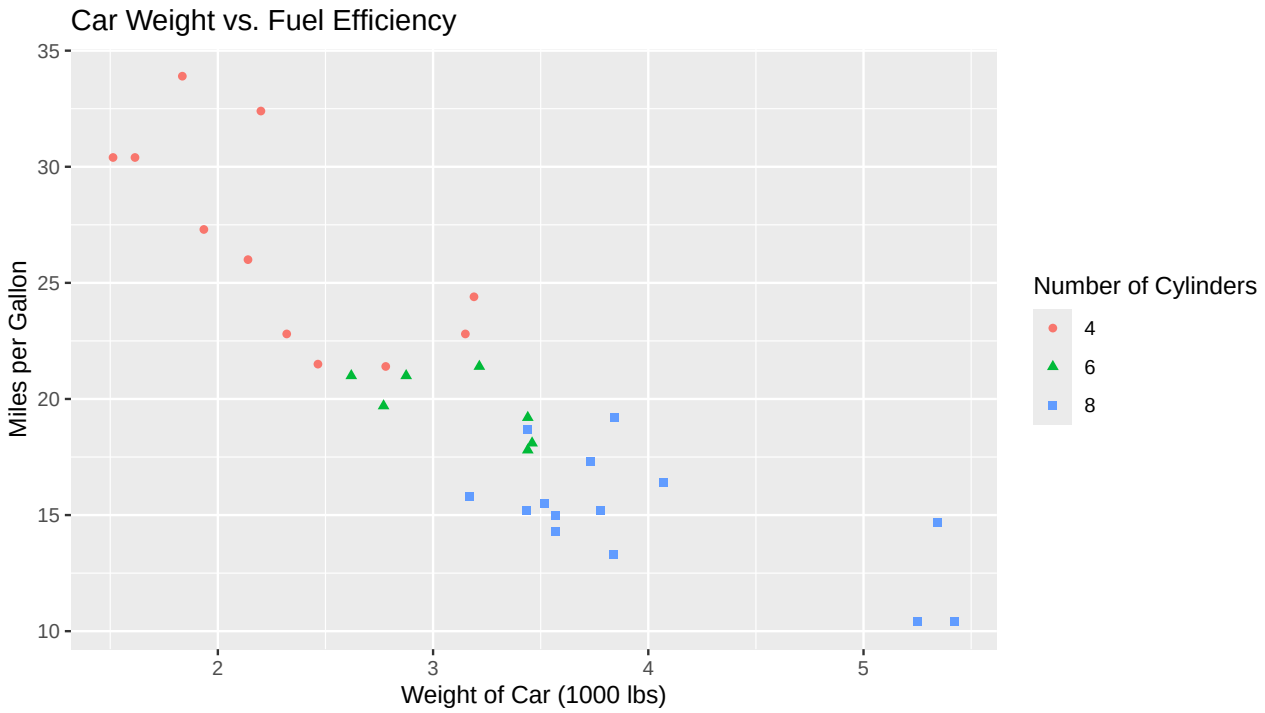


This code sets the title of the plot and labels each axis.

Step 5: Customizing Colors and Shapes

Customizing the appearance of your plot can help highlight specific data or make your plot easier to read. Use color and shape within your aesthetic mappings to differentiate groups or categories.

```
# factor(): stores categorical values; levels fixes their permitted order and labels can
 ↪  rename their display.
# ggplot() defines the data and aes() maps variables to visual properties such as x, y,
 ↪  and color.
# geom_point() adds paired observations when present; labs() and scale functions control
 ↪  labels and axes.
ggplot(data = mtcars, aes(x = wt, y = mpg, color = factor(cyl), shape = factor(cyl))) +
  geom_point() +
  labs(title = "Car Weight vs. Fuel Efficiency",
       x = "Weight of Car (1000 lbs)",
       y = "Miles per Gallon",
       color = "Number of Cylinders",
       shape = "Number of Cylinders")
```

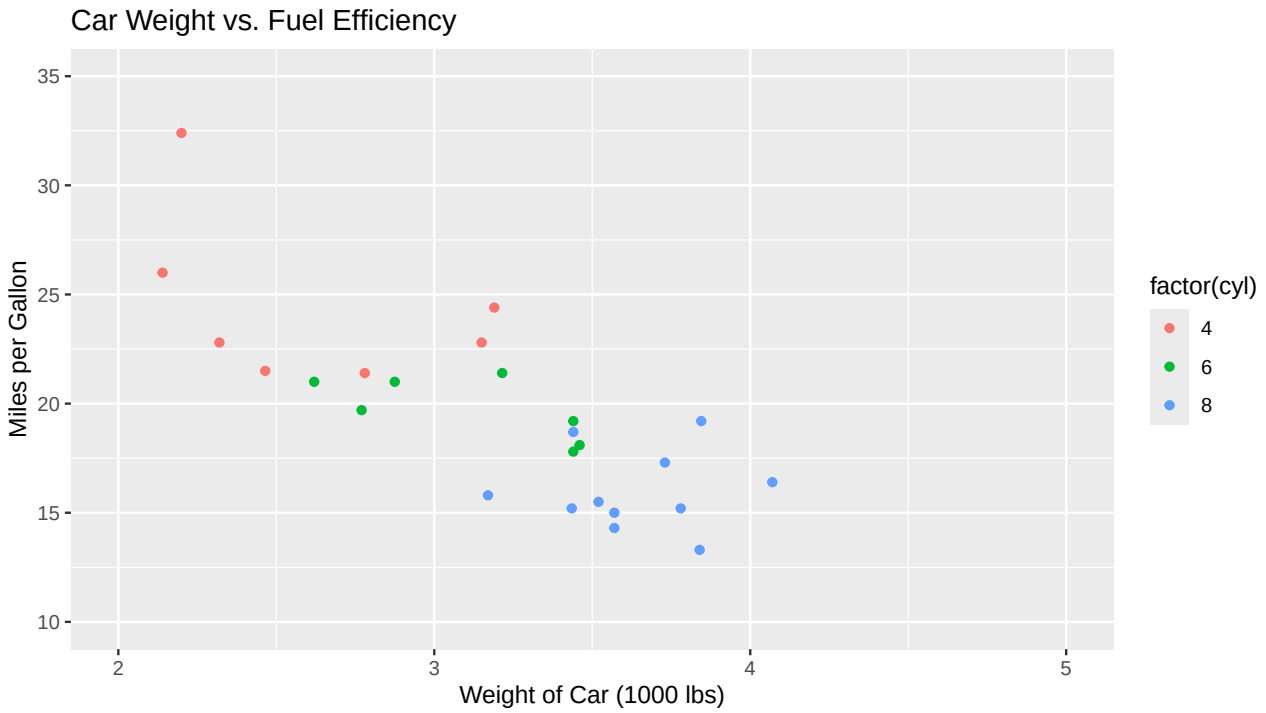


Here, cars are colored and shaped according to the number of cylinders, which helps in distinguishing between different groups visually.

Step 6: Adjusting X and Y Axis Limits

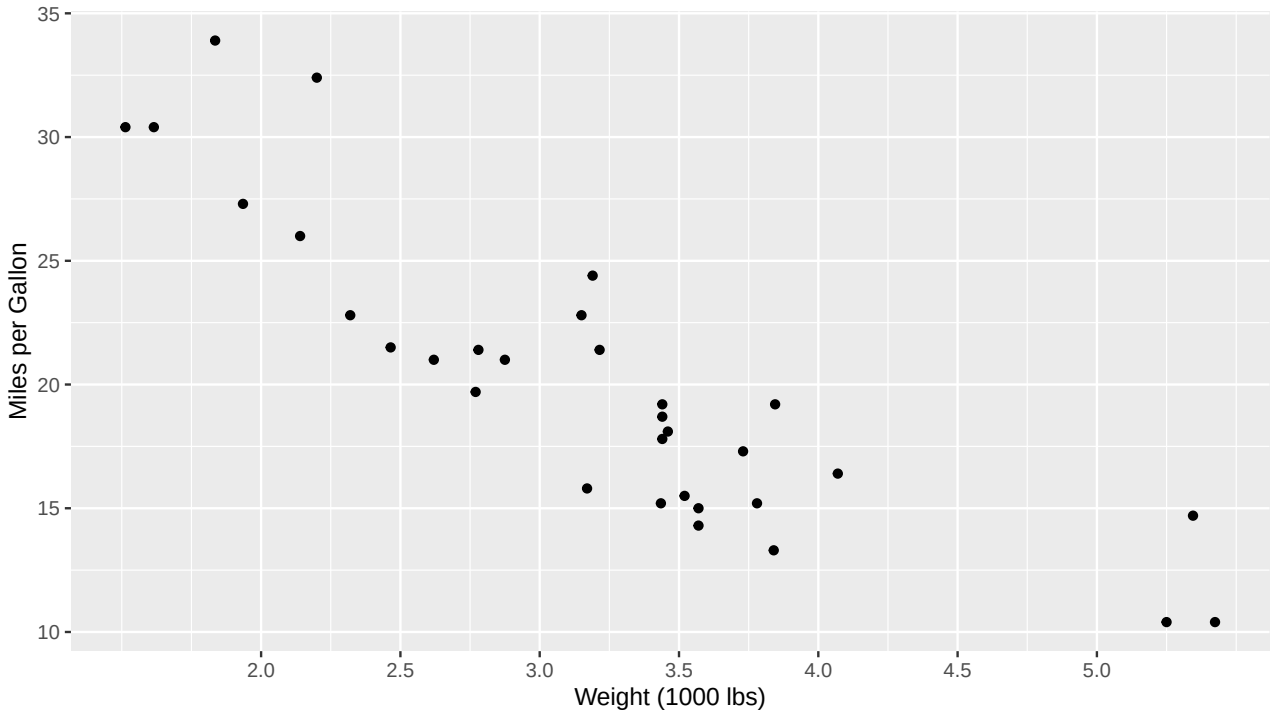
Adjusting the scales and limits of your axes can focus your analysis on specific areas of interest. `xlim()` and `ylim()` functions control the limits of the x-axis and y-axis respectively.

```
# factor(): stores categorical values; levels fixes their permitted order and labels can
 ↪  rename their display.
# xlim(): sets x-axis scale limits; ggplot scale limits can discard observations outside
 ↪  the range.
# ylim(): sets y-axis scale limits; ggplot scale limits can discard observations outside
 ↪  the range.
# ggplot() defines the data and aes() maps variables to visual properties such as x, y,
 ↪  and color.
# geom_point() adds paired observations when present; labs() and scale functions control
 ↪  labels and axes.
ggplot(data = mtcars, aes(x = wt, y = mpg, color = factor(cyl))) +
  geom_point() +
  xlim(2, 5) +
  ylim(10, 35) +
  labs(title = "Car Weight vs. Fuel Efficiency",
       x = "Weight of Car (1000 lbs)",
       y = "Miles per Gallon")
```



This adjustment restricts the display to cars weighing between 2,000 and 5,000 lbs and with MPG between 10 and 35.

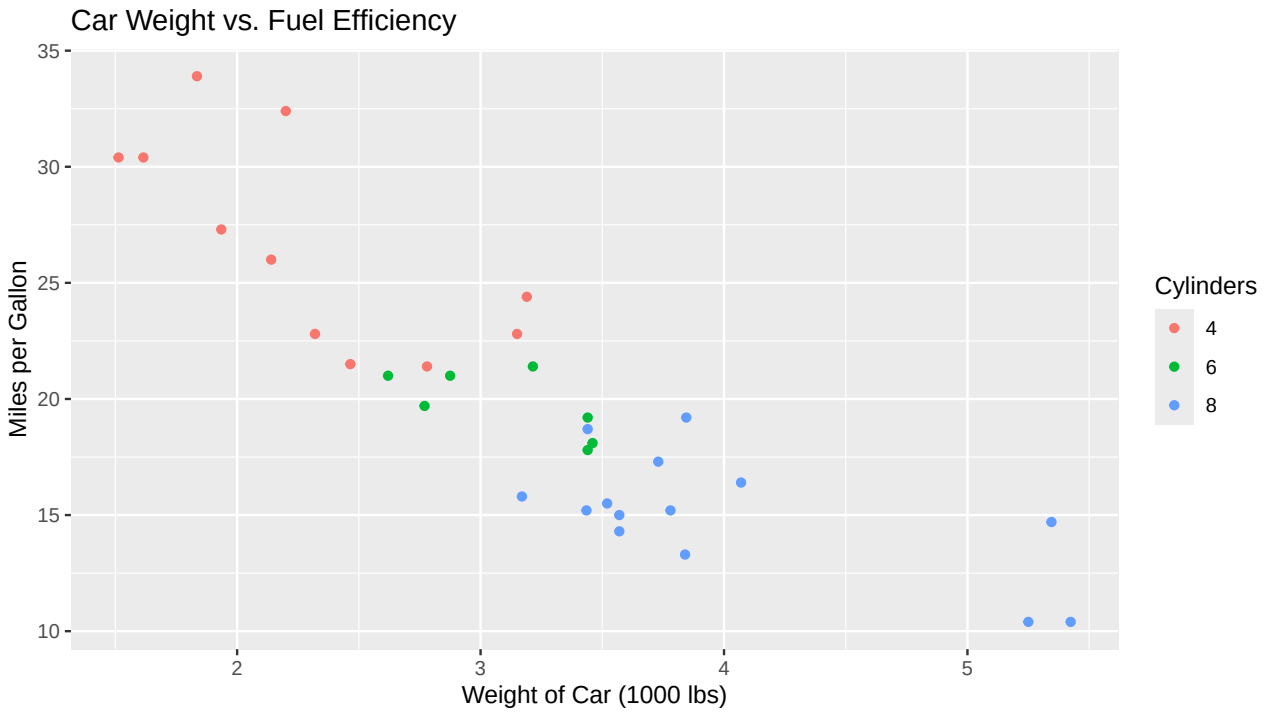
```
# scale_x_continuous(): controls numeric x-axis breaks, labels, limits, and spacing.
# seq(): creates an equally spaced sequence; by sets spacing and length.out sets the
 ↪  number of values.
# scale_y_continuous(): controls numeric y-axis breaks, labels, limits, and spacing.
# ggplot() defines the data and aes() maps variables to visual properties such as x, y,
 ↪  and color.
# geom_point() adds paired observations when present; labs() and scale functions control
 ↪  labels and axes.
# Customizing scales
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  scale_x_continuous(name = "Weight (1000 lbs)", breaks = seq(2, 5, by = 0.5)) +
  scale_y_continuous(name = "Miles per Gallon")
```



Step 7: Modifying the Legend

Legends are key to understanding the plotted data. They can be adjusted with the `guides()` or through direct specifications in the `scale_color_*` or `scale_shape_*` functions.

```
# factor(): stores categorical values; levels fixes their permitted order and labels can
 ↪  rename their display.
# guides(): controls legend display for mapped visual properties.
# guide_legend(): sets legend layout and appearance.
# ggplot() defines the data and aes() maps variables to visual properties such as x, y,
 ↪  and color.
# geom_point() adds paired observations when present; labs() and scale functions control
 ↪  labels and axes.
ggplot(data = mtcars, aes(x = wt, y = mpg, color = factor(cyl))) +
  geom_point() +
  labs(title = "Car Weight vs. Fuel Efficiency",
       x = "Weight of Car (1000 lbs)",
       y = "Miles per Gallon",
       color = "Number of Cylinders") +
  guides(color = guide_legend(title = "Cylinders"))
```

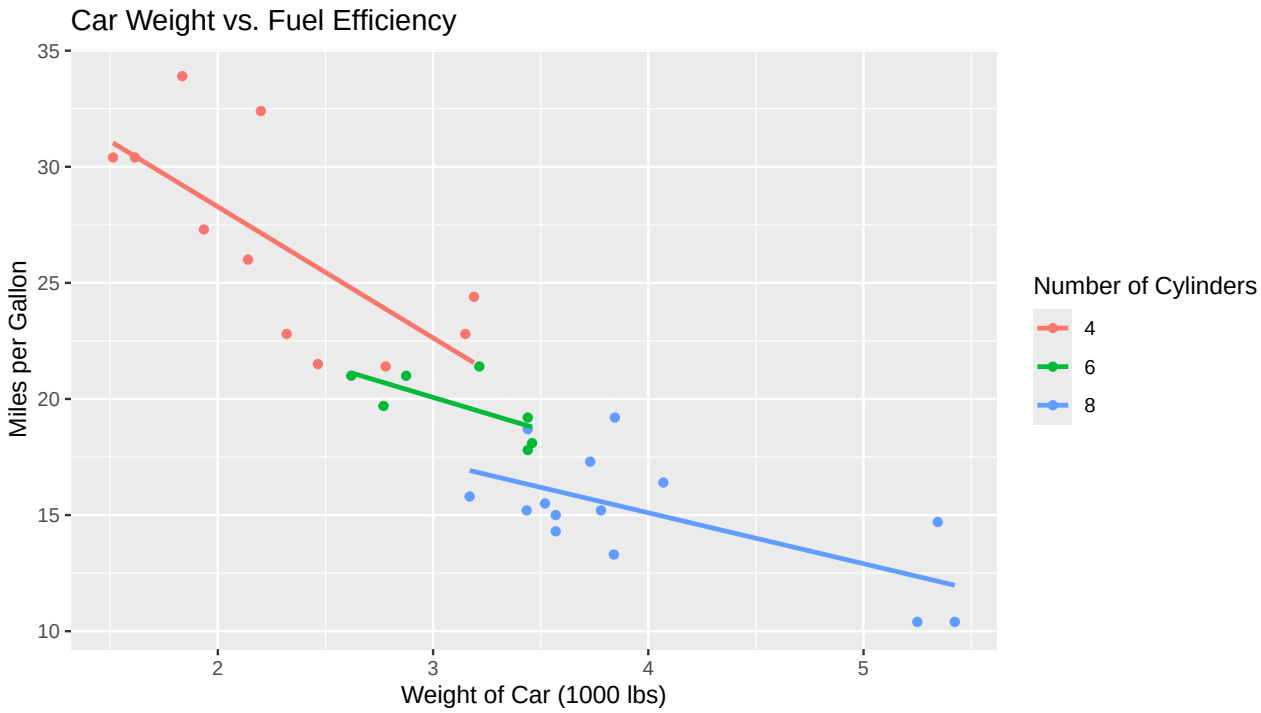


This code ensures that the legend is clearly labeled as “Cylinders”.

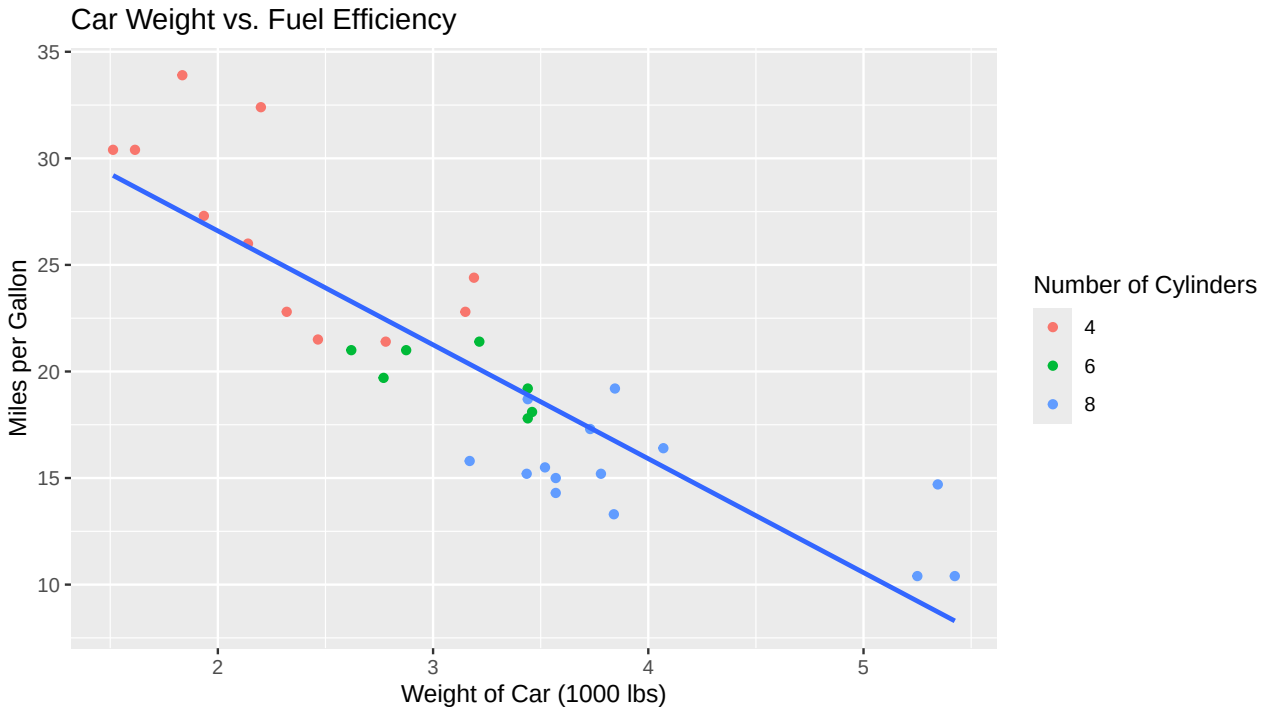
Step 8: Adding More Geoms

Sometimes, additional layers can help further analyze the data. Adding a smooth line can help visualize trends.

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# factor(): stores categorical values; levels fixes their permitted order and labels can
 ↪  rename their display.
# geom_point(): draws a point for each row at its mapped x and y values.
# labs(): sets axis titles, legend titles, plot titles, and captions.
# geom_smooth(method = "lm") adds a fitted linear trend to the plotted pairs.
# Its default shaded band is uncertainty about the mean trend; se=FALSE suppresses that
 ↪  band.
ggplot(data = mtcars, aes(x = wt, y = mpg, color = factor(cyl))) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "Car Weight vs. Fuel Efficiency",
       x = "Weight of Car (1000 lbs)",
       y = "Miles per Gallon",
       color = "Number of Cylinders")
```




```
# Create one line
ggplot(data = mtcars, aes(x = wt, y = mpg)) +
  geom_point(aes(color = factor(cyl))) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(title = "Car Weight vs. Fuel Efficiency",
       x = "Weight of Car (1000 lbs)",
       y = "Miles per Gallon",
       color = "Number of Cylinders")
```



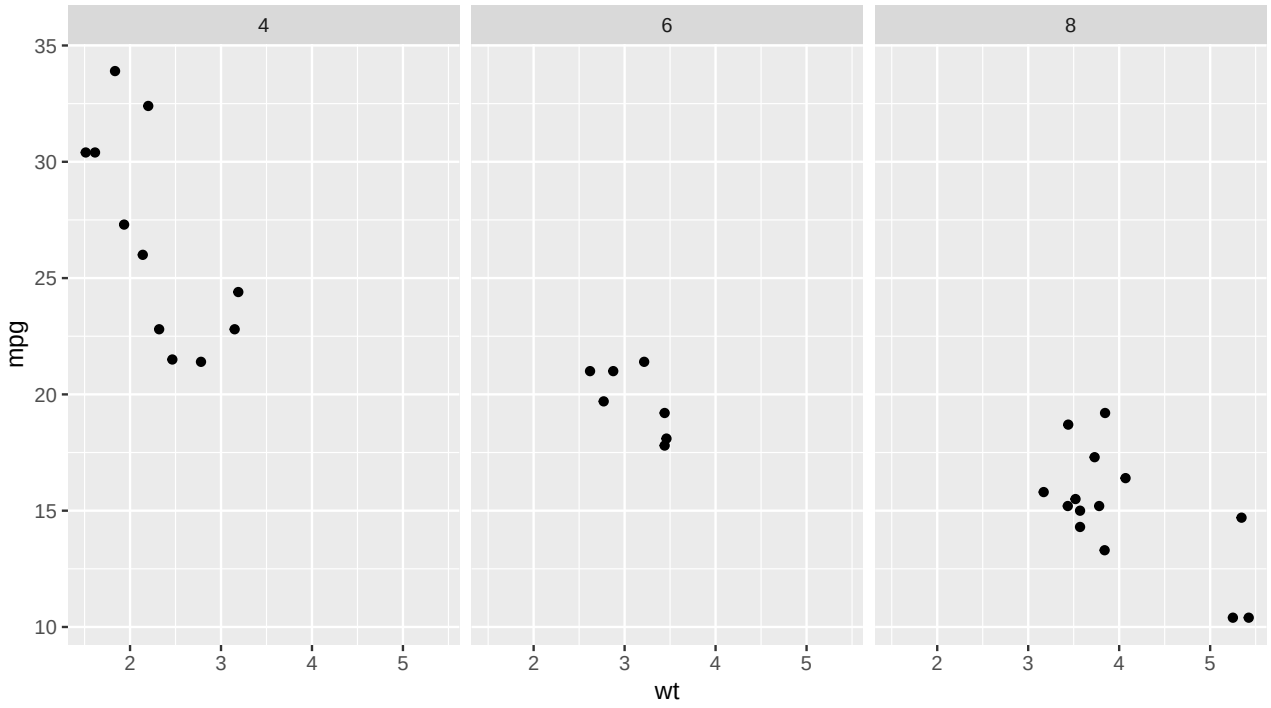
In the first code, multiple regression lines are plotted — one for each cylinder category because **`color = factor(cyl)`** is part of the global aesthetics and affects all layers, including **`geom_smooth()`**.

- In the second code, only one regression line is plotted for all the data combined, with **`color = factor(cyl)`** only applied to **`geom_point()`** and not affecting **`geom_smooth()`**.

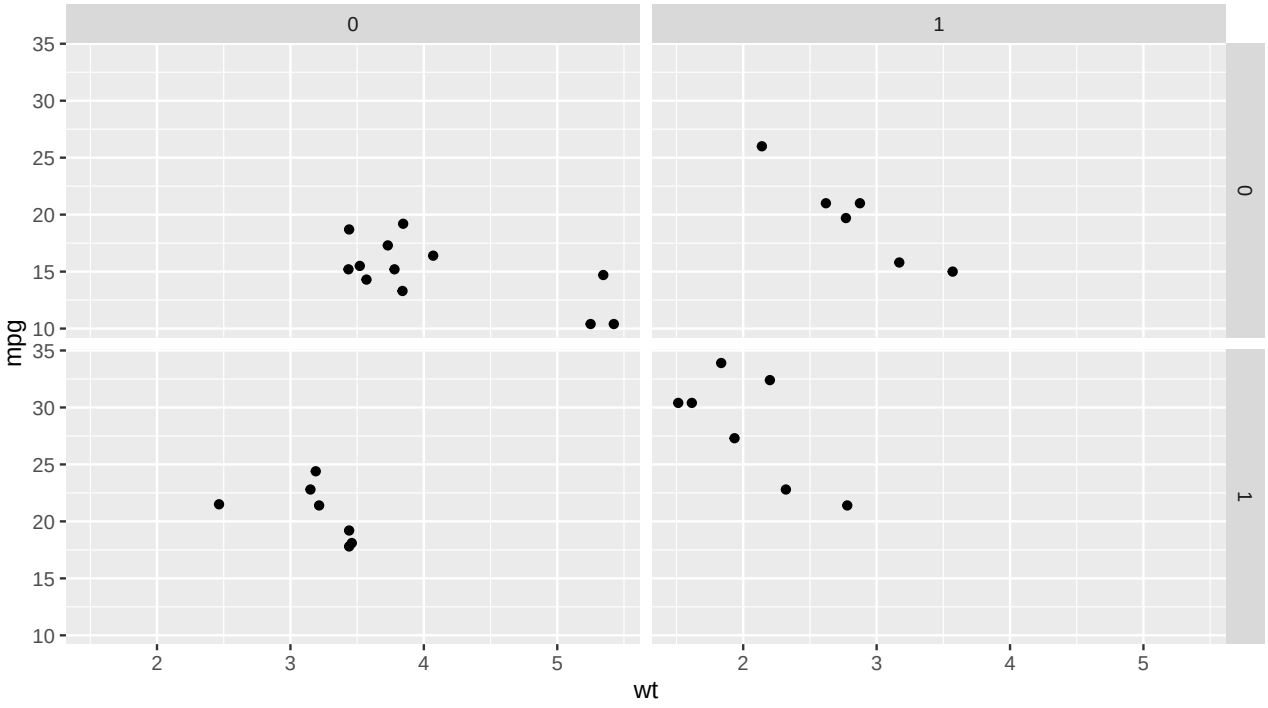
Faceting

Faceting in **ggplot2** is a feature that allows you to split one plot into multiple plots based on a factor or factors contained in the dataset. Each resulting panel (or facet) displays a subset of the data and is a standalone plot with the same axes and scales. This makes it easy to compare different subsets of the data visually.

```
# library(): loads a package so its functions can be used in this session.
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_point(): draws a point for each row at its mapped x and y values.
# facet_wrap() or facet_grid() separates observations into panels using grouping
 ↪  variables.
# The ~ formula defines the grouping layout; compare panels with attention to their axis
 ↪  scales.
library(ggplot2)
# Faceting by number of cylinders
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  facet_wrap(~ cyl)
```

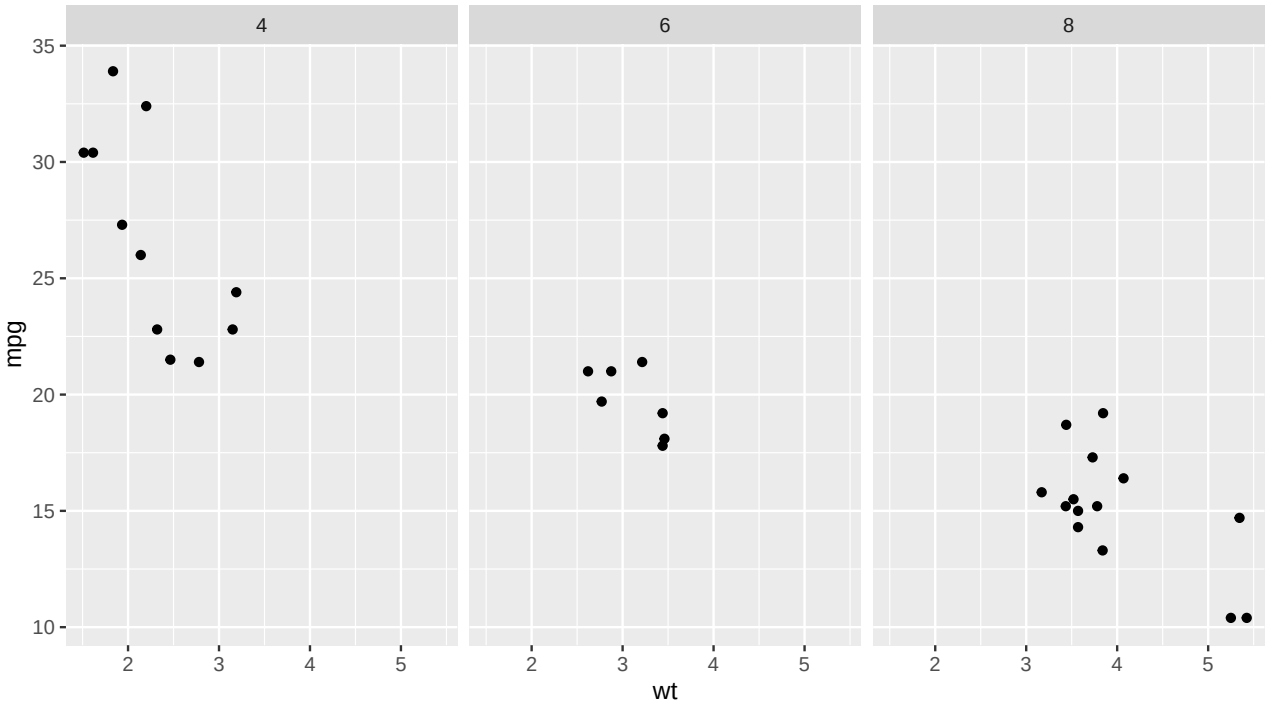



```
# Faceting with grid
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  facet_grid(vs ~ am)
```



Faceting by Number of Cylinders

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_point(): draws a point for each row at its mapped x and y values.
# facet_wrap() or facet_grid() separates observations into panels using grouping
 ↪  variables.
# The ~ formula defines the grouping layout; compare panels with attention to their axis
 ↪  scales.
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  facet_wrap(~ cyl)
```

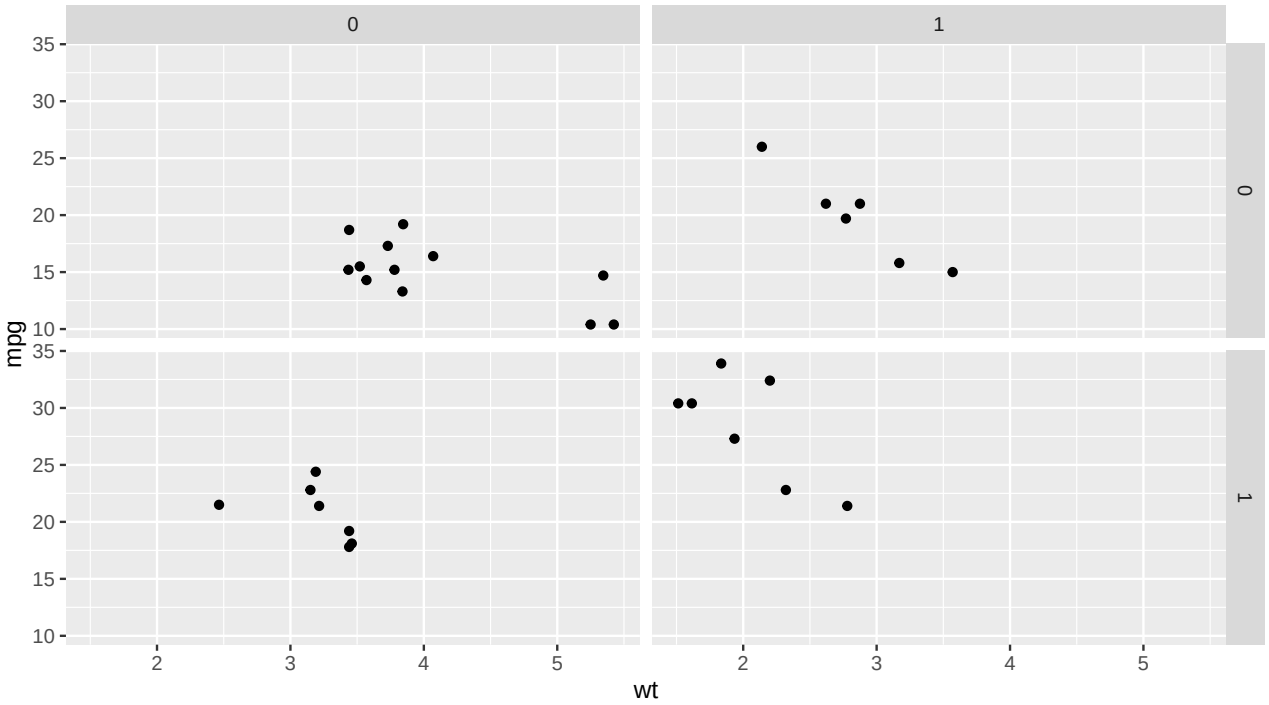


Explanation:

- **`facet_wrap(~ cyl)`**: This function creates a separate plot (facet) for each unique value of the `cyl` (cylinders) variable. The `~ cyl` notation indicates that faceting should be done based on the `cyl` variable. The resulting plot will have multiple panels, each showing the relationship between `wt` and `mpg` for cars with a specific number of cylinders.

Faceting with Grid

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_point(): draws a point for each row at its mapped x and y values.
# facet_wrap() or facet_grid() separates observations into panels using grouping
 ↪  variables.
# The ~ formula defines the grouping layout; compare panels with attention to their axis
 ↪  scales.
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point() +
  facet_grid(vs ~ am)
```



Explanation:

- **`facet_grid(vs ~ am)`**: This function creates a grid of plots based on the combinations of the `vs` and `am` variables. The `vs` variable is used to create rows, and the `am` variable is used to create columns.
- The resulting plot will have multiple panels arranged in a grid, with each panel showing the relationship between `wt` and `mpg` for a specific combination of `vs` (engine type) and `am` (transmission type).

Visual Representation:

- **`facet_wrap(~ cyl)`**: If `cyl` has three unique values (e.g., 4, 6, 8), this will produce three panels, each showing the scatter plot of `wt` vs. `mpg` for cars with 4, 6, and 8 cylinders, respectively.

```
|----------------------|
|      cyl = 4         |
|  (scatter plot)      |
|----------------------|
|      cyl = 6         |
|  (scatter plot)      |
|----------------------|
```

```
|      cyl = 8        |
|  (scatter plot)     |
|---------------------|
```

- **`facet_grid(vs ~ am)`**: If `vs` has two unique values (e.g., 0, 1) and `am` has two unique values (e.g., 0, 1), this will produce a 2x2 grid of panels, each showing the scatter plot of `wt` vs. `mpg` for the combinations of `vs` and `am`.

```
|---------------------|---------------------|
|      vs = 0, am = 0 |      vs = 0, am = 1 |
|  (scatter plot)     |  (scatter plot)     |
|---------------------|---------------------|
|      vs = 1, am = 0 |      vs = 1, am = 1 |
|  (scatter plot)     |  (scatter plot)     |
|---------------------|---------------------|
```

In summary, faceting allows you to split your data into multiple sub-plots based on the values of one or more variables, making it easier to compare subsets of your data visually.

Demonstrating Different GEOM Types

Below are code examples using different geometric objects (`geoms`) in `ggplot2`, utilizing commonly available datasets in R like `mtcars`, `mpg`, and `iris`. Each example demonstrates how to use these geoms effectively:

1. `geom_point()` - Scatter plot

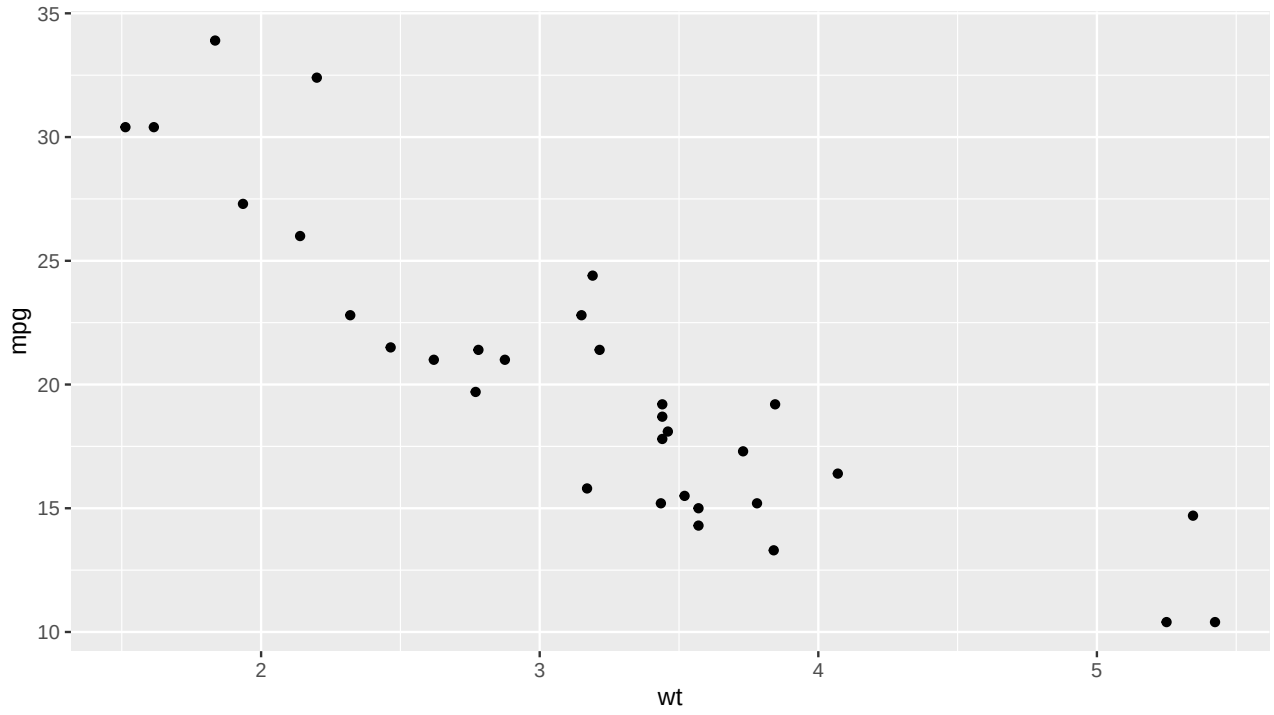
Data Types: Numerical on both axes.

Uses: Primarily used for showing relations between data items. Scatter plots are excellent for visualizing how two continuous variables relate to each other, identifying correlations, trends, clusters, or potential outliers.

Business Examples: Scatter plots are fundamental for examining the relationship or correlation between two continuous variables. They are frequently used in business to analyze trends, such as sales performance against advertising spend, or customer age versus product preference.

Code Example: The point plot shown below uses the **`mtcars`** dataset to create a scatter plot of car weight (**`wt`**) against miles per gallon (**`mpg`**). Each point represents a car, with its position determined by these two variables. The plot can be used to analyze the relationship between a car's weight and its fuel efficiency. Generally, one might expect to see a trend where heavier cars achieve lower fuel efficiency.

```r
# library(): loads a package so its functions can be used in this session.
# ggplot() defines the data and aes() maps variables to visual properties such as x, y,
 ↪  and color.
# geom_point() adds paired observations when present; labs() and scale functions control
 ↪  labels and axes.
library(ggplot2)
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_point()
```



2. `geom_line()` - Line plot

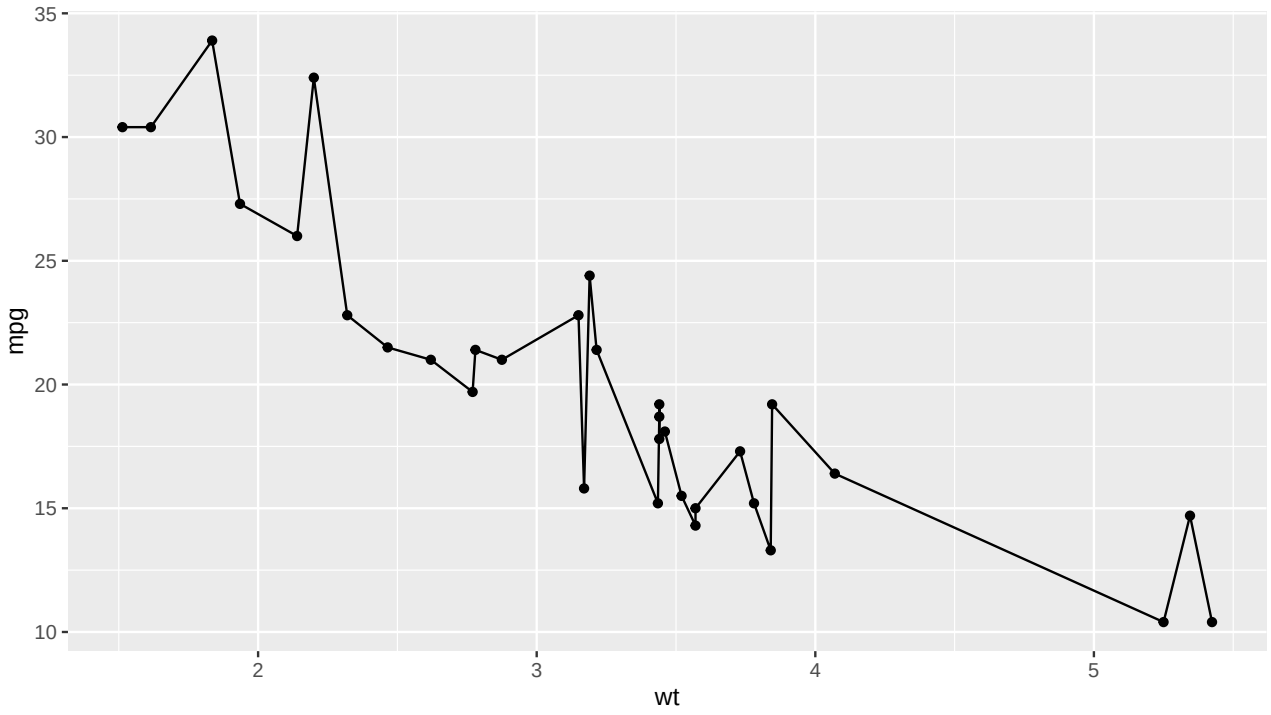
Data Types: Typically, a continuous or ordered categorical variable on the x-axis and a numerical variable on the y-axis.

Uses: Best used for showing trends and relationships over time. Line plots are ideal for tracking changes, making them suitable for time-series data analysis but less so for direct comparisons or distributions

Business Examples: Line plots are ideal for tracking changes over time, such as revenue growth, stock prices, or website traffic trends. This makes them invaluable for time series analysis in financial reporting and market analysis.

Code Example: The line plot presented below is also based on the **`mtcars`** dataset, displaying a continuous line connecting points plotted by car weight (**`wt`**) against miles per gallon (**`mpg`**). The addition of points helps to highlight the actual data positions. This plot is useful for observing the trend of how fuel efficiency changes as car weight increases, suggesting a potential negative correlation.

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_point(): draws a point for each row at its mapped x and y values.
# geom_line() joins observations in x order within each plot group.
# Connecting values implies an ordered progression, so check whether that meaning fits the
 ↪  question.
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_line() + geom_point()  # Adding points for clarity
```



3. `geom_smooth()` - Smoothed line plot

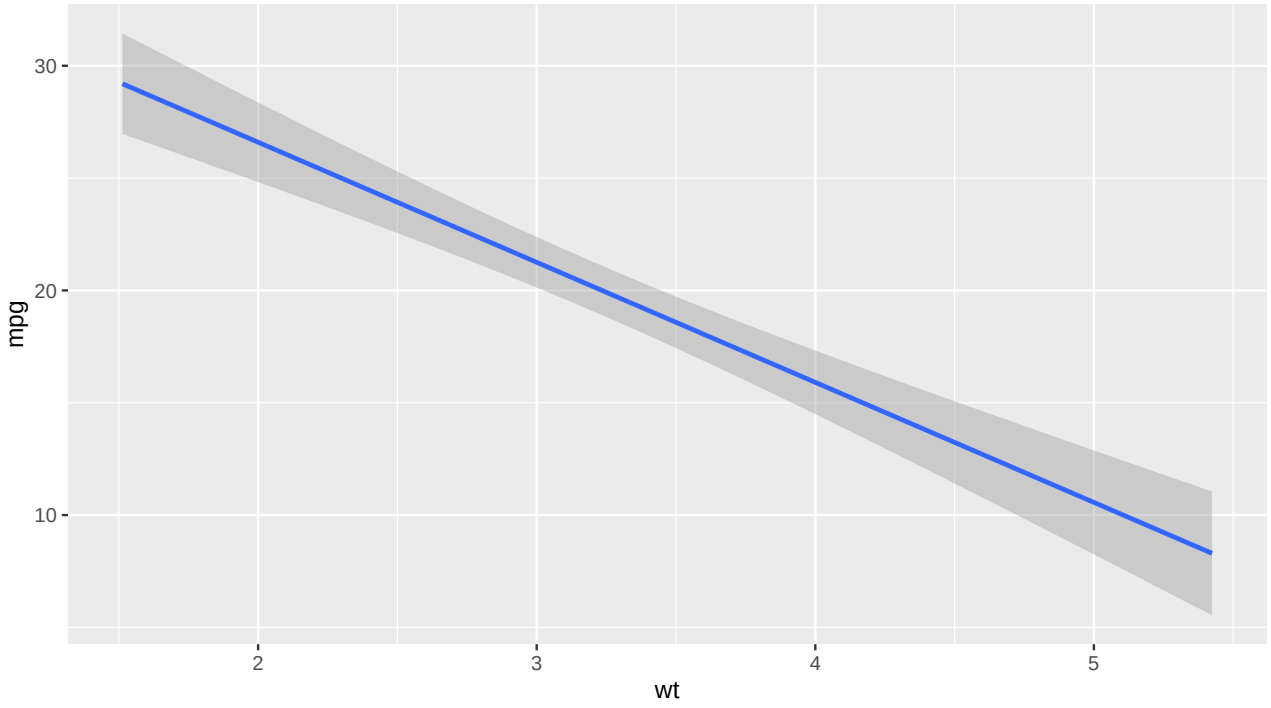
Data Types: Numerical on both axes.

Uses: Focused on showing relationships with a smoothed perspective, often used to highlight underlying trends in data. While it helps in visualizing the general direction and strength of relationships, it's not typically used for showing distributions or detailed comparisons.

Business Examples: Used to visualize trends in data by adding a smoothed line, which can help identify underlying patterns in volatile datasets. Common uses include smoothing out random fluctuations in financial data or sales data to better understand long-term trends.

Code Example: Here, a smoothed line plot is created using a linear model (**`lm`**) to approximate the relationship between **`wt`** (weight of the cars) and **`mpg`** (miles per gallon). This plot helps in visualizing a clear trend or pattern, smoothing out any outliers or anomalies, which can be particularly useful for presentations or reports where a simplified visual explanation is required.

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_smooth(method = "lm") adds a fitted linear trend to the plotted pairs.
# Its default shaded band is uncertainty about the mean trend; se=FALSE suppresses that
 ↪  band.
ggplot(mtcars, aes(x = wt, y = mpg)) +
  geom_smooth(method = "lm")  # Linear model
```



4. `geom_bar()` - Bar chart

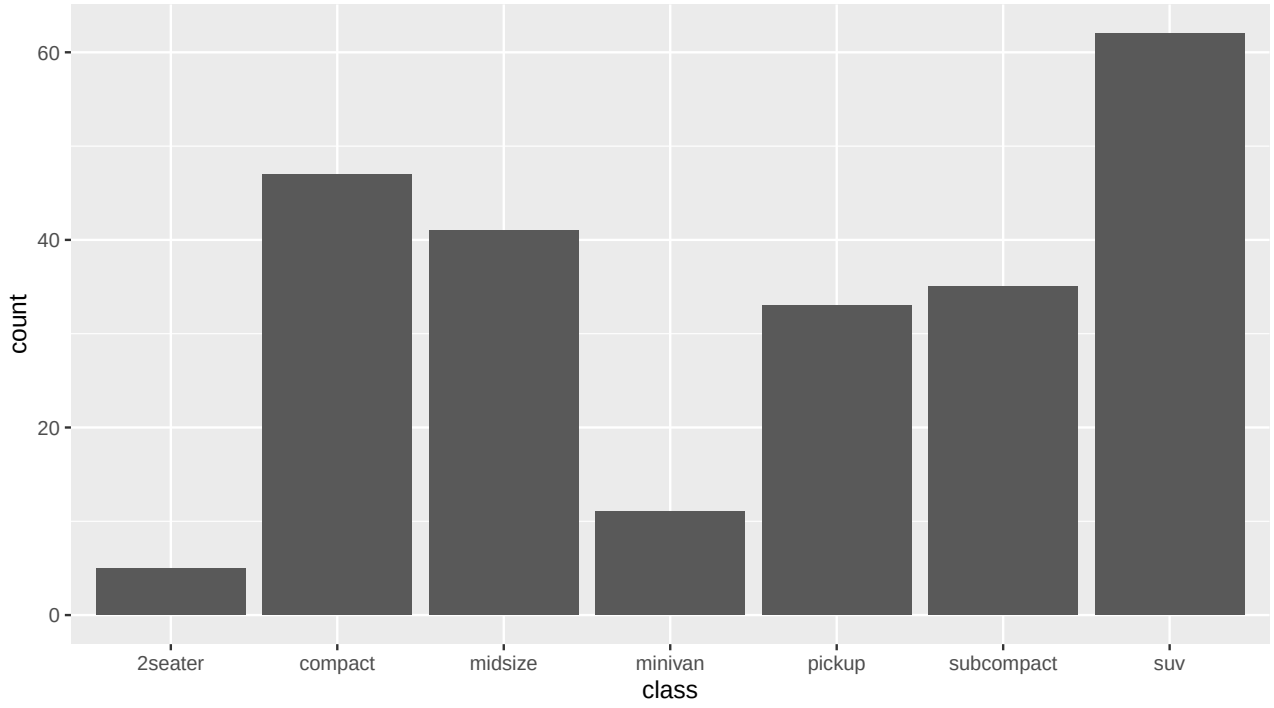
Data Types: Categorical on the x-axis and numerical on the y-axis (counts or computed summaries).

Uses: Excellent for making comparisons and showing components. Bar charts are used to compare the size of different categories using the height of bars, making it clear how individual categories stack up against each other.

Business Examples: Bar charts are one of the most common tools for business reporting, useful for comparing quantities across different categories, such as sales by product category, customer counts by region, or performance metrics across departments.

Code Example: This bar chart presented below uses the **`mpg`** dataset to display the count of cars in each vehicle class. Each bar represents a different class, and its height indicates the number of cars in that class. This type of visualization is excellent for comparing categorical data, showing which vehicle classes are more or less common.

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_bar() counts raw observations within each x category by default.
# A fill mapping subdivides the counts; position="dodge" places subgroup bars side by
 ↪  side.
ggplot(mpg, aes(x = class)) +
  geom_bar()
```



5. `geom_histogram()` - Histogram

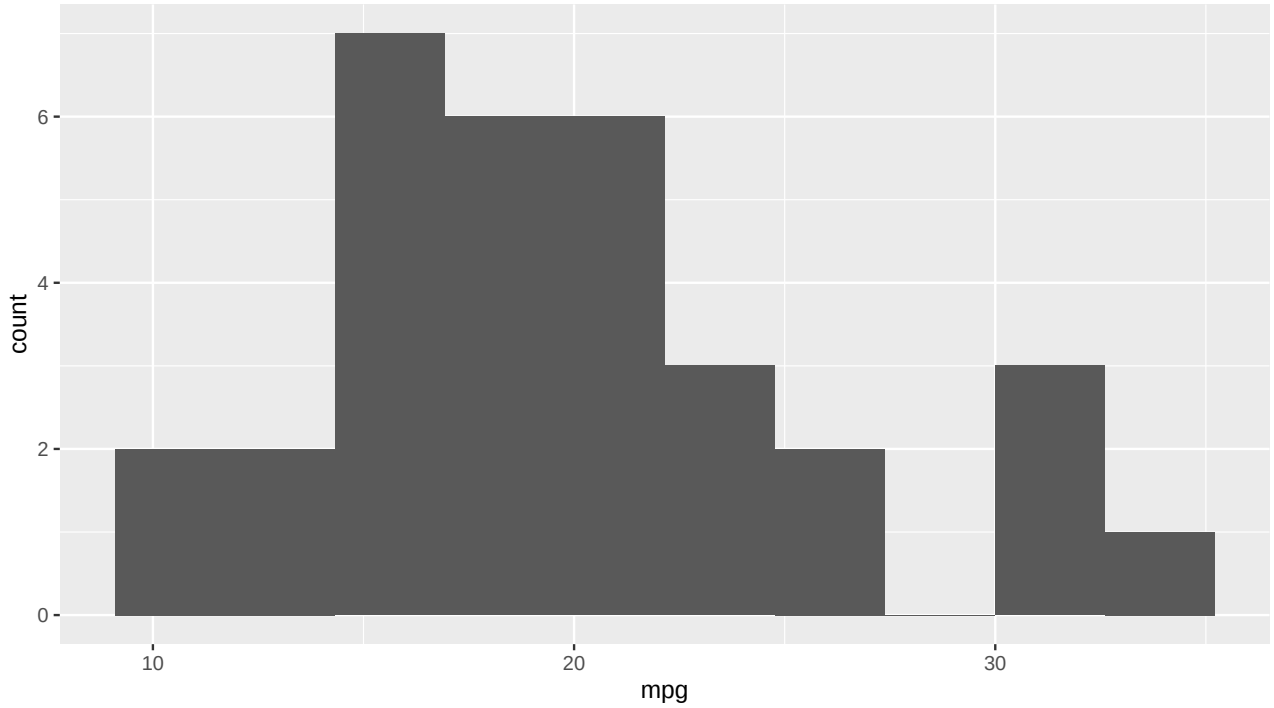
Data Types: Numerical data divided into bins on the x-axis.

Uses: Specifically designed for showing distributions of data. Histograms provide a visual interpretation of numerical data by indicating the number of data points that fall within a range of values, known as bins.

Business Examples: Histograms are used to examine the distribution of a dataset, helping to understand the central tendency, variability, and shape of data distribution. In business, this could be used for analyzing the distribution of transaction sizes, customer age groups, or service response times.

Code Example: The histogram example below plots the distribution of miles per gallon (**`mpg`**) across the cars in the **`mtcars`** dataset. By dividing the **`mpg`** values into bins (in this case, 10 bins), it shows how many cars fall into each range of fuel efficiency. This is useful for understanding the overall distribution of fuel efficiency across the dataset.

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_histogram() groups raw numerical observations into intervals and counts each
 ↪  interval.
# bins requests an approximate number of intervals; binwidth or explicit breaks gives
 ↪  direct width control.
ggplot(mtcars, aes(x = mpg)) +
  geom_histogram(bins = 10)  # Specify number of bins
```



6. `geom_boxplot()` - Boxplot

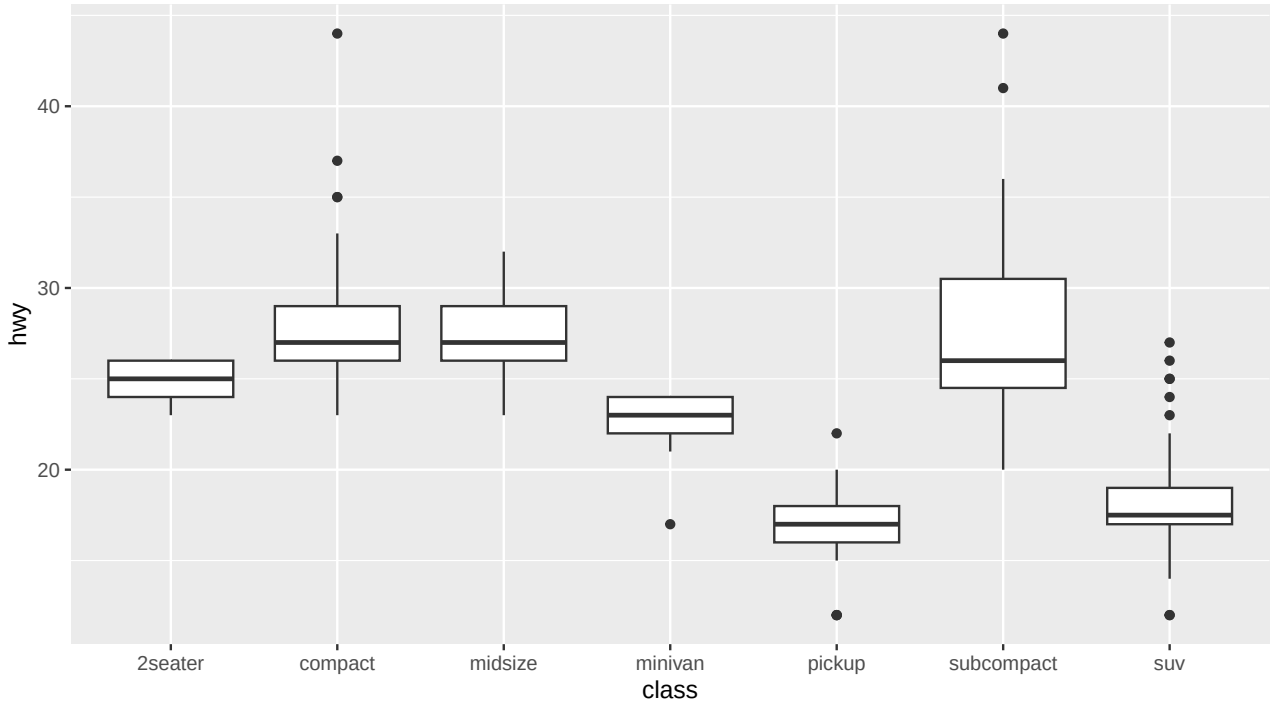
Data Types: Categorical on the x-axis and numerical on the y-axis.

Uses: Highly effective for both showing distributions and making comparisons. Boxplots summarize the distribution of data through quartiles, while also highlighting outliers. They allow comparisons across different categories or groups.

Business Examples: Boxplots provide a concise summary of the distribution of numerical data and are particularly good at highlighting outliers. Businesses use them for comparing distributions across groups, such as comparing employee satisfaction scores across branches or sales distributions across regions.

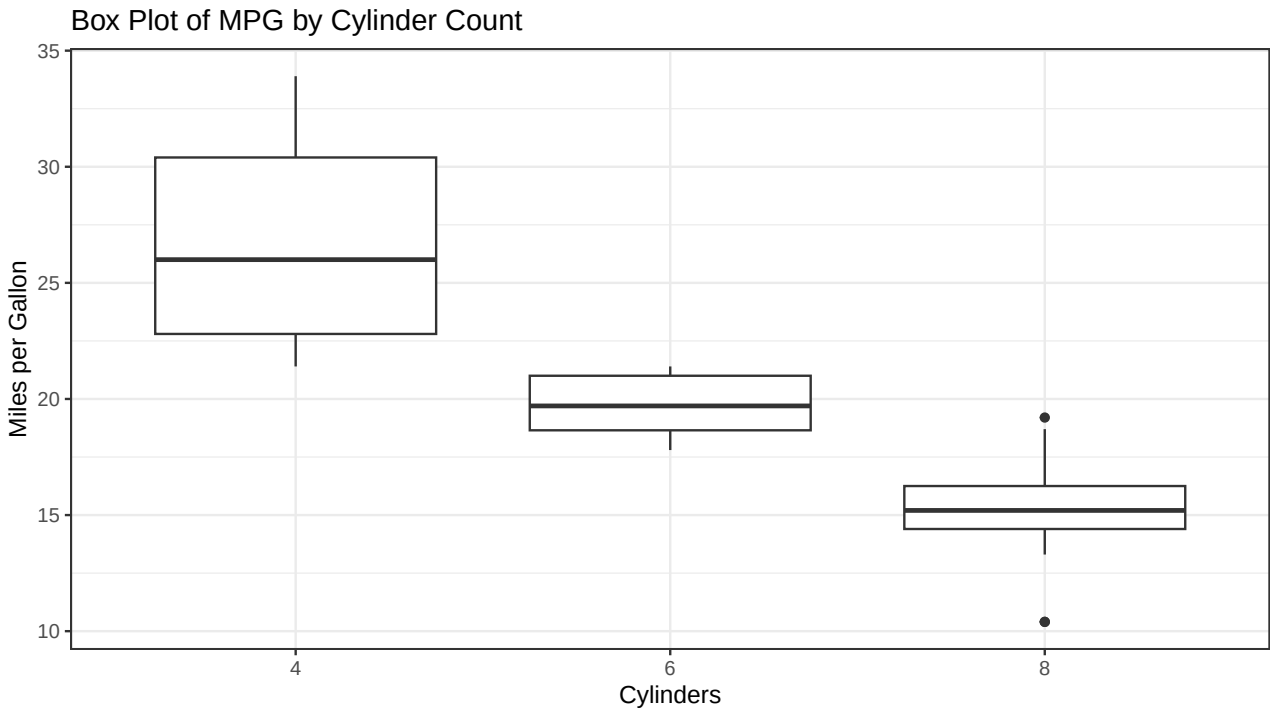
Code Example: This boxplot example provides a visual summary of highway miles per gallon (**`hwy`**) across different vehicle classes in the **`mpg`** dataset. Boxplots show the median, quartiles, and outliers, giving a clear statistical summary of the distribution within each category. This plot is particularly valuable for identifying differences and spread in highway efficiency across vehicle types.

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_boxplot() summarizes each numerical distribution with a box, median, whiskers, and
 ↪  flagged points.
# This plot uses ggplot2 quartile defaults unless explicit statistics are supplied.
ggplot(mpg, aes(x = class, y = hwy)) +
  geom_boxplot()
```

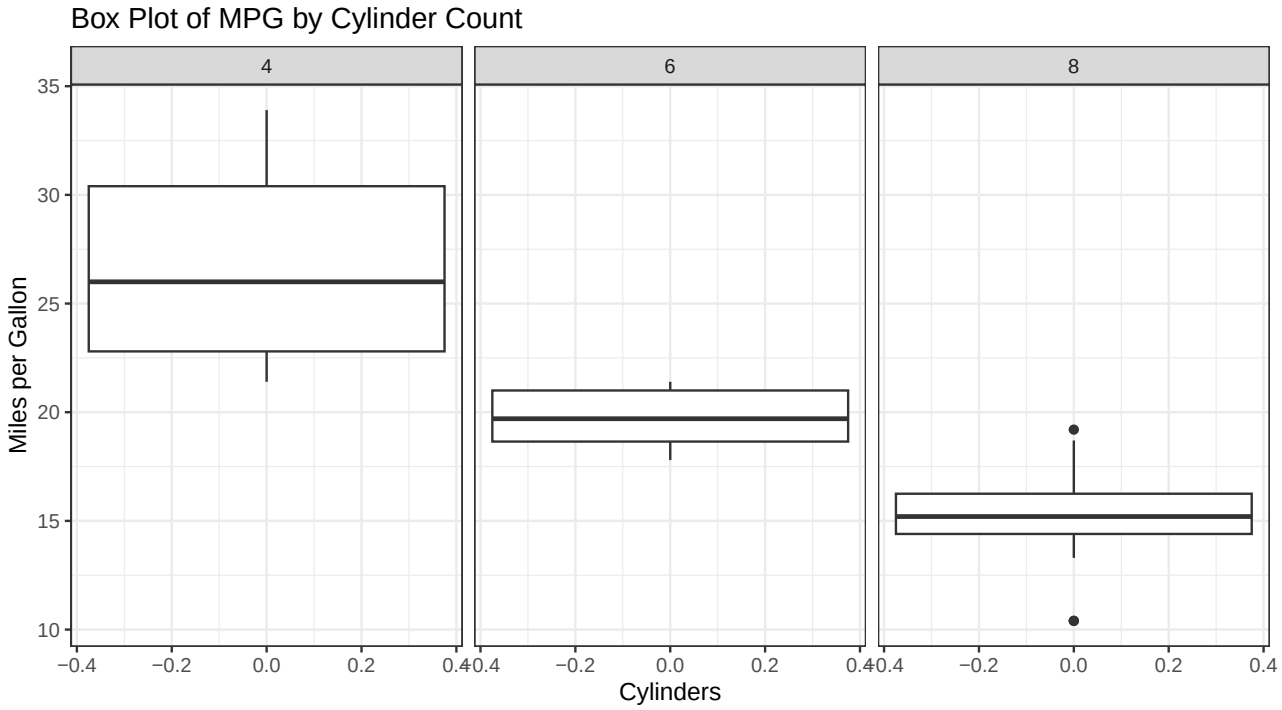


Boxplots are usually used for making comparisons between different categories.

```
# library(): loads a package so its functions can be used in this session.
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# factor(): stores categorical values; levels fixes their permitted order and labels can
 ↪  rename their display.
# geom_boxplot(): draws quartiles, whiskers, and flagged points; a flagged value is not
 ↪  automatically an error.
# labs(): sets axis titles, legend titles, plot titles, and captions.
# theme_bw(): applies a white-background ggplot theme.
# facet_wrap() or facet_grid() separates observations into panels using grouping
 ↪  variables.
# The ~ formula defines the grouping layout; compare panels with attention to their axis
 ↪  scales.
library(ggplot2)
ggplot(data = mtcars, aes(x = factor(cyl), y = mpg)) +
  geom_boxplot() +
  labs(title = "Box Plot of MPG by Cylinder Count", x = "Cylinders", y = "Miles per
   ↪  Gallon") +
  theme_bw()
```




```
ggplot(data = mtcars, aes( y = mpg)) +
  geom_boxplot() +
  facet_wrap(~cyl) +
  labs(title = "Box Plot of MPG by Cylinder Count", x = "Cylinders", y = "Miles per
   ↪  Gallon") +
  theme_bw()
```



7. `geom_area()` - Area plot

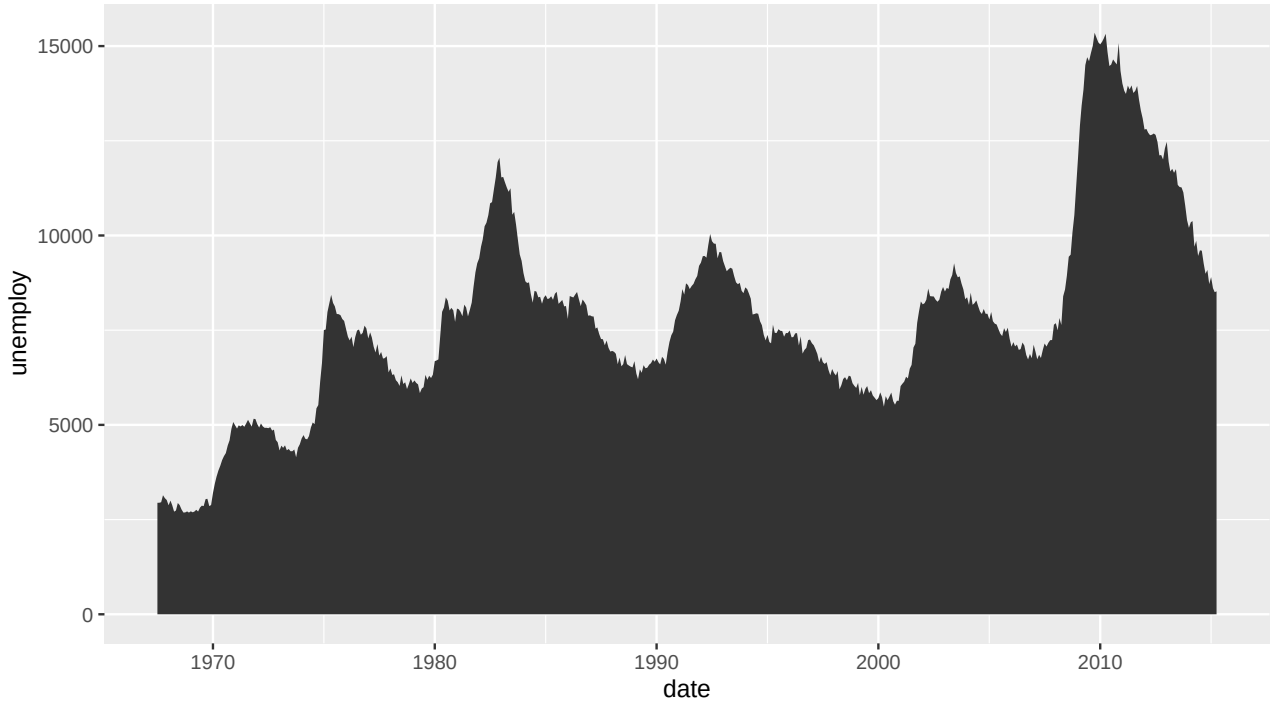
Data Types: Continuous or ordered categorical on the x-axis, numerical on the y-axis.

Uses: Mainly used for showing components of data, especially how they accumulate over time. Area plots are great for visualizing stacked or cumulative data that contribute to a whole, showing how each part contributes to total values.

Business Examples: Area plots are useful for visualizing cumulative totals over time, providing insights into growth trends, and the contribution of different components, such as cumulative sales over time segmented by product type.

Code Example: The area plot show below visualizes unemployment data (**`unemploy`**) from the **`economics`** dataset over time (**`date`**). The area under the line is filled, emphasizing the volume of unemployment over time. This plot is effective for showing trends in data, such as increases or decreases in unemployment rates, and can be particularly impactful for economic reporting.

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_area() fills the area beneath the supplied numerical values along x.
# Here the x-axis is time; the height represents the recorded value rather than a
 ↪  frequency count.
ggplot(economics, aes(x = date, y = unemploy)) +
  geom_area()
```



8. `geom_density()` - Density plot

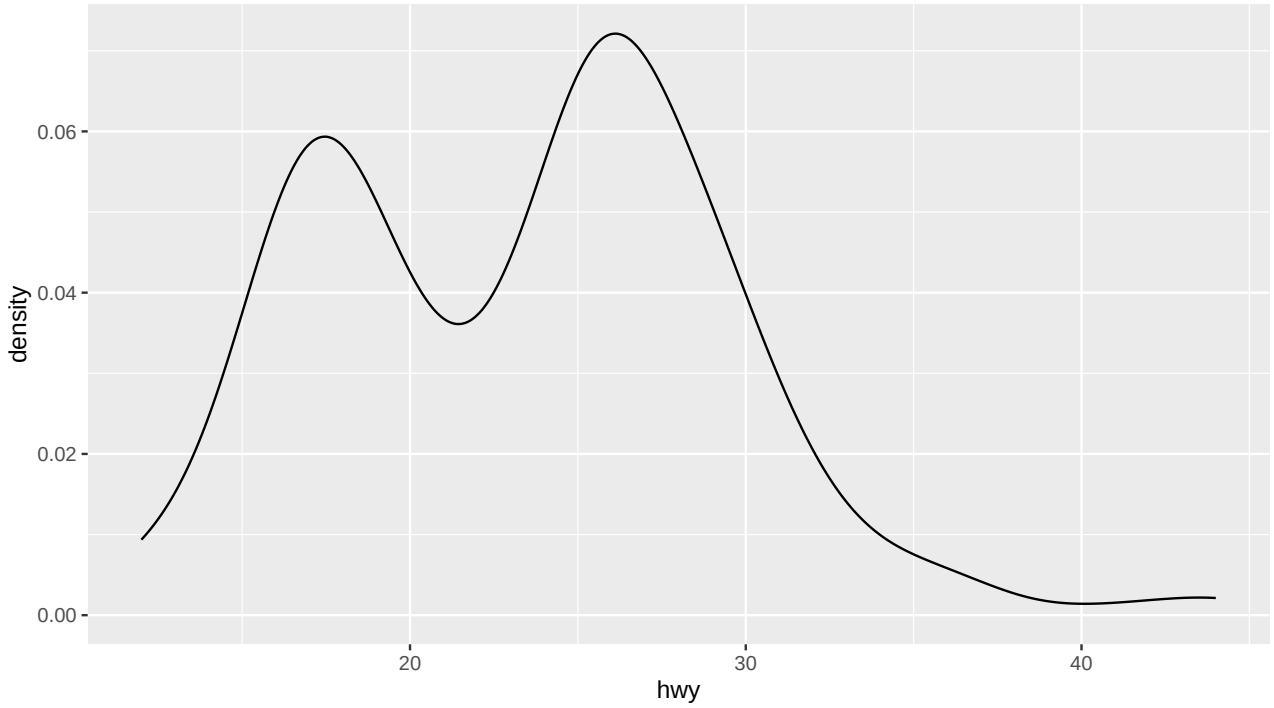
Data Types: Numerical data on the x-axis.

Uses: Ideal for showing distributions of data. Density plots smooth out the frequency of data for a continuous variable, providing a continuous curve that represents the distribution.

Business Examples: Density plots are useful for visualizing the distribution of data where the smoothness of the curve can provide insights into the data's characteristics, such as market analysis for product pricing or understanding income distribution within consumer segments.

Code Example: The density plot is used here to show the distribution of highway miles per gallon (**`hwy`**) in the **`mpg`** dataset. Unlike a histogram, the density plot provides a smooth curve that is useful for identifying where the data concentrates, indicating the most common fuel efficiency values observed on the highway.

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_density() estimates a smooth numerical distribution from the observations.
# The vertical axis is density rather than a raw count; the smoothing choice affects the
 ↪  shape.
ggplot(mpg, aes(x = hwy)) +
  geom_density()
```



9. `geom_violin()` - Violin plot

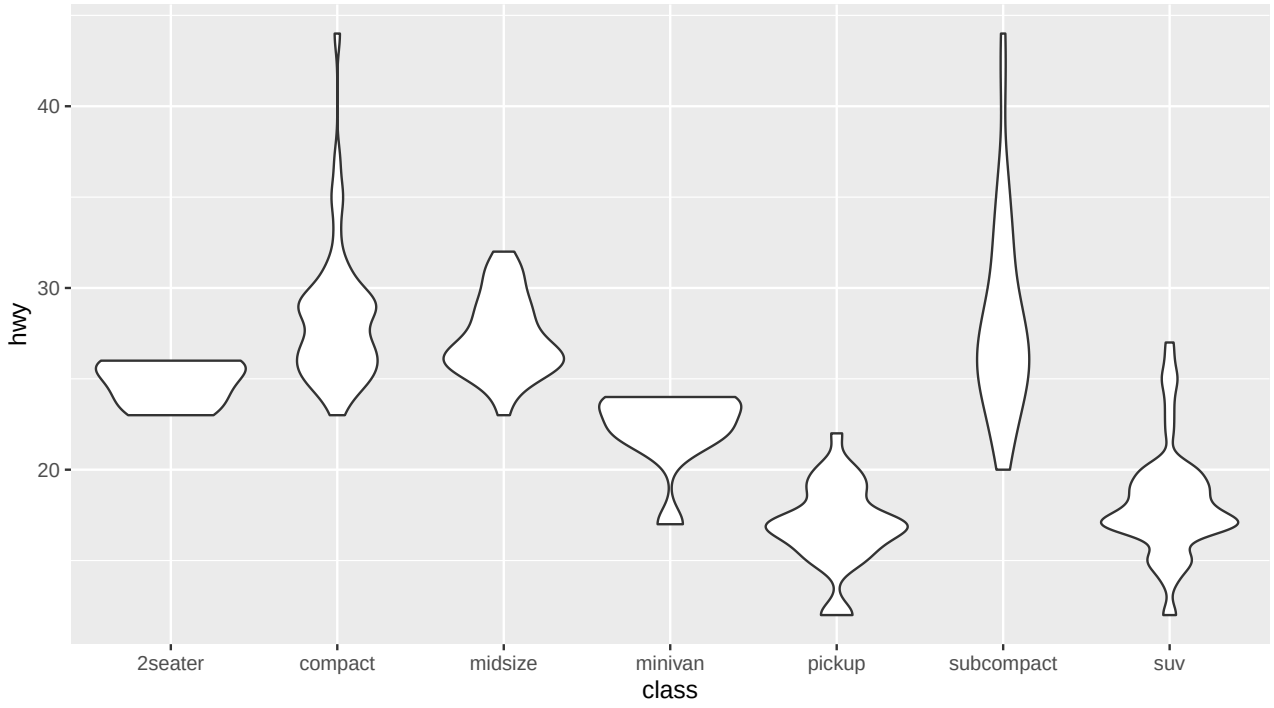
Data Types: Categorical on the x-axis and numerical on the y-axis.

Uses: Suitable for showing distributions and making comparisons. Violin plots are effective at showing the distribution of data within categories and comparing these across different categories.

Business Examples: Violin plots provide rich density estimates visualized similarly to boxplots, and are particularly useful for comparing distributions between groups. They are often used in comparing customer satisfaction scores or product ratings across different categories.

Code Example: This violin plot compares the distribution of highway miles per gallon (**`hwy`**) across different vehicle classes (**`class`**). Violin plots combine boxplot and density plot features, showing both the summary statistics and the density estimates. This helps in comparing how fuel efficiency varies among classes, highlighting differences in distribution shapes and widths.

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_violin() shows a smoothed distribution for each category, mirrored around its
 ↪  center.
# Its shape depends on smoothing and sample size; it does not display every observation
 ↪  individually.
ggplot(mpg, aes(x = class, y = hwy)) +
  geom_violin()
```



10. `geom_col()` - Column chart

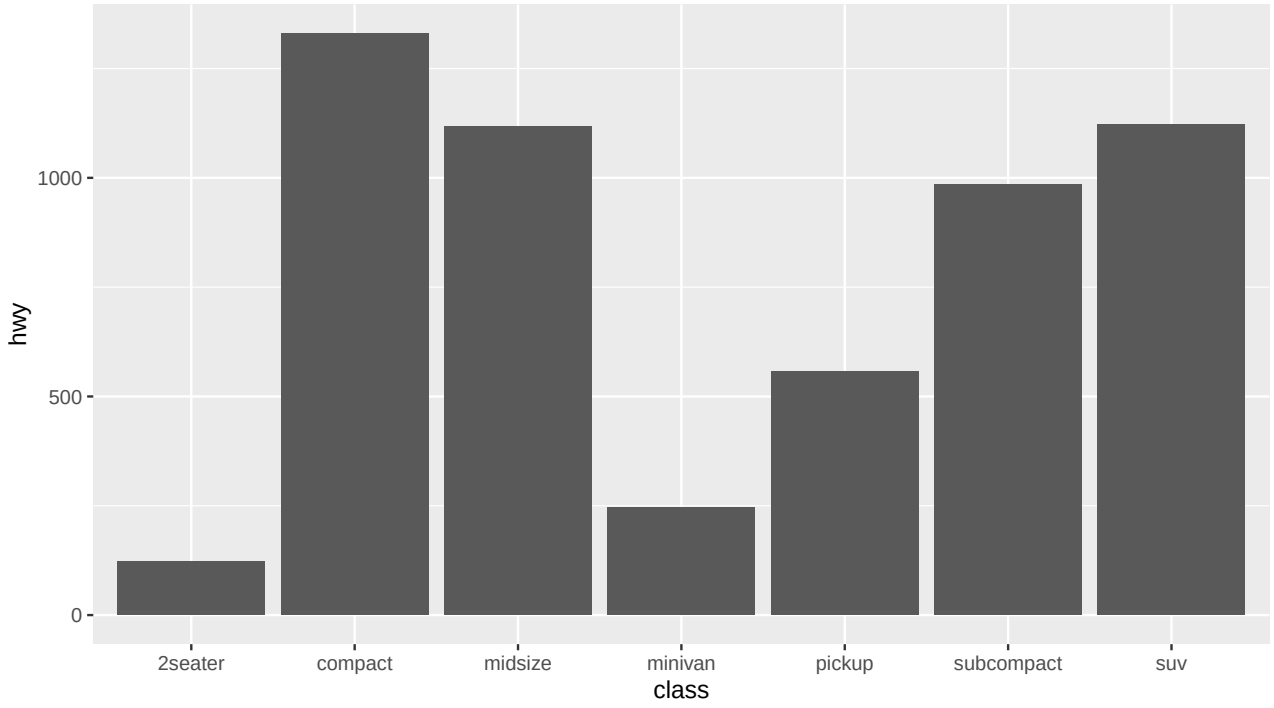
Data Types: Categorical on the x-axis, numerical on the y-axis.

Uses: Best used for making comparisons. Similar to bar charts, column charts display categorical data with the length of each column representing the value, allowing for easy comparison across different categories.

Business Examples: Column charts are used almost identically to bar charts but are particularly effective for emphasizing variation among items. They are commonly used for displaying data such as monthly revenue, sales by category, or inventory levels.

Code Example: The column chart here represents average highway miles per gallon (**hwy**) for different vehicle classes (**class**) in the **mpg** dataset. Each column's height is proportional to the average **hwy** value, allowing for direct comparison across categories. This plot is useful for straightforward visual comparisons and is often used in business contexts to compare averages across different groups or categories.

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_col() draws the supplied hwy heights and stacks records sharing a class.
# This totals the recorded hwy values within a class; calculate group means first if
 ↪  comparing average fuel economy.
ggplot(mpg, aes(x = class, y = hwy)) +
  geom_col()
```



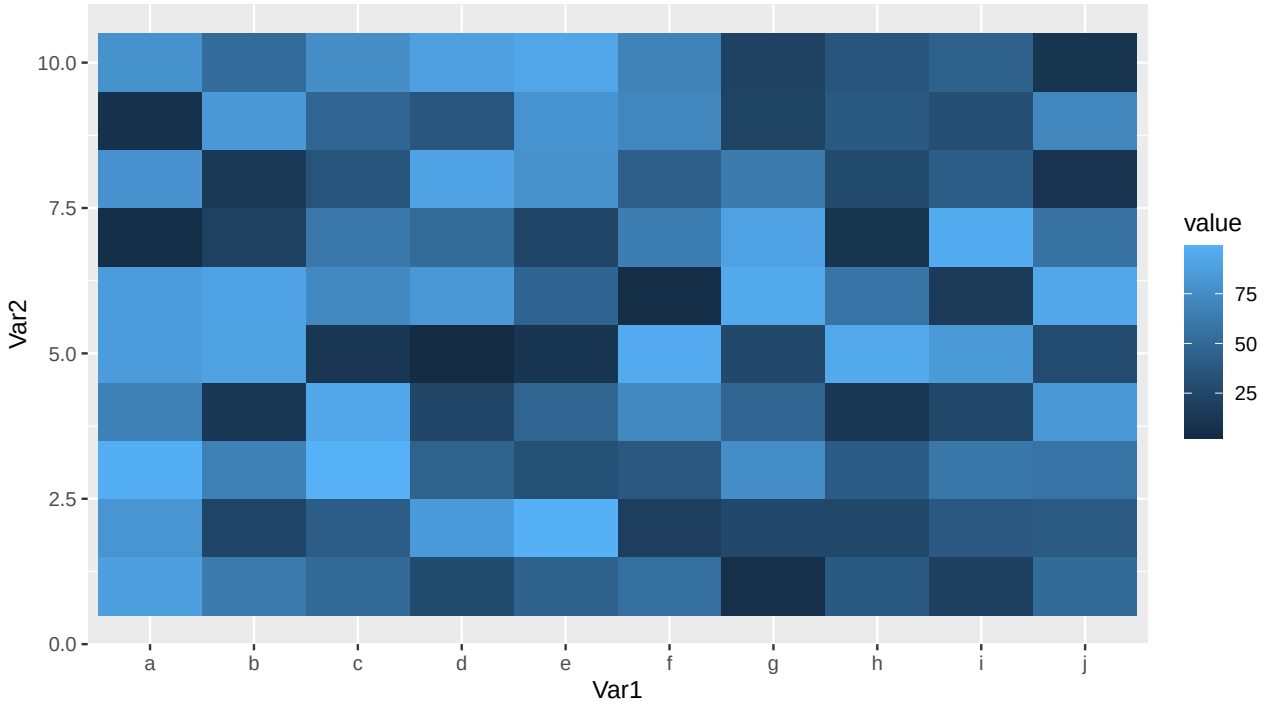
11. `geom_tile()` - Heatmap

Data Types: Categorical or discrete numerical variables on both axes, with a third variable represented by color.

Business Examples: Heatmaps are great for cross-tabulating variables and visualizing the intensity of data, like website traffic by day and hour, sales performance by region and product, or correlation matrices in finance.

Code Example: This heatmap uses an example data frame created with 10 rows and 10 columns labeled by letters and numbers, respectively. The **value** for each tile varies randomly between 0 and 100. In this visualization, each tile's color intensity represents its associated value, providing a quick visual summary of how values are distributed across two dimensions. Heatmaps like this are commonly used in business analytics to display data density across different categories or times, making it easy to spot patterns, concentrations, and outliers.

```
# expand.grid(): constructs every combination of its input values as rows of a data frame.
# runif(): generates artificial uniform draws; min and max set the bounds.
# ggplot(): starts a plot from data; additional layers are joined with +.
# geom_tile() draws one colored tile for each x-y combination supplied in the table.
# aes(fill=value) encodes the numerical value by color; these generated values are
 ↪  artificial.
# Create some example data
data <- expand.grid(Var1 = letters[1:10], Var2 = 1:10)
data$value <- runif(100, min = 0, max = 100)
ggplot(data, aes(x = Var1, y = Var2, fill = value)) +
  geom_tile()
```



12. `geom_text()` and `geom_label()` - Adding text/labels

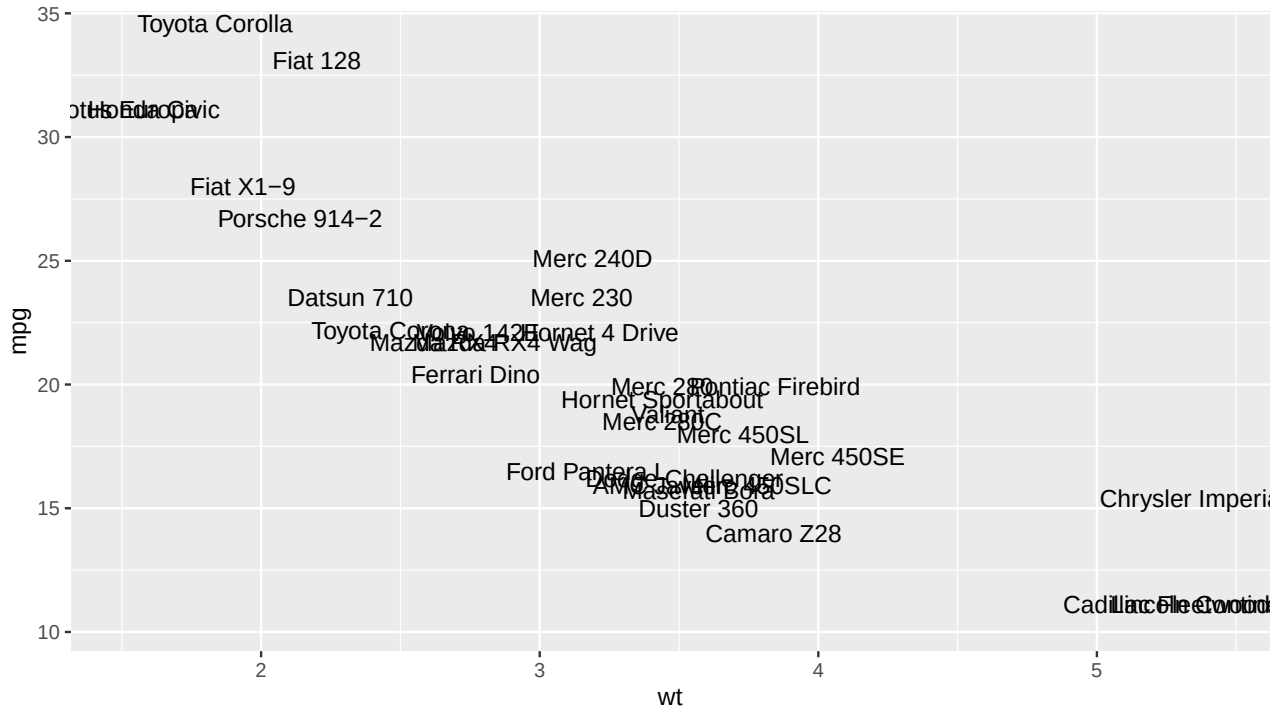
Data Types: Numerical or categorical data on both axes.

Uses: These are auxiliary geoms used to enhance clarity and provide additional information in plots, rather than for showing distributions, comparisons, or components on their own.

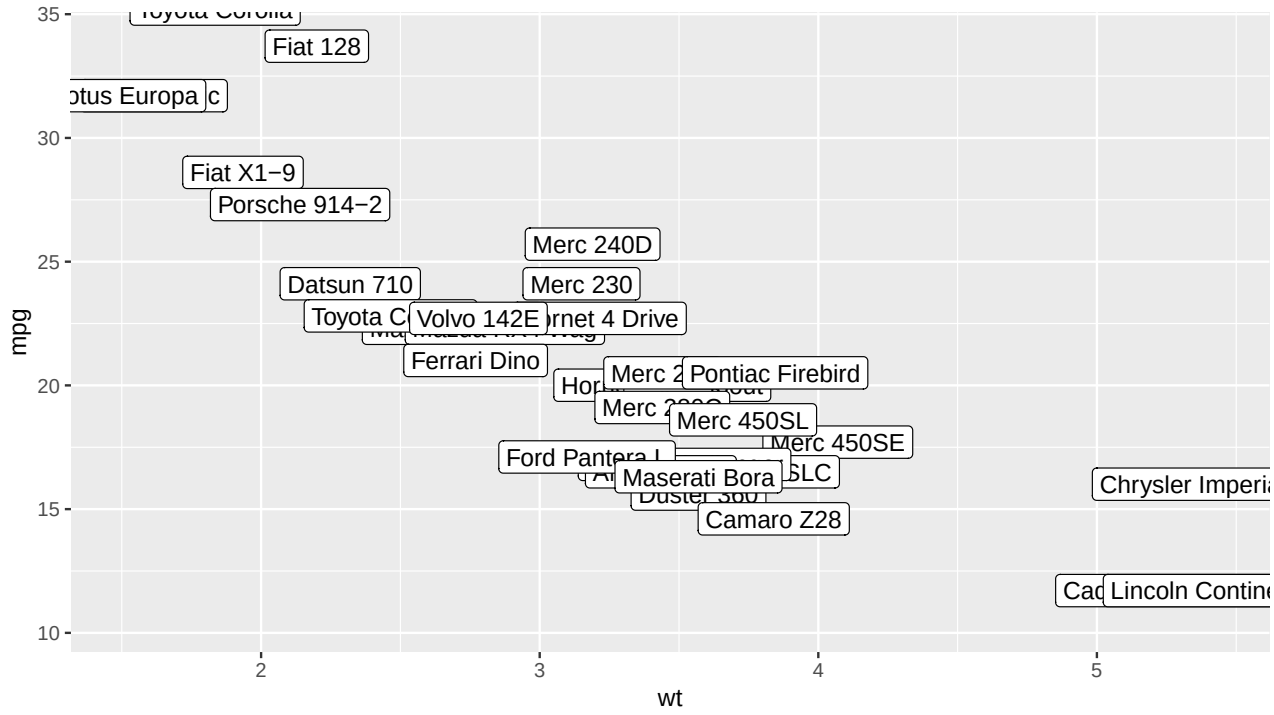
Business Examples: Adding text or labels to plots enhances clarity, particularly in dashboards and presentations where specific data points need to be highlighted or annotated, such as labeling outliers, significant points, or totals directly on charts.

Code Example: Both of these plots use the **`mtcars`** dataset, mapping car weight to miles per gallon, and label each point with the car model. **`geom_text()`** adds labels directly to the plot without any background, which can help in identifying specific points clearly. **`geom_label()`**, however, adds a background to each label, enhancing readability especially in crowded plots. These methods are useful in presentations or detailed reports where clarity in data point identification is crucial.

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# rownames(): reads or replaces the row labels of a matrix or data frame.
# geom_text() or geom_label() places text at coordinates defined through aes().
# The label mapping supplies the text; nudges or justification control its placement.
# Use geom_text for adding text
ggplot(mtcars, aes(x = wt, y = mpg, label = rownames(mtcars))) +
  geom_text(vjust = -0.5)
```




```
# Use geom_label for labels with background
ggplot(mtcars, aes(x = wt, y = mpg, label = rownames(mtcars))) +
  geom_label(vjust = -0.5)
```



13. `geom_ribbon()` - Ribbon plot

Data Types: Continuous or ordered categorical on the x-axis, with a range for the y-axis.

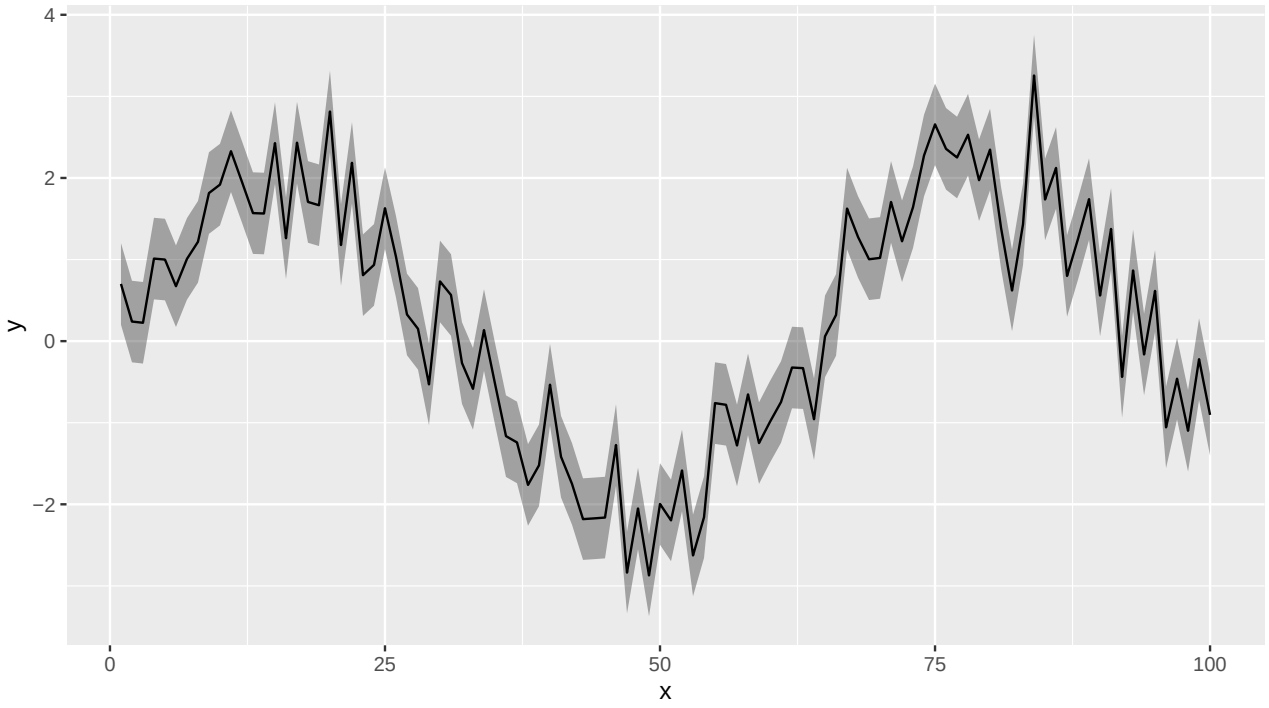
Uses: Used for showing relationships and components, particularly useful in time series to show confidence intervals or prediction ranges around a central measure, thus providing a sense of variability or uncertainty.

Business Examples: Ribbons are often used to indicate uncertainty or variability around a central measure, ideal for financial forecasts, economic data predictions, or displaying confidence intervals in scientific measurements.

Example: The ribbon plot shown below illustrates variability around a central line, created using simulated data. It shows a sinusoidal trend (the central line), with ribbons representing a confidence interval or variability range around this trend. This type of visualization is particularly useful for representing uncertainty in predictions, such as economic forecasts, stock price projections, or any other metric where expressing the confidence of predictions is valuable.

```
# data.frame(): builds a table whose named columns have compatible lengths.
# sin(): computes the sine of an angle measured in radians.
# rnorm(): generates artificial normal draws; n is the draw count and sd is standard
 ↪  deviation.
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_line(): joins observations in x order; group determines which observations form
 ↪  each line.
# geom_ribbon() fills the band between the supplied ymin and ymax values along x.
# The band has the meaning assigned by its input values; a ribbon is not automatically a
 ↪  confidence interval.
# Simulate some data
df <- data.frame(x = 1:100)
df$y <- sin(df$x / 10) * 2 + rnorm(100, sd = 0.5)
df$ymax <- df$y + 0.5
df$ymin <- df$y - 0.5

ggplot(df, aes(x = x, ymin = ymin, ymax = ymax, y = y)) +
  geom_ribbon(alpha = 0.4) +
  geom_line()
```



14. `geom_jitter()` - Jitter plot

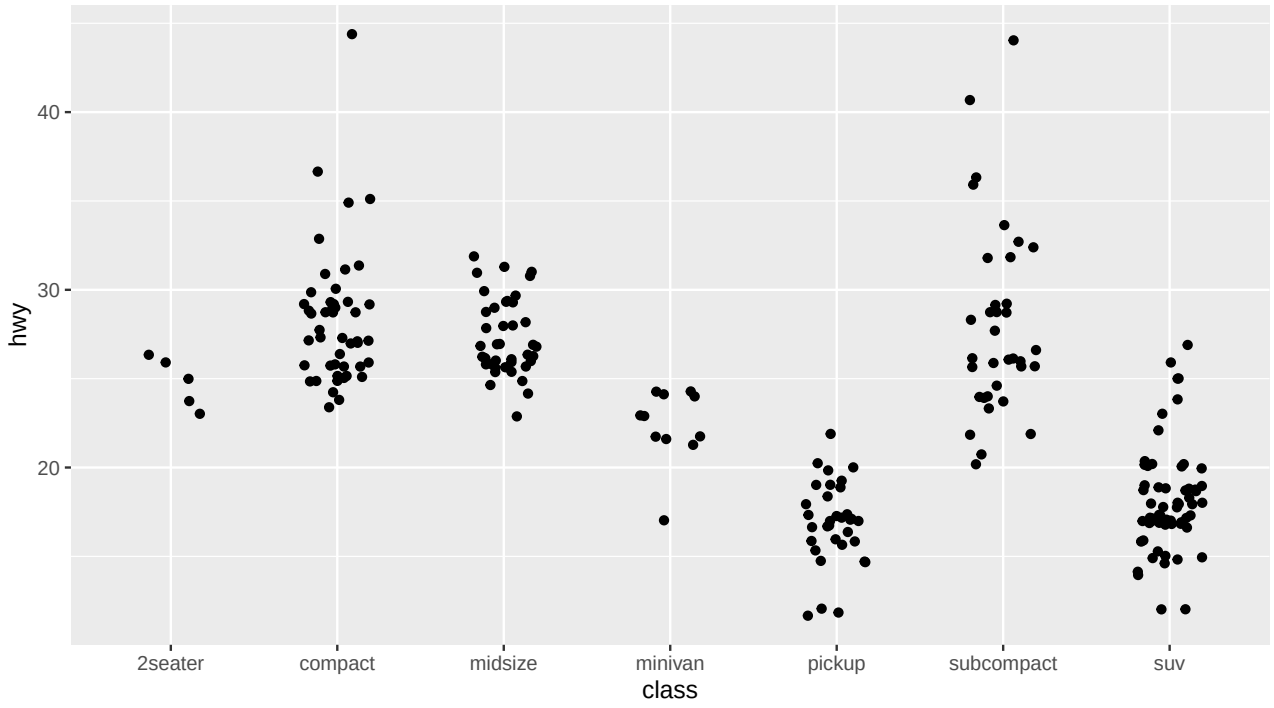
Data Types: Categorical and numerical.

Uses: Primarily enhances scatter plots to show relationships and avoid overplotting, which helps in understanding the distribution of data points across categorical axes, thus aiding in comparisons and distributions indirectly.

Business Examples: Jitter plots help to avoid overplotting when data points overlap, useful in displaying individual transaction data over time or customer ratings where data points may stack vertically.

Code Example: The jitter plot is an adaptation of a scatter plot that adds a small, random displacement to each point. This is particularly useful when dealing with categorical data on one axis (like **`class`** in this example) to avoid overplotting where multiple points have exactly the same position. In the example show below which uses the **`mpg`** dataset, the jitter helps visualize the distribution of highway miles per gallon (**`hwy`**) for different vehicle classes, making it easier to discern individual data points within each category.

```
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_jitter() offsets point positions slightly so overlapping observations become
 ↪  visible.
# width controls horizontal displacement; the jittered positions are display adjustments,
 ↪  not changed source values.
ggplot(mpg, aes(x = class, y = hwy)) +
  geom_jitter(width = 0.2)
```



Each of these examples is tailored to show we how different types of visual representations can be achieved using `ggplot2` and R's built-in datasets. They illustrate how versatile and powerful `ggplot2` can be for data visualization.

15. `geom_point()` - Bubble plot

Uses: Bubble plots are an extension of scatter plots, allowing we to represent three dimensions of data. Each point on the plot has coordinates (x and y), and the size of each bubble (point) represents the third dimension. This type of plot is used for showing relationships between numerical data points and can also visually represent the magnitude of the data.

Business Examples: Bubble plots are particularly useful in business for visualizing complex relationships that include sizes or volumes, which can't be effectively captured with traditional scatter plots. For instance, a company might use a bubble plot to visualize the relationship between advertising spend and sales, with bubble size representing the number of sales transactions. This can provide insights into the efficiency of advertising spend per transaction volume. They are

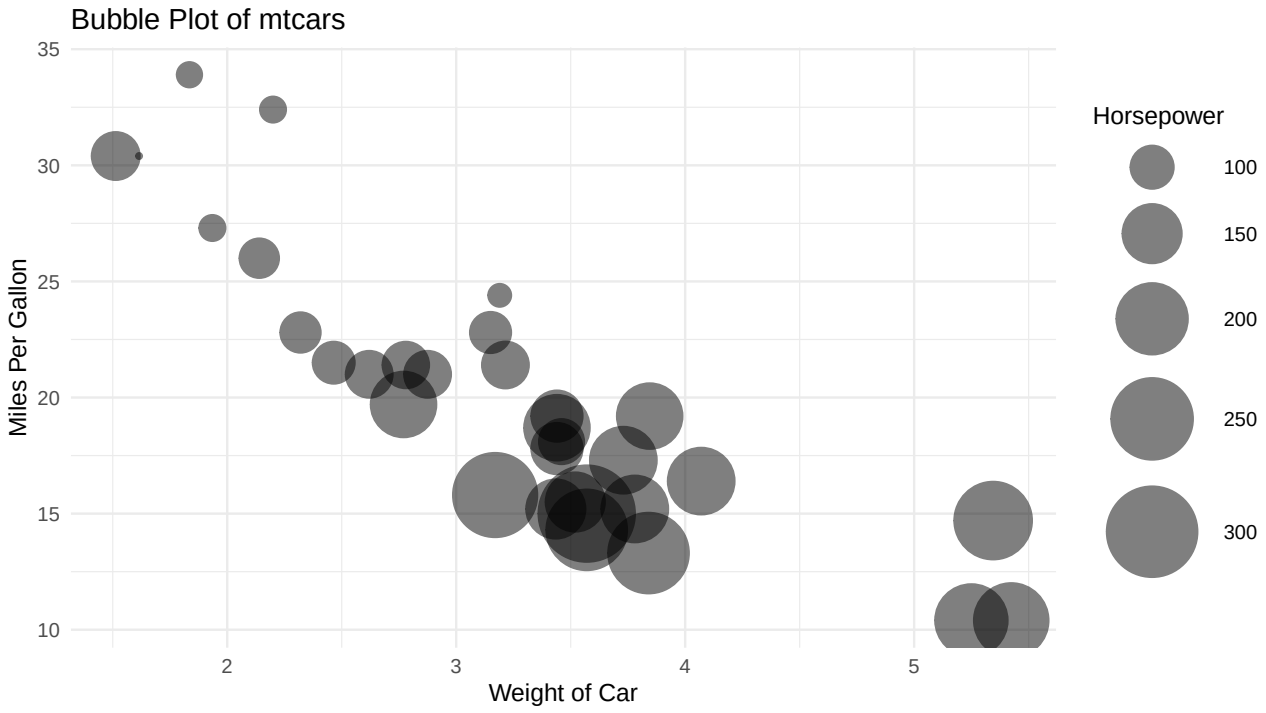
also used to compare and visualize financial metrics across different business units, regions, or product categories where the bubble size might represent profit margin, customer base size, or market potential.

Code Example: For this example, we'll use the `mtcars` dataset that comes built into R. We'll plot the relationship between `wt` (car weight) and `mpg` (miles per gallon) with the bubble size representing hp (horsepower).

```
# library(): loads a package so its functions can be used in this session.
# data(): loads a named dataset supplied by an installed package.
# ggplot(): starts a plot from data; additional layers are joined with +.
# aes(): maps variables to visual properties such as x, y, fill, color, or group.
# geom_point(): draws a point for each row at its mapped x and y values.
# scale_size_continuous(): maps numeric values to symbol sizes over the chosen range.
# c(): combines values into a vector, converting them to a common type when needed.
# labs(): sets axis titles, legend titles, plot titles, and captions.
# theme_minimal(): uses a minimal chart style; base_size controls the base text size in
 ↪  points.
# The size mapping encodes an additional numerical variable using point area.
# alpha controls transparency so overlapping bubbles can still be seen.
library(ggplot2)

# Load the mtcars dataset
data(mtcars)

# Create a bubble plot
ggplot(mtcars, aes(x = wt, y = mpg, size = hp)) +
  geom_point(alpha = 0.5) +  # Set transparency to see overlapping points
  scale_size_continuous(range = c(1, 20)) +  # Adjust the range of bubble sizes
  labs(title = "Bubble Plot of mtcars",
       x = "Weight of Car",
       y = "Miles Per Gallon",
       size = "Horsepower") +
  theme_minimal()
```



Explanation of the Code:

- `aes(x = wt, y = mpg, size = hp)`: This sets the aesthetics for the plot, with car weight on the x-axis, miles per gallon on the y-axis, and bubble size proportional to horsepower.

- `geom_point(alpha = 0.5)`: This creates the bubbles, with `alpha = 0.5` providing some transparency so that overlapping points can be viewed more easily.

- `scale_size_continuous(range = c(1, 20))`: This controls the range of the bubble sizes, making it clear how they correspond to horsepower.

- `labs(...)`: Adds labels for the axes and the legend.

- `theme_minimal()`: Applies a minimalistic theme for a cleaner look.

This bubble plot will visually demonstrate how weight and fuel efficiency interact and how horsepower varies within those parameters, providing a multi-dimensional analysis in a single plot, ideal for strategic planning or operational analysis in business contexts.

Module 1: Basic Concepts — Practice

Lesson 2 | Apply the R fundamentals

Exercise 1: Greeting Script in R

Question A: Construct an R script that outputs the greeting “Hello, World!” Then, alter the script to replace “World” with your own name.

Solution A:

Question B: Utilize the `cat()` function to display “Hello, World!” in the console. Observe and report any variations in the output compared to the `print()` function.

Solution B:

Exercise 2: Storing Personal Favorites

Store your preferred number, color, and a boolean value that signifies your liking for pizza. Display these stored values.

Solution:

Exercise 3: Types and Collections

Question A: Generate a vector with the numbers from 1 to 10. Determine and document the vector’s type as well as the types of the elements within it.

Solution A:

Question B: Form a list holding a numeric value, a character string, and a boolean value. Ascertain the list’s type and the types of its individual elements.

Solution B:

Exercise 4: Checking Number Sign

Compose a script that discerns whether a given number (hard-coded in the script) is positive, negative, or zero, and relays an appropriate message.

Solution:

Exercise 5: Iterative Loops

Question A: Write a `for` loop that computes and exhibits the squares of numbers from 1 to 10.

Solution A:

Question B: Implement a `while` loop to effectuate a countdown from 10 to 1, displaying each countdown step.

Solution B:

Exercise 6: Area Calculation Function

Craft a function named `calculate_area` that accepts a circle's radius and returns its area. Check the function's accuracy with a variety of radii.

Solution:

Exercise 7: Vector Mathematics

Question A: Form two vectors, a and b, each with 5 random numbers. Execute and print the results of element-wise addition, subtraction, multiplication, and division.

Solution A:

Question B: Use comparison operators to compare the elements of a and b and communicate the outcomes.

Solution B:

Exercise 8: Working with Factors

Institute a factor variable that delineates three kinds of fruit. Illustrate the factor variable, its levels, and transmute it into numerical values.

Solution:

Exercise 9: Data Frame Adjustments

Forge a modest data frame that enlists student IDs, names, and scores. Accomplish these tasks:

- Select and unveil only the students' names and scores.
- Incorporate a column indicating if the student passed (score >= 50).

Solution:

Exercise 10: Interactive User Query (Optional)

Write code that asks the user to enter his name and age, then returns a message along with a mention of whether they are above the age of 18.

Solution:

Home Work

Scenario: You are a Data Scientist working for a consulting firm. One of your colleagues from the Auditing department has asked you to help them assess the financial statement of organization X.

You have been supplied with two vectors of data: monthly revenue and monthly expenses for the financial year in question. Your task is to calculate the following financial metrics:

1. **Profit for each month**: Calculate the profit for each month by subtracting expenses from revenue.
2. **Profit after tax for each month**: The tax rate is 30%. Calculate the profit after tax for each month.
3. **Profit margin for each month**: The profit margin is equal to the profit after tax divided by revenue, expressed as a percentage.
4. **Good months**: Identify the months where the profit after tax was greater than the mean profit after tax for the year.
5. **Bad months**: Identify the months where the profit after tax was less than the mean profit after tax for the year.
6. **The best month**: Identify the month where the profit after tax was maximum for the year.
7. **The worst month**: Identify the month where the profit after tax was minimum for the year.

Additional Requirements:

- All results need to be presented as vectors.
- Results for dollar values need to be calculated with $0.01 precision but presented in units of $1,000 (i.e., 1k) with no decimal points.
- Results for the profit margin ratio need to be presented in units of percentage with no decimal points.
- Note: It is okay for tax for any given month to be negative (in accounting terms, negative tax translates into a deferred tax asset).

```
# c() combines compatible values into a vector in the supplied order.
# Assignments save the vector under a name; printing that name displays its current
 ↪  values.
# Given data
revenue <- c(14574.49, 7606.46, 8611.41, 9175.41, 8058.65, 8105.44, 11496.28,
 ↪  9766.09, 10305.32, 14379.96, 10713.97, 15433.50)
expenses <- c(12051.82, 5695.07, 12319.20, 12089.72, 8658.57, 840.20, 3285.73,
 ↪  5821.12, 6976.93, 16618.61, 10054.37, 3803.96)
```

Module 1: dplyr Practice

Lesson 3 | Work with the Restaurant CSV

Table of contents

Preparation 1
Tasks 1

Preparation

Read Lesson 3. Load `data/module02/restaurant_ratings.csv` from the course project. Preserve the original values and ordered quality categories. Use pipes and save each requested result under a descriptive name.

Tasks

1. Inspect column types, row count, missing values, and duplicate restaurant identifiers. State what one row means.
2. Select restaurant ID, quality, and price. Retain Excellent restaurants priced at least USD 30. How many rows remain?
3. Return the five highest-priced restaurants, ordering tied prices by restaurant ID.
4. Create four price bands [10,20), [20,30), [30,40), [40,50), and a logical indicator for price at least USD 30. Verify that no price is left unclassified.
5. Count restaurants by quality category. Calculate group size, mean price, and median price using `group_by()` and `summarise()`. Return an ungrouped result.
6. Create a lookup containing one median price per quality group, then left-join it to the restaurant rows. Verify the row count and explain why duplicate lookup keys could change it.
7. Reshape the mean and median summary columns into a long table with `measure` and `price_usd` columns. Reshape it back to wide form.
8. Write a brief explanation of why repeated prices are not necessarily duplicate observations, and why ignoring missingness can mislead an analysis.

Return to Module 1.

Module 1: ggplot2 Practice

Lesson 4 | Visualize the textbook CSVs

Table of contents

Preparation 1
Tasks 1

Preparation

Read Lesson 4. Use the checked CSVs in `data/module02/`. The data guide explains their variables and units. These exercises focus on constructing clear plots; Module 2 develops their statistical interpretation.

Tasks

1. Load Restaurant. Create a count bar chart for its quality ratings, ordered Good, Very Good, Excellent. Explain why `geom_bar()` is appropriate for raw rows.
2. Summarize those counts into a separate table, then reproduce the chart with `geom_col()`. Explain the difference between the two functions.
3. Load Audit. Draw a histogram of `audit_days` with bin width 5 and lower boundary 10. Label both axes and state what a bar represents.
4. Load Electronics. Draw a scatterplot of `commercials` against `sales_hundreds_usd`. Label the sales units explicitly. Add week labels without connecting the points.
5. Convert sales to USD in a new column and redraw the plot with corrected labels. Explain what changes and what remains the same visually.
6. Improve one chart using an informative title, a readable theme, and purposeful color. Explain why a continuous color gradient would be unsuitable for unordered brand categories.
7. Save one plot manually under `_output/` using `ggsave()`. Keep source data unchanged.

Avoid causal claims based on a chart alone. Use the shared course stylesheet for the report title; choose chart colors to communicate the data.

Return to Module 1.